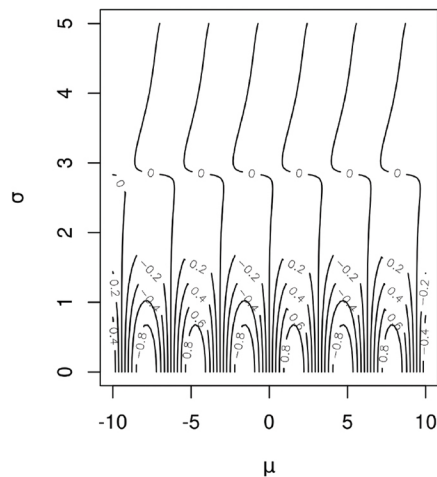
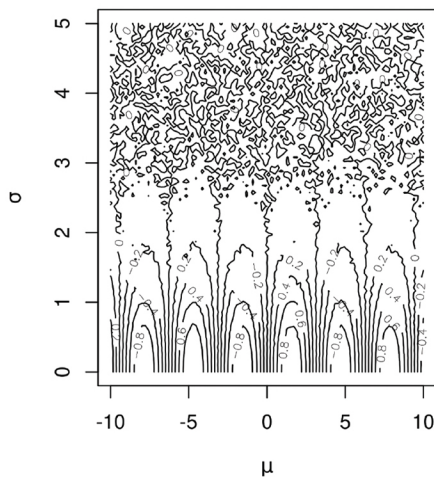


An Introduction to R and Python For Data Analysis

A Side-By-Side Approach



Taylor R. Brown

An Introduction to R and Python For Data Analysis

An Introduction to R and Python For Data Analysis helps teach students to code in both R and Python simultaneously. As both R and Python can be used in similar manners, it is useful and efficient to learn both at the same time, helping lecturers and students to teach and learn more, save time, whilst reinforcing the shared concepts and differences of the systems. This tandem learning is highly useful for students, helping them to become literate in both languages, and develop skills that will be handy after their studies. This book presumes no prior experience with computing and is intended to be used by students from a variety of backgrounds. The side-by-side formatting of this book helps introductory graduate students quickly grasp the basics of R and Python, with the exercises providing helping them to teach themselves the skills they will need upon the completion of their course, as employers now ask for competency in both R and Python. Teachers and lecturers will also find this book useful in their teaching, providing a singular work to help ensure their students are well trained in both computer languages. All data for exercises can be found here: https://github.com/tbrown122387/r_and_python_book/tree/master/data.

Key Features:

- Teaches R and Python in a “side-by-side” way
- Examples are tailored to aspiring data scientists and statisticians, not software engineers
- Designed for introductory graduate students
- Does not assume any mathematical background



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

An Introduction to R and Python For Data Analysis

A Side-By-Side Approach

Taylor R. Brown



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

Designed cover image: © Taylor R. Brown

MATLAB® is a trademark of The MathWorks, Inc. and is used with permission. The MathWorks does not warrant the accuracy of the text or exercises in this book. This book's use or discussion of MATLAB® software or related products does not constitute endorsement or sponsorship by The MathWorks of a particular pedagogical approach or particular use of the MATLAB® software.

First edition published 2023

by CRC Press

6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press

4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2023 Taylor R. Brown

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

ISBN: 978-1-032-20325-6 (hbk)

ISBN: 978-1-032-20338-6 (pbk)

ISBN: 978-1-003-26324-1 (ebk)

DOI: [10.1201/9781003263241](https://doi.org/10.1201/9781003263241)

Typeset in Latin Modern font

by KnowledgeWorks Global Ltd.

Publisher's note: This book has been prepared from camera-ready copy provided by the authors.

Access the companion website: https://github.com/tbrown122387/r_and_python_book/tree/master/data

To Clare



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

List of Figures	xiii
Welcome	xv
Preface	xvii
I Introducing the Basics	1
1 Introduction	3
1.1 Hello World in R	3
1.2 Hello World in Python	6
1.3 Getting Help	9
1.3.1 Reading Documentation	9
1.3.2 Understanding File Paths	9
2 Basic Types	11
2.1 Basic Types in Python	11
2.1.1 Type Conversions in Python	12
2.2 Basic Types in R	13
2.2.1 Type Conversions in R	13
2.2.2 R's Simplification	14
2.3 Exercises	14
2.3.1 R Questions	14
2.3.2 Python Questions	15
3 R <i>vectors</i> versus Numpy <i>Arrays</i> and Pandas' <i>Series</i>	17
3.1 Overview of R	17
3.2 Overview of Python	18
3.3 Vectorization in R	19
3.4 Vectorization in Python	20
3.5 Indexing Vectors in R	24
3.6 Indexing Numpy arrays	24
3.7 Indexing Pandas' Series	25
3.8 Some Gotchas	27

3.8.1	Shallow versus Deep Copies	27
3.8.2	How R and Python Handle Missing Values	30
3.9	An Introduction to Regular Expressions	32
3.9.1	Literal Characters versus Metacharacters	33
3.9.2	The Trouble with Backslashes: Escape Sequences	35
3.9.3	More Examples of Using Regular Expressions	36
3.10	Exercises	39
3.10.1	R Questions	39
3.10.2	Python Questions	42
4	Numpy ndarrays versus R's Matrix and Array Types	47
4.1	Numpy ndarrays in Python	47
4.2	The Matrix and Array Classes in R	48
4.3	Exercises	52
4.3.1	R Questions	52
4.3.2	Python Questions	54
5	R's lists versus Python's lists and dicts	59
5.1	lists in R	59
5.2	lists in Python	60
5.3	Dictionaries in Python	61
5.4	Exercises	63
5.4.1	R Questions	63
5.4.2	Python Questions	63
6	Functions	65
6.1	Defining R Functions	65
6.2	Defining Python Functions	66
6.3	More Details on R's User-Defined Functions	67
6.4	More Details on Python's User-Defined Functions	69
6.5	Function Scope in R	70
6.6	Function Scope in Python	73
6.7	Modifying a Function's Arguments	75
6.7.1	Passing by Value in R	75
6.7.2	Passing by Assignment in Python	76
6.8	Accessing and Modifying Captured Variables	79
6.8.1	Accessing Captured Variables in R	79
6.8.2	Accessing Captured Variables in Python	81
6.8.3	Modifying Captured Variables in R	81
6.8.4	Modifying Captured Variables in Python	82
6.9	Exercises	83
6.9.1	R Questions	83
6.9.2	Python Questions	84

7	Categorical Data	87
7.1	factors in R	87
7.2	Two Options for Categorical Data in Pandas	90
7.3	Exercises	94
7.3.1	R Questions	94
7.3.2	Python Questions	94
8	Data Frames	97
8.1	Data Frames in R	97
8.2	Data Frames in Python	100
8.3	Exercises	105
8.3.1	R Questions	105
8.3.2	Python Questions	107
II	Common Tasks and Patterns	111
9	Input and Output	113
9.1	General Input Considerations	113
9.2	Reading in Text Files with R	116
9.3	Reading in Text Files with Pandas	118
9.4	Saving Data in R	121
9.4.1	Writing Out Tabular Plain Text Data in R	122
9.4.2	Serialization in R	122
9.5	Saving Data in Python	124
9.5.1	Writing Out Tabular Plain Text Data in Python	124
9.5.2	Serialization in Python	124
9.6	Exercises	125
9.6.1	R Questions	125
9.6.2	Python Questions	126
10	Using Third-Party Code	127
10.1	Installing Packages in R	127
10.2	Installing Packages in Python	127
10.3	Loading Packages in R	128
10.4	Loading Packages in Python	130
10.4.1	Importing Examples	132
10.5	Exercises	134
11	Control Flow	137
11.1	Conditional Logic	137
11.2	Loops	138
11.3	Exercises	142

11.3.1	R Questions	142
11.3.2	Python Questions	145
12	Reshaping and Combining Data Sets	149
12.1	Ordering and Sorting Data	149
12.2	Stacking Data Sets and Placing Them Shoulder to Shoulder	151
12.3	Merging or Joining Data Sets	153
12.4	Long versus Wide Data	156
12.4.1	Long versus Wide in R	156
12.4.2	Long versus Wide in Python	159
12.5	Exercises	161
12.5.1	R Questions	161
12.5.2	Python Questions	164
13	Visualization	167
13.1	Base R Plotting	167
13.2	Plotting with <code>ggplot2</code>	170
13.3	Plotting with <code>Matplotlib</code>	175
13.4	Plotting with <code>Pandas</code>	179
13.5	Exercises	180
13.5.1	R Questions	180
13.5.2	Python Questions	181
III	Programming Styles	183
14	An Introduction to Object-Oriented Programming	185
14.1	OOP in Python	186
14.1.1	Overview	186
14.1.2	A First Example	187
14.1.3	Adding Inheritance	190
14.1.4	Adding in Composition	192
14.2	OOP in R	194
14.2.1	S3 Objects: The Big Picture	194
14.2.2	Using S3 Objects	195
14.2.3	Creating S3 Objects	199
14.2.4	S4 Objects: The Big Picture	200
14.2.5	Using S4 Objects	200
14.2.6	Creating S4 Objects	201
14.2.7	Reference Classes: The Big Picture	204
14.2.8	Creating Reference Classes	204
14.2.9	Creating R6 Classes	205
14.3	Exercises	206

14.3.1	Python Questions	206
14.3.2	R Questions	209
15	An Introduction to Functional Programming	213
15.1	Functions as Function Inputs in R	215
15.1.1	<code>sapply()</code> and <code>vapply()</code>	215
15.1.2	<code>lapply()</code>	216
15.1.3	<code>apply()</code>	217
15.1.4	<code>tapply()</code>	219
15.1.5	<code>mapply()</code>	220
15.1.6	<code>Reduce()</code> and <code>do.call()</code>	221
15.2	Functions as Function Inputs in Python	223
15.2.1	Functions as Function Inputs in Base Python	223
15.2.2	Functions as Function Inputs in Numpy	224
15.2.3	Functional Methods in Pandas	225
15.3	Functions as Function Outputs in R	228
15.4	Functions as Function Outputs in Python	233
15.4.1	Writing Our Own Decorators	234
15.5	Exercises	236
15.5.1	Python Questions	236
15.5.2	R Questions	238
	Bibliography	241
	Index	245



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

List of Figures

1.1	RStudio.	4
1.2	Anaconda navigator.	6
1.3	Spyder.	7
3.1	An indicator function.	45
3.2	The marginal density of Y.	46
10.1	The environment window in RStudio.	129
11.1	Visualizing our three functions.	143
13.1	A simple histogram.	168
13.2	Some scatterplots.	169
13.3	A second scatterplot.	172
13.4	Adding some color.	173
13.5	Changing scales.	173
13.6	Changing the fill.	174
13.7	Changing the fill again.	175
13.8	Another simple histogram.	176
13.9	Side-by-side line plots in matplotlib.	178
14.1	A scatterplot matrix.	197
14.2	Plotting an empirical cumulative distribution function.	198
15.1	Our spline function.	224
15.2	Monte Carlo approximations versus exact evaluations.	230
15.3	Monte Carlo: With and without common random numbers.	231



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Welcome

Teaching a Course With This Textbook

You will notice that some of the exercise questions are unusually specific. For example, they will ask the student to assign the answer in a certain form to a variable with a very specific name. This is because they are written with automatic grading in mind.

All of the exercises in this text have been very generously “battle tested” by the Fall 2021 STAT 5430 class at the University of Virginia.

License(s)

The textbook is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. The code used to generate the text is licensed under a Creative Commons Zero v1.0 Universal license.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

About this book

This book is written to be used in a computing class that teaches both R and Python to graduate students in a statistics or data science department. This book is written for students that do not necessarily possess any previous familiarity with writing code.

- If you are using them for analyzing data, R and Python do a lot of the same things in pretty similar ways, so it does not always make sense to either a.) teach one language after the other, or b.) choose one over the other. The side-by-side approach helps to teach/learn more, save time, and reinforce shared concepts and key differences.
- This text does not describe statistical modeling techniques in detail, but many exercises will ask students to implement them. These exercises will not assume mathematical familiarity. The objective is to test programming ability, and an added benefit is that the work might be useful later in their careers.
- This book is written for aspiring data scientists, not necessarily aspiring software developers. Why do I draw the distinction? When discussing different types, for example, I do not discuss data structures in any depth. Rather, I discuss examples of applications where different types would be most useful.
- Generally speaking, chapters should be read in order, but occasionally skipping ahead can be beneficial. Later chapters are more may assume some familiarity with previous chapters. Also, within a chapter's section, sometimes a discussion for a particular topic in Python, say, might follow a discussion about the same topic in R. In this case, the later section will often assume that the previous section has been read first.

Conventions

Sometimes R and Python code look very similar, or even identical. This is why I usually separate R and Python code into separate sections. However, sometimes I do not, so whenever it is necessary to prevent confusion, I will remind you what language is being used in comments (more about comments in 1.2).

```
# in python
print('hello world')
## hello world
```

```
# in R
print('hello world')
## [1] "hello world"
```

Installing the Required Software

To get started, you must install both R and Python. The installation process depends on what kind of machine you have (e.g. what type of operating system your machine is running, is your processor 32 or 64 bit, etc.).

Below, I suggest running R with RStudio, and Python with Anaconda, and I provide some helpful links. I suggest downloading these two bundles separately; however, I should note that the recommendation below is not the only installation method. For example: - one can run R and Python without downloading RStudio or Anaconda, - one can install RStudio with Anaconda, - one can run Python from within Rstudio, - one can run Python from within Rstudio that is managed by Anaconda, etc., and - options and procedures are very likely to change over time.

Instructors can prefer alternative strategies, if they wish. If they do, they should verify that Python's version is ≥ 3.6 , and R's is $\geq 4.0.0$. If so, all the code in this book should run.

Installing R (and RStudio)

It is recommended that you install R and *RStudio Desktop*. *RStudio Desktop* is a graphical user interface with many tools that making writing R easier and more fun.

Install R from the Comprehensive R Archive Network (CRAN). You can access instructions for your specific machine by clicking here.¹

You can get RStudio Desktop directly from the company's website².

Installing Python by Installing Anaconda

It is recommended that you install *Anaconda*, which is a package manager, environment manager, and Python distribution with many third party open source packages. It provides a graphical user interface for us, too, just as RStudio does. You can access instructions for your specific machine and OS by clicking here³.

¹<https://cran.r-project.org/>

²<https://www.rstudio.com/products/rstudio/download/#download>

³<https://docs.anaconda.com/anaconda/install/#>



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part I

Introducing the Basics



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

1

Introduction

Now that you have both R and Python installed, we can get started by taking a tour of our two different **integrated development environments** (IDEs), RStudio and Spyder.

In addition, I will also discuss a few topics superficially, so that we can get our feet wet:

- printing,
- creating variables, and
- calling functions.

1.1 Hello World in R

Go ahead and open up RStudio. It should look something like this:

I changed my “Editor Theme” from the default to “Cobalt” because it’s easier on my eyes. If you are opening RStudio for the first time, you probably see a lot more white. You can play around with the theme, if you wish, after going to `Tools -> Global Options -> Appearance`.

The **console**, which is located by default on the lower left panel, is the place where all of your code gets run. For short one-liners, you can type the code directly into the console. Try typing the following code in there. Here we are making use of the `print()` function.

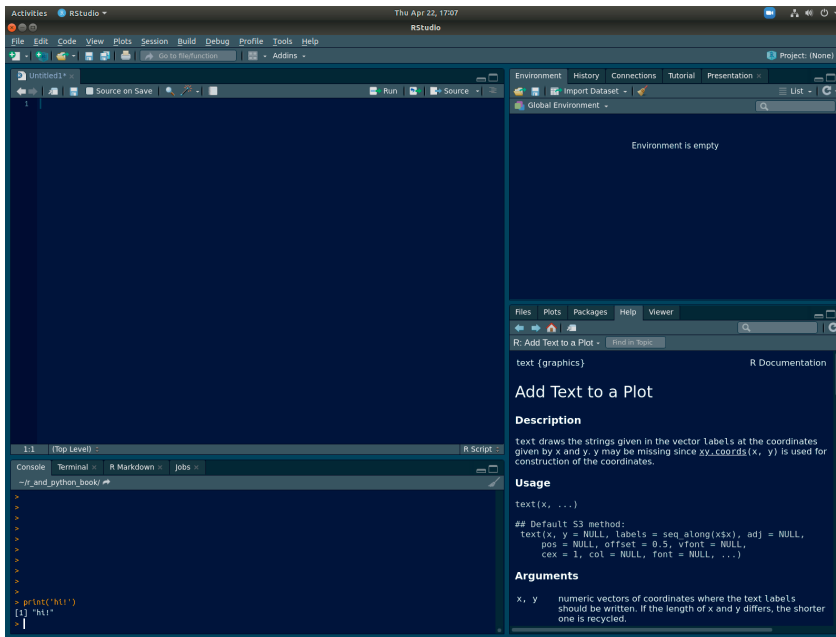


FIGURE 1.1: RStudio.



In R, functions are “first-class objects,” which means you can refer to the name of a function without asking it to do anything. However, when we *do* want to use it, we put parentheses after the name. This is called **calling** the function or **invoking** the function. If a function call takes any **arguments** (aka inputs), then the programmer supplies them between the two parentheses. A function may **return** values to be subsequently used, or it may just produce a “side-effect” such as printing some text, displaying a chart, or reading/writing information to an external data source.

```
print('hello R world')
## [1] "hello R world"
```

During the semester, we will write more complicated code. Complicated code is usually written incrementally and stored in a text file called a **script**. Click File -> New File -> R Script to create a new script. It should appear at the top left of the RStudio window (see Figure 1.1). After that, copy and paste the following code into your script window:

```
print('hello world')
print("this program")
```

```
print('is not incredibly interesting')
print('but it would be a pain')
print('to type it all directly into the console')
myName <- "Taylor"
print(myName)
```

This script will run five print statements and then create a variable called `myName`. The print statements are of no use to the computer and will not affect how the program runs. They just display messages to the human running the code.

The variable created on the last line is more important because it is used by the computer, and so it can affect how the program runs. The operator `<-` is the **assignment operator**¹. It takes the character constant `"Taylor"`, which is on the right, and stores it under the name `myName`. If we added lines to this program, we could refer to the variable `myName` in subsequent calculations.

Save this file wherever you want on your hard drive. Call it `awesomeScript.R`. Personally, I saved it to my desktop.

After we have a saved script, we can run it by sending all the lines of code over to the console. One way to do that is by clicking the **Source** button at the top right of the script window (see [Figure 1.1](#)). Another way is that we can use R's `source()` function². We can run the following code in the console:

```
# Anything coming after the pound/hash-tag symbol
# is a comment to the human programmer.
# These lines are ignored by R
setwd("/home/taylor/Desktop/")
source("awesomeScript.R")
```

The first line changes the working directory³ to `Desktop/`. The **working directory** is the first place your program looks for files. You, dear reader, should change this line by replacing `Desktop/` to whichever folder you chose to save `awesomeScript.R` in. If you would like to find out what your working directory is currently set to, you can use `getwd()`.

¹<https://stat.ethz.ch/R-manual/R-devel/library/base/html/assignOps.html>

²A third way is to tell R to run `awesomeScript.R` from the command line, but unfortunately, this will not be discussed in this text.

³<https://stat.ethz.ch/R-manual/R-devel/library/base/html/getwd.html>



Every computer has a different folder or directory structure—that is why it is highly recommended you refer to file locations as seldom as possible in your scripts. This makes your code more *portable*. When you send your file to someone else (e.g. your instructor or your boss), she will have to remove or change every mention of any directory. This is because those directories (probably) won't exist on her machine.

The second line calls `source()`. This function finds the script file and executes all the commands found in that file sequentially.



Deleting all saved variables, and then `source()`ing your script can be a very good debugging strategy. You can remove all saved variables by running `rm(list=ls())`. Don't worry—the variables will come back as soon as you `source()` your entire script again!

1.2 Hello World in Python

First, start by opening *Anaconda Navigator* (see [Figure 1.2](#)). It should look something like this:

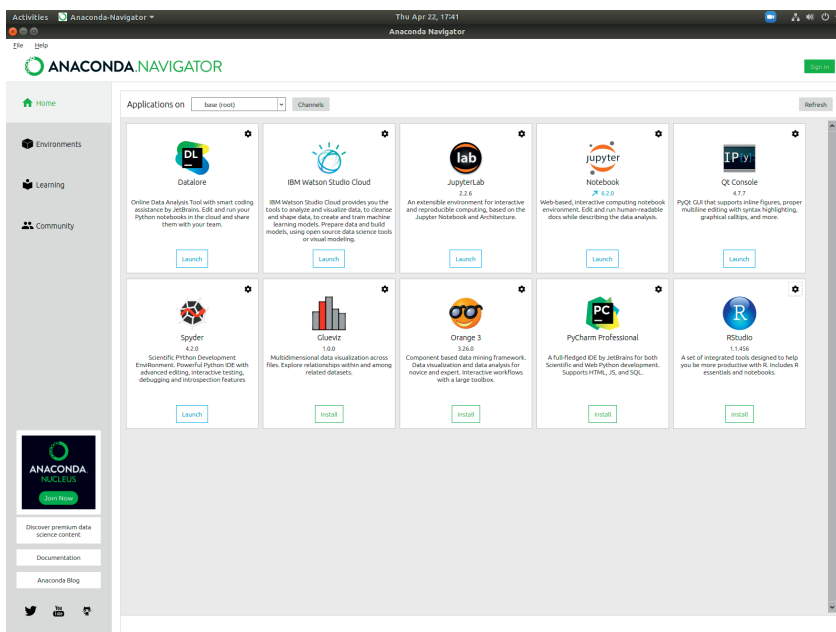


FIGURE 1.2: Anaconda navigator.

Recall that we will exclusively assume the use of *Spyder* (see [Figure 1.3](#)) in this textbook. Open that up now. It should look something like this:

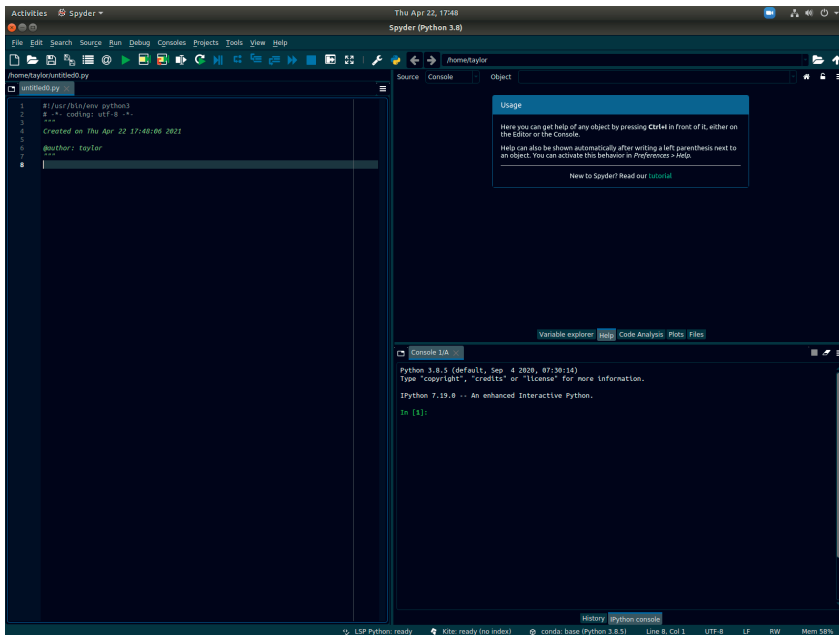


FIGURE 1.3: Spyder.

It looks a lot like RStudio, right? The script window is still on the left hand side, but it takes up the whole height of the window this time. However, you will notice that the console window has moved. It's over on the bottom right now.

Again, you might notice a lot more white when you open this for the first time. Just like last time, I changed my color scheme. You can change yours by going to **Tools -> Preferences** and then exploring the options available under the **Appearances** tab.

Try typing the following line of code into the console:

```
# this looks like R code but it's Python code!
print("hello Python world")
## hello Python world
```

Already we have many similarities between our two languages. Both R and Python have a `print()` function, and they both use the same symbol to start a comment: `#`. Finally, they both define character/string constants with quotation marks. In both languages, you can use either single or double quotes.

We will also show below that both languages share the same three ways to run scripts. Nice!

Let's try writing our first Python script. R scripts end in `.r` or `.R`, while Python scripts end in `.py`. Call this file `awesomeScript.py`.

```
# save this as awesomeScript.py
print('hello world')
print("this program")
print('is pretty similar to the last program')
print('it is not incredibly interesting, either')
my_name = "Taylor"
print(my_name)
```

Notice that the assignment operator is different in Python. It's an `=`⁴.

Just like RStudio, Spyder has a button that runs the entire script from start to finish. It's the green triangle button (see [Figure 1.3](#)).

You can also write code to run `awesomeScript.py`. There are a few ways to do this, but here's the easiest.

```
import os
os.chdir('/home/taylor/Desktop')
runfile("awesomeScript.py")
```

This is also pretty similar to the R code from before. `os.chdir()` sets our working directory to the `Desktop`. Then `runfile()` runs all of the lines in our program, sequentially, from start to finish⁵.

The first line is new, though. We did not mention anything like this in R, yet. We will talk more about importing modules in [section 10.4](#). Suffice it to say that we imported the `os` module to make the `chdir()` function available to us.

⁴You can use this symbol in R, too, but it is less common.

⁵Python, like R, allows you to run scripts from the command line, but this will not be discussed in this text.

1.3 Getting Help

1.3.1 Reading Documentation

Programming is not about memorization. Nobody can memorize, for example, every function and all of its arguments. So what do programmers do when they get stuck? The primary way is to find and read the documentation.

Getting help in R is easy. If you want to know more about a function, type into the console the name of the function with a leading question mark. For example, `?print` or `?setwd`. You can also use `help()` and `help.search()` to find out more about functions (e.g. `help(print)`). Sometimes you will need to put quotation marks around the name of the function (e.g. `?":")`.

This will not open a separate web browser window, which is very convenient. If you are using RStudio, you have some extra benefits. Everything will look very pretty, and you can search through the text by typing phrases into the search bar in the “Help” window.

In Python, the question mark comes *after* the name of the function⁶ (e.g. `print?`), and you can use `help(print)` just as in R.

In Spyder, if you want the documentation to appear in the Help window (it looks prettier), then you can type the name of the function, and then `Ctrl-i` (`Cmd-i` on a mac keyboard).

1.3.2 Understanding File Paths

File paths look different on different operating systems. Mac and Linux machines tend to have forward slashes (i.e. `/`), while Windows machines tend to use backslashes (i.e. `\`).

Depending on what kind of operating system is running your code, you will need to change the file paths. It is important for everyone writing R and Python code to understand how things work on both types of machines—just because you’re writing code on a Windows machine doesn’t mean that it won’t be run on a Mac, or vice versa.

The directory repeatedly mentioned in the code above was `/home/taylor/Desktop`. This is a directory on my machine, which is running Ubuntu Linux. The leading forward slash is the *root directory*. Inside that is the directory `home/`, and inside that is `taylor/`, and inside that is `Desktop/`. If you are running

⁶If you did not install Anaconda, then this may not work for you because this is an IPython (<https://ipython.org>) feature.

MacOS, these file paths will look very similar. The folder `home/` will most likely be replaced with `Users/`.

On Windows, things are a bit different. For one, a full path starts with a drive (e.g. `C:`). Second, there are backslashes (not forward slashes) to separate directory names (e.g. `C:\Users\taylor\Desktop`).

Unfortunately, backslashes are a special character in both R and Python (read [Section 3.9](#) to find out more about this). Whenever you type a `\`, it will change the meaning of whatever comes after it. In other words, `\` is known as an **escape character**.



In both R and Python, the backslash character is used to start an “escape” sequence. You can see some examples in R by clicking [here](#)^a, and some examples in Python by clicking [here](#)^b. In Python it may also be used to allow long lines of code to take up more than one line in a text file.^c

^a<https://stat.ethz.ch/R-manual/R-devel/library/base/html/Quotes.html>

^bhttps://docs.python.org/3/reference/lexical_analysis.html

^chttps://docs.python.org/3/reference/lexical_analysis.html#explicit-line-joining

The recommended way of handling this is to just use forward slashes instead. For example, if you are running Windows, `C:/Users/taylor/Desktop/myScript.R` will work in R, and `C:/Users/taylor/Desktop/myScript.py` will work in Python.

You may also use “raw string constants” (e.g. `r'C:\Users\taylor\my_file.txt'`). “Raw” means that `\` will be treated as a literal character instead of an escape character. Alternatively, you can “escape” the backslashes by replacing each single backslash with a double backslash. Please read [Section 3.9](#) for more details about these choices.

2

Basic Types

In every programming language, data is stored in different ways. Writing a program that manipulates data requires understanding all of the choices. That is why we must be concerned with the different **types** of data in our R and Python programs. Different types are suitable for different purposes.

There are similarities between Python's and R's type systems. However, there are many differences as well. Be prepared for these differences. There are many more of them in this chapter than there were in the previous chapter!

If you're ever unsure what type a variable has, use `type()` (in Python) or `typeof()` (in R) to query it.

Storing an individual piece of information is simple in both languages. However, while Python has scalar types, R does not draw as strong of a distinction between scalar and compound types.

2.1 Basic Types in Python

In Python, the simplest types we frequently use are `str` (short for string), `int` (short for integer), `float` (short for floating point) and `bool` (short for Boolean). This list is not exhaustive, but these are a good collection to start thinking about. For a complete list of built-in types in Python, click [here](https://docs.python.org/3/library/stdtypes.html)¹.

```
print(type('a'), type(1), type(1.3))
## <class 'str'> <class 'int'> <class 'float'>
```

Strings are useful for processing text data such as names of people/places/things and messages such as texts, tweets and emails (Beazley and Jones, 2014). If you are dealing with numbers, you need floating points if you have a number that might have a fractional part after its decimal; otherwise, you'll need an integer. Booleans are useful for situations where you need to record whether something

¹<https://docs.python.org/3/library/stdtypes.html>

is true or false. They are also important to understand for control-flow in section 11.

In the next section we will discuss the Numpy library. This library has a broader collection² of basic types that allows for finer control over any script you write.

2.1.1 Type Conversions in Python

We will often have to convert between types in a Python program. This is called **type conversion**, and it can be either implicitly or explicitly done.

For example, consider two types to store individual numbers: ints versus floats. floats and ints are ideal for storing numbers with and without a fractional part, respectively. ints are often implicitly converted to floats, so that arithmetic operations work.

```
my_int = 1
my_float = 3.2
my_sum = my_int + my_float
print("my_int's type", type(my_int))
## my_int's type <class 'int'>
print("my_float's type", type(my_float))
## my_float's type <class 'float'>
print(my_sum)
## 4.2
print("my_sum's type", type(my_sum))
## my_sum's type <class 'float'>
```

You might be disappointed if you always count on this behavior, though. For example, try the following piece of code on your machine. You will receive the following error: `TypeError: unsupported operand type(s) for +: 'float' and 'str'`.

```
3.2 + "3.2"
```

Explicit conversions occur when we as programmers explicitly ask Python to perform a conversion. We will do this with the functions such as `int()`, `str()`, `float()`, and `bool()`.

²<https://numpy.org/doc/stable/user/basics.types.html>

```
my_date = "5/2/2021"
month_day_year = my_date.split('/')
my_year = int(month_day_year[-1])
print('my_year is', my_year, 'and its type is', type(my_year))
## my_year is 2021 and its type is <class 'int'>
```

2.2 Basic Types in R

In R, the names of basic types are only slightly different. They are `logical` (instead of `bool`), `integer` (instead of `int`), `double` or `numeric` (instead of `float`)³, `character` (instead of `str`), `complex` (for calculations involving imaginary numbers), and `raw` (useful for working with bytes).

```
# cat() is kind of like print()
cat(typeof('a'), typeof(1), typeof(1.3))
## character double double
```

In this case, R automatically upgraded 1 to a double. If you wanted to force it to be an integer, you can add a capital “L” to the end of the number.

```
# cat() is kind of like print()
cat(typeof('a'), typeof(1L), typeof(1.3))
## character integer double
```

2.2.1 Type Conversions in R

You can explicitly and implicitly convert types in R just as you did in Python. Implicit conversion looks like this.

```
myInt = 1
myDouble = 3.2
mySum = myInt + myDouble
print(paste0("my_int's type is ", typeof(myInt)))
## [1] "my_int's type is double"
print(paste0("my_float's type is ", typeof(myDouble)))
```

³“double” is short for “double precision floating point.” In other programming languages, the programmer might choose how many decimal points of precision he or she wants.

```
## [1] "my_float's type is double"
print(mySum)
## [1] 4.2
print(paste0("my_sum's type is ", typeof(mySum)))
## [1] "my_sum's type is double"
```

Explicit conversion can be achieved with functions such as `as.integer`, `as.logical`, `as.double`, etc.

```
print(typeof(1))
## [1] "double"
print(typeof(as.logical(1)))
## [1] "logical"
```

2.2.2 R's Simplification

The basic types of R are a little different than the basic types of Python. On the one hand, Python has basic types for individual elements, and it uses separate types as containers for storing many elements. On the other hand, R uses the same type to store a single element as it does to store many elements. Strictly speaking, R does not have a scalar type.

Technically, all of the examples we just did in R are using length one **vectors**—logical integer double, character, complex, and raw are the possible **modes** of a vector. **vectors** will be discussed further section 3.

Think about which option you prefer. What are the benefits of using separate types for scalars and collections? What are the benefits of using the same type?

2.3 Exercises

2.3.1 R Questions

1.

Which R base type is ideal for each piece of data? Assign your answers to a character vector of length four called `questionOne`.

- a) An individual's IP address
- b) whether or not an individual attended a study

- c) the number of seeds found in a plant
- d) the amount of time it takes for a car to race around a track

2.

Floating points are weird. What gets printed is not the same as what is stored! In R, you can control how many digits get printed by using the `options` function.

- a) Assign `a` to `2/3`
- b) `print a`, and copy/paste what you see into the variable `aPrint`. Make sure it is a character.
- c) Take a look at the documentation for `options`. Assign the value of `options()$digits` to `numDigitsStart`
- d) Change the number of digits to 22
- e) Again, `print, a` and copy/paste what you see into the variable `aPrintv2`. Make sure it is a character.
- f) Assign the output of `options()$digits` to `numDigitsEnd`

3.

Floating points are weird. What gets stored might not be what you want. “The only numbers that can be represented exactly in R’s numeric type are integers and fractions whose denominator is a power of 2.”⁴ As a consequence, you should never test strict equality (i.e. using `==`) between two floating points.

- a) Assign the square root of 2 to `mySqrt`
- b) Print the square of this variable
- c) Test (using `==`) that this variable is equal to 2. Assign the result of this test to `isTwoRecoverable`
- d) Test for near equality (using `all.equal`) that this variable is “equal” to 2. Assign the result of this test to `closeEnough`. Make sure to read the documentation for this function because the return type can be tricky!

2.3.2 Python Questions

1.

Which Python type is ideal for each piece of data? Assign your answers to a list of strings called `question_one`.

⁴https://cran.r-project.org/doc/FAQ/R-FAQ.html#Why-doesn_0027t-R-think-these-numbers-are-equal_003f

- a) An individual's IP address
- b) whether or not an individual attended a study
- c) the number of seeds found in a plant
- d) the amount of time it takes for a car to race around a track

2.

Floating points are weird. What gets printed is not the same as what is stored! In Python, you need to edit a class's `__str__` method if you want to control how many digits get printed for a user-defined type/class, but we won't do that. Instead, we'll use `str.format()`⁵ to return a string directly (instead of copy/paste-ing it).

- a) Assign `a` to `2/3`
- b) `print a`, and copy/paste what you see into the variable `a_print`
- c) Create a `str` that displays 22 digits of `2/3`. Call it `a_printv2`
- d) `print` the above string

3.

Floating points are weird. What gets stored might not be what you want. The Python documentation has an excellent discussion of how storage behavior can be surprising. Click here⁶ to read it.

- a) Assign the square root of 2 to `my_sqrt`
- b) `print` the square of this variable
- c) Test (using `==`) that this variable is equal to 2. Assign the result of this test to `is_two_recoverable`
- d) Test for near equality (using `np.isclose`) that this variable is "equal" to 2. Assign the result of this test to `close_enough`.

⁵<https://docs.python.org/3/library/stdtypes.html#str.format>

⁶<https://docs.python.org/3/tutorial/floatingpoint.html>

3

R Vectors versus Numpy Arrays and Pandas' Series

This section is for describing the data types that let us store collections of elements that all **share the same type**. Data is very commonly stored in this fashion, so this section is quite important. Once we have one of these collection objects in a program, we will be interested in learning how to extract and modify different elements in the collection, as well as how to use the entire collection in an efficient calculation.

3.1 Overview of R

In the previous section, I mentioned that R does not have scalar types—it just has **vectors**¹. So, whether you want to store one number (or `logical`, or character, or ...), or many numbers, you will need a **vector**.

■ For many, the word “vector” evokes an impression that these objects are designed to be used for performing matrix arithmetic (e.g. inner products, transposes, etc.). You can perform these operations on ■ vectors, but in my opinion, this preconception can be misleading, and I recommend avoiding it. Most of the things you can do with vectors in R have little to do with linear algebra!

How do we create one of these? There are many ways. One common way is to read in elements from an external data set. Another way is to generate **vectors** from code.

```
1:10          # consecutive integers
## [1]  1  2  3  4  5  6  7  8  9 10
seq(1,10,2)   # arbitrary sequences
## [1]  1  3  5  7  9
rep(2,5)      # repeating numbers
```

¹<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Vector-objects>

```
## [1] 2 2 2 2 2
# combine elements without relying on a pattern
c("5/2/2021", "5/3/2021", "5/4/2021")
## [1] "5/2/2021" "5/3/2021" "5/4/2021"
# generate Gaussian random variables
rnorm(5)
## [1] 0.6403955 0.2524081 1.3740108 2.2544786 -1.2988103
```

`c()` is short for “combine”. `seq()` and `rep()` are short for “sequence” and “replicate”, respectively. `rnorm()` samples normal (or Gaussian) random variables. There is plenty more to learn about these functions, so I encourage you to take a look at their documentation.

I should mention that functions such as `rnorm()` don’t create truly random numbers, just *pseudorandom* ones. Pseudorandom numbers are nearly indecipherable from truly random ones, but the way the computer generates them is actually deterministic.

First, a *seed*, or starting number is chosen. Then, the *pseudorandom number generator (PRNG)* maps that number to another number. The sequence of all the numbers appears to be random, but is actually deterministic.



Sometimes you will be interested in setting the seed on your own because it is a cheap way of sharing and communicating data with others. If two people use the same starting seed and the same PRNG, then they should simulate the same data. This can be important if you want to help other people reproduce the results generated by code you share. Most of the time, though, I don’t set the seed, and I don’t think about the distinction between random and pseudorandom numbers.

3.2 Overview of Python

If you want to store many elements of the same type (and size) in Python, you will probably need a Numpy array. Numpy is a highly-regarded third party library (Harris et al., 2020) for Python. Its `array` objects store elements of the same type, just as R’s `vectors` do.

There are five ways to create numpy arrays (source²). Here are some examples that complement the examples from above.

```
import numpy as np
np.array([1,2,3])
## array([1, 2, 3])
np.arange(1,12,2)
## array([ 1,  3,  5,  7,  9, 11])
np.random.normal(size=3)
## array([0.64255822, 0.01151642, 0.32897288])
```

Another option for storing a homogeneous collection of elements in Python is a `Series` object³ from the Pandas library. The benefit of these is that they play nicely with Pandas' data frames (more information about Pandas' data frames can be found in [Section 8.2](#)), and that they have more flexibility with accessing elements by name (see here⁴ for more information).

```
import pandas as pd
first = pd.Series([2, 4, 6])
second = pd.Series([2, 4, 6], index = ['a','b','c'])
print(first[0])
## 2
print(second['c'])
## 6
```

3.3 Vectorization in R

An operation in R is **vectorized** if it applies to all of the elements of a **vector** at once. An operator that is not vectorized can only be applied to individual elements. In that case, the programmer would need to write more code to instruct the function to be applied to all of the elements of a vector. You should prefer writing vectorized code because it is usually easier to read. Moreover, many of these vectorized functions are written in compiled code, so they can often be much faster.

²<https://numpy.org/doc/stable/user/basics.creation.html>

³<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.html#pandas-series>

⁴<https://jakevdp.github.io/PythonDataScienceHandbook/03.01-introducing-pandas-objects.html#Series-as-generalized-NumPy-array>

Arithmetic (e.g. `+`, `-`, `*`, `/`, `^`, `%%`, `%/%`, etc.) and logical (e.g. `!`, `|`, `&`, `>`, `>=`, `<`, `<=`, `==`, etc.) operators are commonly applied to one or two vectors. Arithmetic is usually performed *element-by-element*. Numeric vectors are converted to logical vectors if they need to be. Be careful of operator precedence if you seek to minimize your use of parentheses.

Note that there are an extraordinary amount of named functions (e.g. `sum()`, `length()`, `cumsum()`, etc.) that operate on entire vectors, as well. Here are some examples:

```
(1:3) * (1:3)
## [1] 1 4 9
(1:3) == rev(1:3)
## [1] FALSE TRUE FALSE
sin( (2*pi/3)*(1:4))
## [1] 8.660254e-01 -8.660254e-01 -2.449294e-16 8.660254e-01
```

In the last example, there is **recycling** happening. `(2*pi/3)` is taking three length-one vectors and producing another length-one vector. The resulting length-one vector is multiplied by a length four vector `1:4`. The single element in the length one vector gets *recycled* so that its value is multiplied by every element of `1:4`. This makes sense most of the time, but sometimes recycling can be tricky. Notice that the following code does not produce an error—just a warning: longer object length is not a multiple of shorter object length. Try running it on your machine to confirm this.

```
(1:3) * (1:4)
```

3.4 Vectorization in Python

The Python's Numpy library makes extensive use of vectorization as well. Vectorization in Numpy is accomplished with **universal functions**⁵, or “ufuncs” for short. Some ufuncs can be invoked using the same syntax as in R (e.g. `+`). You can also refer to function by its name (e.g. `np.sum()` instead of `+`). Mixing and matching is allowed, too.

⁵<https://numpy.org/doc/stable/reference/ufuncs.html>

Ufuncs are called *unary* if they take in one array, and *binary* if they take in two. At the moment, there are fewer than 100 available⁶, all performing either mathematical operations, boolean-emitting comparisons, or bit-twiddling operations. For an exhaustive list of Numpy’s universal functions, click here.⁷ Here are some code examples:

```
np.arange(1,4)*np.arange(1,4)
## array([1, 4, 9])
np.zeros(5) > np.arange(-3,2)
## array([ True,  True,  True, False, False])
np.exp( -.5 * np.linspace(-3, 3, 10)**2) / np.sqrt( 2 * np.pi)
## array([0.00443185, 0.02622189, 0.09947714, 0.24197072, 0.37738323,
##        0.37738323, 0.24197072, 0.09947714, 0.02622189, 0.00443185])
```

Instead of calling it “recycling”, Numpy calls reusing elements of a shorter array in a binary operation **broadcasting**⁸. It’s the same idea as in R, but in general, Python is stricter and disallows more scenarios. For example, try running the following code on your machine. You should receive an error: `ValueError: operands could not be broadcast together with shapes (2,) (3,)`.

```
np.arange(1,3)*np.arange(1,4)
```

If you are working with string arrays, Numpy has a `np.char` module with many useful functions⁹.

```
a = np.array(['a','b','c'])
np.char.upper(a)
## array(['A', 'B', 'C'], dtype='<U1')
```

Then there are the `Series` objects from Pandas. Ufuncs continue to work in the same way on `Series` objects, and they respect common index values¹⁰.

```
s1 = pd.Series(np.repeat(100,3))
s2 = pd.Series(np.repeat(10,3))
```

⁶<https://numpy.org/doc/stable/reference/ufuncs.html#available-ufuncs>

⁷<https://numpy.org/doc/stable/reference/ufuncs.html#available-ufuncs>

⁸<https://numpy.org/devdocs/user/theory.broadcasting.html>

⁹<https://numpy.org/doc/stable/reference/routines.char.html#module-numpy.char>

¹⁰<https://jakevdp.github.io/PythonDataScienceHandbook/03.03-operations-in-pandas.html>

```
s1 + s2
## 0      110
## 1      110
## 2      110
## dtype: int64
```

If you feel more comfortable, and you want to coerce these `Series` objects to Numpy arrays before working with them, you can do that. For example, the following works.

```
s = pd.Series(np.linspace(-1,1,5))
np.exp(s.to_numpy())
## array([0.36787944, 0.60653066, 1.          , 1.64872127, 2.71828183])
```

In addition, `Series` objects possess many extra attributes and methods¹¹.

```
ints = pd.Series(np.arange(10))
ints.abs()
## 0      0
## 1      1
## 2      2
## 3      3
## 4      4
## 5      5
## 6      6
## 7      7
## 8      8
## 9      9
## dtype: int64
ints.mean()
## 4.5
ints.floordiv(2)
## 0      0
## 1      0
## 2      1
## 3      1
## 4      2
```

¹¹<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.html#pandas-series>

```
## 5      2
## 6      3
## 7      3
## 8      4
## 9      4
## dtype: int64
```

Series objects that have text data¹² are a little bit different. For one, you have to access the `.str` attribute of the Series before calling any vectorized methods¹³. Here are some examples.

```
s = pd.Series(['a', 'b', 'c', '33'])
s.dtype
```

```
## dtype('O')
```

```
s.str.isdigit()
```

```
## 0      False
## 1      False
## 2      False
## 3       True
## dtype: bool
```

```
s.str.replace('a', 'z')
```

```
## 0      z
## 1      b
## 2      c
## 3     33
## dtype: object
```

String operations can be a big game changer, and we discuss text processing strategies in more detail in [Section 3.9](#).

¹²https://pandas.pydata.org/pandas-docs/stable/user_guide/text.html#working-with-text-data

¹³<https://jakevdp.github.io/PythonDataScienceHandbook/03.10-working-with-strings.html>

3.5 Indexing Vectors in R

It is very common to want to extract or modify a subset of elements in a vector. There are a few ways to do this. All of the ways I discuss will involve the square bracket operator (i.e. `[]`). Feel free to retrieve the documentation by typing `?[']`.

```
allElements <- 1:6
allElements[seq(2,6,2)] # extract evens
## [1] 2 4 6
allElements[-seq(2,6,2)] <- 99 # replace all odds with 99
allElements[allElements > 2] # get nums bigger than 2
## [1] 99 99 4 99 6
```

To access the first element, we use the index 1. To access the second, we use 2, and so on. Also, the `-` sign tells R to remove elements. Both of these functionalities are *very different* from Python, as we will see shortly.

We can use names to access elements elements, too, but only if the elements are named.

```
sillyVec <- c("favorite"=1, "least favorite" = 2)
sillyVec['favorite']
## favorite
##          1
```

3.6 Indexing Numpy arrays

Indexing Numpy arrays¹⁴ is very similar to indexing vectors in R. You use the square brackets, and you can do it with logical arrays or index arrays. There are some important differences, though.

For one, indexing is 0-based in Python. The 0th element is the first element of an array. Another key difference is that the `-` isn't used to remove elements like it is in R, but rather to count backwards. Third, using one or two `:` inside square brackets is more flexible in Python. This is syntactic sugar for using the `slice()` function, which is similar to R's `seq()` function.

¹⁴<https://numpy.org/doc/stable/user/basics.indexing.html>

```

one_through_ten = np.arange(1, 11)
one_through_ten[np.array([2,3])]
## array([3, 4])
one_through_ten[1:10:2] # evens
## array([ 2,  4,  6,  8, 10])
one_through_ten[::-1] # reversed
## array([10,  9,  8,  7,  6,  5,  4,  3,  2,  1])
one_through_ten[-2] = 99 # second to last
one_through_ten
## array([ 1,  2,  3,  4,  5,  6,  7,  8, 99, 10])
one_through_ten[one_through_ten > 3] # bigger than three
## array([ 4,  5,  6,  7,  8, 99, 10])

```

3.7 Indexing Pandas' Series

At a minimum, there is little that is new that you *need* to learn to go from Numpy arrays to Pandas' Series objects. They still have the `[]` operator, and many methods are shared across these two types¹⁵. The following is almost equivalent to the code above, and the only apparent difference is that the results are printed a little differently.

```

import pandas as pd
one_through_ten = pd.Series(np.arange(1, 11))
one_through_ten[np.array([2,3])]
## 2    3
## 3    4
## dtype: int64
one_through_ten[1:10:2] # evens
## 1    2
## 3    4
## 5    6
## 7    8
## 9   10
## dtype: int64
one_through_ten[::-1] # reversed
## 9   10

```

¹⁵<https://pandas.pydata.org/docs/reference/api/pandas.Series.html>

```
## 8      9
## 7      8
## 6      7
## 5      6
## 4      5
## 3      4
## 2      3
## 1      2
## 0      1
## dtype: int64
one_through_ten[-2] = 99 # second to last
one_through_ten
## 0      1
## 1      2
## 2      3
## 3      4
## 4      5
## 5      6
## 6      7
## 7      8
## 8      9
## 9     10
## -2     99
## dtype: int64
one_through_ten[one_through_ten > 3] # bigger than three
## 3      4
## 4      5
## 5      6
## 6      7
## 7      8
## 8      9
## 9     10
## -2     99
## dtype: int64
one_through_ten.sum()
## 154
```

However, Pandas' Series have `.loc` and `.iloc` methods¹⁶. We won't talk much about these two methods now, but they will become very important when we start to discuss Pandas' data frames in [Section 8.2](#).

```
one_through_ten.iloc[2]
## 3
one_through_ten.loc[2]
## 3
```

3.8 Some Gotchas

3.8.1 Shallow versus Deep Copies

In R, assignment usually produces a **deep copy**. In the code below, we create `b` from `a`. If we modify `b`, these changes don't affect `a`. This takes up more memory, but our program is easier to follow as we don't have to keep track of connections between objects.

```
# in R
a <- c(1,2,3)
b <- a
b[1] <- 999
a # still the same!
## [1] 1 2 3
```

With Numpy arrays in Python, “shallow copies” can be created by simple assignment, or by explicitly constructing a **view**¹⁷. In the code below, `a`, `b`, `c`, and `d` all share the same data. If you modify one, you change all the others. This can make the program more confusing, but on the other hand, it can also improve computational efficiency.

```
# in python
a = np.array([1,2,3])
b = a # b is an alias
c = a.view() # c is a view
d = a[:]
```

¹⁶https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#different-choices-for-indexing

¹⁷<https://numpy.org/devdocs/user/quickstart.html#copies-and-views>


```
b[0] = 999
a # two names for the same object in memory
## array([999,  2,  3])
b
## array([999,  2,  3])
c
## array([999,  2,  3])
d
## array([999,  2,  3])
```

It's the same story with Pandas' Series objects. You're usually making a "shallow" copy.

```
# in python
import pandas as pd
s1 = pd.Series(np.array([100.0,200.0,300.0]))
s2 = s1
s3 = s1.view()
s4 = s1[:]
s1[0] = 999
s1
## 0    999.0
## 1    200.0
## 2    300.0
## dtype: float64
s2
## 0    999.0
## 1    200.0
## 2    300.0
## dtype: float64
s3
## 0    999.0
## 1    200.0
## 2    300.0
## dtype: float64
s4
## 0    999.0
## 1    200.0
## 2    300.0
## dtype: float64
```

If you want a “deep copy” in Python, you usually want a function or method called `copy()`. Use `np.copy` or `np.ndarray.copy`¹⁸ when you have a Numpy array.

```
# in python
a = np.array([1,2,3])
b = np.copy(a)
c = a.copy()
b[0] = 999
a
## array([1, 2, 3])
b
## array([999,  2,  3])
c
## array([1, 2, 3])
```

Use `pandas.Series.copy`¹⁹ with Pandas’ Series objects. Make sure not to set the `deep` argument to `False`. Otherwise you’ll get a shallow copy.

```
# in python
s1 = pd.Series(np.array([1,2,3]))
s2 = s1.copy()
s3 = s1.copy(deep=False)
s1[0] = 999
s1
## 0    999
## 1     2
## 2     3
## dtype: int64
s2
## 0     1
## 1     2
## 2     3
## dtype: int64
s3
## 0    999
## 1     2
```

¹⁸<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.copy.html#numpy-ndarray-copy>

¹⁹<https://pandas.pydata.org/docs/reference/api/pandas.Series.copy.html#pandas-series-copy>

```
## 2      3
## dtype: int64
```

3.8.2 How R and Python Handle Missing Values

R has `NULL`, `NaN`, and `NA`. Python has `None` and `np.nan`. If your eyes are glazing over already and you're thinking "they all look like the same"—they are not.

R's `NULL` and Python's `None` are similar. Both represent "nothingness." This is *not* the same as `0`, or an empty string, or `FALSE/False`. This is commonly used to detect if a user fails to pass in an argument to a function, or if a function fails to "return" (more information on functions can be found in Section 6) anything meaningful.

In R, for example, if a function fails to return anything, then it actually returns a `NULL`. A `NULL` object has its own type.²⁰

```
NULL == FALSE
## logical(0)
NULL == NULL
## logical(0)
# create a function that doesn't return anything
# more information on this later
doNothingFunc <- function(a){}
thing <- doNothingFunc() # call our new function
is.null(thing)
## [1] TRUE
typeof(NULL)
## [1] "NULL"
```

In Python, we have the following.

```
None == False
## False
None == None
# create a function that doesn't return anything
# more information on this later
## True
def do_nothing_func():
```

²⁰<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#NULL-object>

```

pass
thing = do_nothing_func()
if thing is None:
    print("thing is None!")
## thing is None!
type(None)
## <class 'NoneType'>

```

“NaN” stands for “not a number.” NaN is an object of type `double` in R, and `np.nan` is of type `float` in Python. It can come in handy when you (deliberately or accidentally) perform undefined calculations such as $0/0$ or $\infty/-\infty$.

```

# in R
0/0
## [1] NaN
Inf/Inf
## [1] NaN
is.na(0/0)
## [1] TRUE

```

```

# in Python
# 0/0
# the above yields a ZeroDivisionError
import numpy as np
np.inf/np.inf
## nan
np.isnan(np.nan)
## True

```

“NA” is short for “not available.” Missing data is a fact of life in data science. Observations are often missing in data sets, introduced after joining/merging data sets together (more on this in [Section 12.3](#)), or arise from calculations involving underflow and overflow. There are many techniques designed to estimate quantities in the presence of missing data. When you code them up, you’ll need to make sure you deal with NAs properly.

```

# in R
babyData <- c(0,-1,9,NA,21)
NA == TRUE

```

```
## [1] NA
is.na(babyData)
## [1] FALSE FALSE FALSE TRUE FALSE
typeof(NA)
## [1] "logical"
```

Unfortunately, Python's support of an NA-like object is more limited. There is no NA object in base Python. And often NaNs will appear in place of an NA. There are a few useful tools, though. The Numpy library offers “masked arrays”²¹, for instance.

Also, as of version 1.0.0, the pandas library²² has an experimental `pd.NA` object. However, they warn²³ that “the behaviour of `pd.NA` can still change without warning.”

```
import numpy as np
import numpy.ma as ma
baby_data = ma.array([0,-1,9,-9999, 21]) # -9999 "stands for" missing
baby_data[3] = ma.masked
np.average(baby_data)
## 7.25
```

- Be careful of using extreme values to stand in for what should be an NA. Be aware that some data providers will follow this strategy. I
- recommend that you avoid it yourself. Failing to represent a missing value correctly would lead to extremely wrong calculations!

3.9 An Introduction to Regular Expressions

We have already talked a little about how to work with text data in this book. Regarding Python, [Section 3.4](#) mentioned that Pandas `Series` objects have a `.str` accessor attribute²⁴ that has plenty of special methods that will work on

²¹<https://numpy.org/devdocs/reference/maskedarray.html>

²²https://pandas.pydata.org/docs/user_guide/index.html#user-guide

²³https://pandas.pydata.org/pandas-docs/dev/user_guide/missing_data.html#missing-data-na

²⁴https://pandas.pydata.org/pandas-docs/version/1.3/user_guide/text.html#string-methods

string data. The same tools can be used whether or not these `Series` objects are contained in a Pandas `DataFrame`.

Regarding R, `character` vectors were first mentioned in [Section 3.1](#). There are many functions that operate on these, too, regardless of whether they are held in a `data.frame`. The functions might be a little harder to find because they aren't methods, so pressing `<Tab>` and using your GUI's autocomplete feature doesn't reveal them as easily.

Suppose you're interested in replacing lowercase letters with uppercase ones, removing certain characters from text, or counting the number of times a certain expression appears. Up until now, as long as you can find a function or method that performs the task, you were doing just fine. If you need to do something with text data, there's probably a function for it.

Notice what all of these tasks have in common—they all require the ability to find patterns. When your patterns are easy to describe (e.g. find all lowercase “a”s), then all is well. What can make matters more complicated, however, is when the patterns are more difficult to describe (e.g. find all valid email addresses). That is why this section is primarily concerned with discussing **regular expressions**, which are a tool that help you describe the patterns in text (Wickham and Grolemund, 2017) (López, 2014).

3.9.1 Literal Characters versus Metacharacters

Every character in a regular expression is interpreted in one of two ways. Either it is interpreted as a

1. literal character, or as a
2. metacharacter.

If it is a literal character, then the character is the *literal* pattern. For example, in the regular expression “e”, the character “e” has a literal interpretation. If you seek to capitalize all instances of “e” in the following phrase, you can do it pretty easily. As long as you know which function performs find-and-replace, you're good. The pattern is trivial to specify.

On the other hand, if I asked you to remove \$s from price or salary data, you might have a little more difficulty. This is because \$ is a *metacharacter* in regular expressions, and so it has a special meaning.²⁵ In the examples below,

²⁵The dollar sign is useful if you only want to find certain patterns that finish a line. It takes the characters preceding it, and says, only look for that pattern if it comes at the end of a string.

if `$` is interpreted as a regular expression, the pattern will not be found at all, despite the prevalence of *literal* dollar signs.

There are a few functions in R that perform find-and-replace, but in this case, I use `gsub()`. In Pandas, I can use `.str.replace()`²⁶, to do this. Here are the examples that find patterns that are described by literal characters.

```
# in R
gsub(pattern = "e", replacement = "E",
      x = "I don't need a regex for this!")
## [1] "I don't nEEd a rEgEx for this!"
```

```
# in Python
import pandas as pd
s = pd.Series(["I don't need a regex for this!"])
s.str.replace(pat="e",repl="E")
## 0      I don't nEEd a rEgEx for this!
## dtype: object
```

On the other hand, here are a few examples that remove dollar signs. We generally have two options to recognize symbols that happen to be metacharacters.

1. We can *escape* the dollar sign. That means you need to put a backslash (i.e. `\`) before the dollar sign. The backslash is a metacharacter looks at the character coming after it, and it either removes the special meaning from a metacharacter, or adds special meaning to a literal character.
2. Alternatively, we can tell the function to ignore regular expressions. `gsub()` can take `fixed=TRUE`, and `.str.replace()` can take `regex=False`.

```
# in Python
pd.Series(["$100, $200"]).str.replace(pat="$",repl="",regex=False)
## 0      100, 200
## dtype: object
pd.Series(["$100, $200"]).str.replace(pat="\$",repl="")
## 0      100, 200
## dtype: object
```

²⁶<https://pandas.pydata.org/docs/reference/api/pandas.Series.str.replace.html>

```
# in R
gsub(pattern = "$", replacement = "", x = c("$100, $200"), fixed=TRUE)
## [1] "100, 200"
stringr::str_remove_all(c("$100, $200"), pattern = "\\$")
## [1] "100, 200"
```

3.9.2 The Trouble with Backslashes: Escape Sequences

You might have noticed above and gotten confused—sometimes in Python and in R, we need *two* backslashes instead of *one*. This is because backslashes have another purpose that can complicate our using them to escape metacharacters. They also help us write untypeable characters, also known as **escape sequences**. We need to be able to do this even if we aren’t using regular expressions.

For instance, consider the way we type the “newline” character. Even though it is understood by the computer as one character, it takes us two keystrokes to write it with our keyboard. `\` is one character, and `n` is another, but together they are one!

```
nchar('\n') #in R
## [1] 1
```

```
len('\n') #in Python
## 1
```

`strs` in Python and `character vectors` in R will look for these combinations by default. When we specify regular expressions with strings, the backslashes will be used first for this purpose. Their regular expression purpose is a second priority.

The reason we used `\\$` in the above example is to escape the second backslash. `\$` is not a special character, but Python and R will handle it differently. Python will not recognize it, and it won’t complain that it didn’t. On the other hand, R will throw an error that it can’t recognize it.

```
len('\$') # in Python, not special
## 2
```



```
nchar('\$') # R gets confused
## Error: '\$' is an unrecognized escape in character string starting "'\$"
```

There is another way to deal with this issue—**raw strings**! Raw strings make life easier because they do not interpret backslashes as the beginning of escape sequences. You can make them in R and Python by putting an “r” in front of the quotation marks. However, it is slightly more complicated in R because you need a delimiter pair inside the quotation marks—for more information type `?Quotes` in your R console.

```
len(r'\n') # in Python
## 2
```

```
nchar(r'{\$}') # in R
## [1] 2
```

3.9.3 More Examples of Using Regular Expressions

Regular expressions that match many different types of characters are often very useful—these are called **character classes**. For example, `.` represents any character except a newline, `\d` represents any digit, and `\s` represents any whitespace character. You can sometimes capitalize the letters in the regular expression to get the opposite pattern.

```
# anonymize phone numbers in R
gsub(pattern = r"{\d}", replacement = "X", x = "(202)555-0191")
## [1] "(XXX)XXX-XXXX"
```

```
# remove everything that isn't a number in Python
pd.Series(["$100"]).str.replace(pat="\D",repl="")
## 0    100
## dtype: object
```

Many character classes feature an opening and closing square brackets. For instance, `[1-5]` matches any digit between 1 and 5 (inclusive), `[aeiouy]` matches any lowercase vowel, and `[\^\\-]` matches either `^` or `-` (we had to escape these two metacharacters because we are only interested in the literal pattern).

```
# remove vowels in R
gsub(pattern = "[aeiouy]", replacement = "",
      x = "Can you still read this?")
## [1] "Cn  stll rd ths?"
```

Concatenating two patterns, one after another, forms a more specific pattern to be matched.

```
# convert date formats in Python
s = pd.Series(["2021-10-23", "2021:10:23"])
s.str.replace(pat="[:\\-]", repl="/")
## 0      2021/10/23
## 1      2021/10/23
## dtype: object
```

If you would like one pattern or another to appear, you can use the **alternation operator** `|`.

```
# one or the other in Python
pd.Series(["this", "that"]).str.contains(pat="this|that")
## 0      True
## 1      True
## dtype: bool
```

In addition to concatenation, alternation, and grouping, there are more general ways to *quantify* how many times the desired pattern will appear. `?` means zero or one time, `*` means zero or more, `+` will mean one or more, and there are a variety of ways to be even more specific with curly braces (e.g. `{3,17}` means anywhere from three to seventeen times).

```
# detect double o's in R
grepl(pattern = "o{2}", x = c("look", "book", "box", "hellooo"))
## [1] TRUE TRUE FALSE TRUE
```

```
# detects phone number formats in Python
s = pd.Series(["(202)555-0191", "202-555-0191"])
s.str.contains(pat=r"\(\d{3}\)\d{3}-\d{4}")
## 0      True
```

```
## 1    False
## dtype: bool
```

Notice in the double “o” example, the word with three matched. To describe that not being desirable requires the ability to *look ahead* of the match, to the next character, and evaluate that. You can look ahead, or behind, and make assertions about what patterns are required or disallowed.

Lookaround Regex	Meaning
(?=pattern)	Positive looking ahead for pattern
(?!pattern)	Negative looking ahead for pattern
(?<=pattern)	Positive looking behind for pattern
(?<!pattern)	Negative looking behind for pattern

After oo we specify (?!o) to disallow a third, trailing o.

```
# exactly two "o"s in Python?
s = pd.Series(["look","book","box", "hellooo"])
s.str.contains(pat="oo(?!o)")
## 0    True
## 1    True
## 2    False
## 3    True
## dtype: bool
```

However, this does not successfully remove "hellooo" because it will match on the *last* two “o”s of the word. To prevent this, we can prepend a (?<!o), which disallows a leading “o”, as well. In R, we also have to specify perl=TRUE to use Perl-compatible regular expressions.

```
# exactly two "o"s in R
grep(pattern = "(?<!o)oo(?!o)",
      x = c("look","book","box", "hellooo"), perl = TRUE)
## [1] 1 2
```

We also mention *anchoring*. If you only want to find a pattern at the beginning of text, use ^. If you only want to find a pattern at the end of text, use \$. Below we use `.str.extract()`²⁷, whose documentation makes reference to *capture*

²⁷<https://pandas.pydata.org/docs/reference/api/pandas.Series.str.extract.html>

groups. Capture groups are just regular expressions grouped inside parentheses (e.g. (this)).

```
# extract emails with Pandas
s = pd.Series(["my email is fake@aol.com", "fake2@aol.com is hidden"])
s.str.extract(r".([a-z]+@[a-z]+\..com$)")
##                                0
## 0    fake@aol.com
## 1                NaN
```

3.10 Exercises

3.10.1 R Questions

1.

Let's flip some coins! Generate a thousand flips of a fair coin. Use `rbinom`, and let heads be coded as 1 and tails coded as 0.

- Assign the thousand raw coin flips to a variable `flips`. Make sure the elements are integers, and make sure you flip a "fair" coin ($p = .5$).
- Create a length 1000 logical vector called `isHeads`. Whenever you get a heads, make sure the corresponding element is `TRUE` and `FALSE` otherwise.
- Create a variable called `numHeads` by tallying up the number of heads.
- Calculate the percent of time that the number changes in `flips`. Assign your number to `acceptanceRate`. Try to write only one line of code to do this.

2.

Compute the elements of the tenth order Taylor approximation to $\exp(3)$ and store them in `taylorElms`. Do not sum them. Use only one expression, and do not use any loop. The approximation is,

$$1 + 3 + 3^2/2! + 3^3/3! + \dots 3^{10}/10!$$

You want to store each of those eleven numbers separately in a numeric vector.

3.

Do the following.

- a) Create a vector called `nums` that contains all consecutive integers from `-100` to `100`.
- b) Create a `logical` vector that has the same length as the above, and contains `TRUE` whenever an element of the above is even, and `FALSE` otherwise. Call it `myLogicals`.
- c) Assign the total number of `TRUE`s to `totalEven`.
- d) Create a vector called `evens` of all even numbers from the above vector.
- e) Create a vector called `reverse` that holds the reversed elements of `nums`.

4.

Let's say we wanted to calculate the following sum $\sum_{i=1}^N x_i$. If N is large, or most of the x_i s are large, then we might bump up against the largest allowable number. This is the problem of *overflow*. The biggest integer and biggest floating point can be recovered by typing `.Machine$integer.max` and `.Machine$double.xmax`, respectively.

- a) Assign `sumThese` to `exp(rep(1000,10))`. Are they finite? Can you sum them? If everything is all good, assign `TRUE` to `allGood`.
- b) Theoretically, is the logarithm of the sum less than `.Machine$double.xmax`? Assign `TRUE` or `FALSE` to `noOverflowIssue`.
- c) Assign the *naive* log-sum of these to `naiveLogSum`. Is the naive log sum finite?
- d) Compute `betterSum`, one that doesn't overflow, using the *log-sum-exp* trick:

$$\log \left(\sum_{i=1}^{10} x_i \right) = \log \left(\sum_{i=1}^{10} \exp[\log(x_i) - m] \right) + m$$

m is usually chosen to be $\max_i \log x_i$. This is the same formula as above, which is nice. You can use the same code to combat both overflow and underflow. e) If you're writing code, and you have a bunch of very large numbers, is it better to store those numbers, or store the logarithm of those numbers? Assign your answer to `whichBetter`. Use either the phrase `"logs"` or `"nologs"`.

5.

Say you have a **vector** of prices of some financial asset:

```
prices <- c(100.10, 95.98, 100.01, 99.87)
```

- a) Use the natural logarithm and convert this vector into a vector of *log returns*. Call the variable `logReturns`. If p_t is the price at time t , the log return ending at time t is

$$r_t = \log \left(\frac{p_t}{p_{t-1}} \right) = \log p_t - \log p_{t-1} \quad (3.1)$$

- b) Do the same for *arithmetic returns*. These are regular percent changes if you scale by 100. Call the variable `arithReturns`. The mathematical formula you need is

$$a_t = \left(\frac{p_t - p_{t-1}}{p_{t-1}} \right) \times 100 \quad (3.2)$$

6.

Consider the **mixture density** $f(y) = \int f(y | x)f(x)dx$ where

$$Y | X = x \sim \text{Normal}(0, x^2) \quad (3.3)$$

and

$$X \sim \text{half-Cauchy}(0, 1). \quad (3.4)$$

This distribution is a special case of a prior distribution that is used in Bayesian statistics (Carvalho et al., 2009). Note that the second parameter of the Normal distribution is its variance, not its standard deviation.

Suppose further that you are interested in calculating the probability that one of these random variables ends up being too far from the median:

$$\mathbb{P}[|Y| > 1] = \int_{y:|y|>1} f(y)dy = \int_{y:|y|>1} \int_{-\infty}^{\infty} f(y | x)f(x)dx dy. \quad (3.5)$$

The following steps will demonstrate how you can use the **Monte-Carlo** (Robert and Casella, 2005) method to approximate this probability.

- a) Simulate $X_1, \dots, X_{5000}$ from a half-Cauchy(0, 1) and call these samples `xSamps`. Hint: you can simulate from a t distribution with one degree

of freedom to sample from a Cauchy. Once you have regular Cauchy samples, take the absolute value of each one.

- b) Simulate $Y_1 \mid X_1, \dots, Y_{5000} \mid X_{5000}$ and call the samples `ySamps`.
- c) Calculate the approximate probability using `ySamps` and call it `approxProbDev1`.
- d) Why is simply “ignoring” `xSamps` (i.e. not using it in the averaging part of the computation), the samples you condition on, “equivalent” to “integrating out x ”? Store a string response as a length 1 character vector called `integratingOutResp`.
- e) Calculate another **Rao-Blackwellized** Monte Carlo estimate of $\mathbb{P}[|Y| > 1]$ from `xSamps`. Call it `approxProbDev2`. Hint: $\mathbb{P}[|Y| > 1] = \mathbb{E}[\mathbb{P}(|Y| > 1 \mid X)]$. Calculate $\mathbb{P}(|Y| > 1 \mid X = x)$ with pencil and paper, notice it is a function in x , apply that function to each of `xSamps`, and average all of it together.
- f) Are you able to calculate an exact solution to $\mathbb{P}[|Y| > 1]$?

7.

Store the ordered uppercase letters of the alphabet in a length 26 character vector called `myUppcaseLetters`. Do not hardcode this. Use a function, along with the variable `letters`.

- a) Create a new vector called `withReplacements` that's the same as the previous vector, but replace all vowels with "---". Again, do not hardcode this. Find a function that searches for patterns and performs a replacement whenever that pattern is found.
- b) Create a length 26 logical vector that is `TRUE` whenever an element of `letters` is a consonant, and `FALSE` everywhere else. Call it `consonant`.

3.10.2 Python Questions

1.

Let's flip some coins (again)! Generate a thousand flips of a fair coin. Use `np.random.binomial`, and let heads be coded as 1 and tails coded as 0.

- a) Assign the thousand raw coin flips to a variable `flips`. Make sure the elements are integers, and make sure you flip a “fair” coin ($p = .5$).

- b) Create a length 1000 list of bools called `is_heads`. Whenever you get a heads, make sure the corresponding element is `True` and `False` otherwise.
- c) Create a variable called `num_heads` by tallying up the number of heads.
- d) Calculate the percent of time that the number changes in `flips`. Assign your number to `acceptance_rate`. Try to write only one line of code to do this.

2.

Create a Numpy array containing the numbers $1/2, 1/4, 1/8, \dots, 1/1024$. Make sure to call it `my_array`.

3.

Do the following:

- a) Create a `np.array` called `nums` that contains one hundred equally-spaced numbers starting from -100 and going to 100 (inclusive).
- b) Create a `bool np.array` that has the same length as the above, and contains `TRUE` whenever an element of the above is less than ten units away from 0, and `FALSE` otherwise. Call it `my_logicals`.
- c) Assign the total number of `Trues` to `total_close`.
- d) Create a `np.array` called `evens` of all even numbers from the above `np.array` (even numbers are necessarily integers).
- e) Create a `np.array` called `reverse` that holds the reversed elements of `nums`.

4.

Let's say we wanted to calculate the following sum $\sum_{i=1}^N x_i$. We run into problems when this sum is close to 0, too. This is the problem of *underflow*. The smallest positive floating point can be recovered by typing `np.nextafter(np.float64(0), np.float64(1))`.

- a) Assign `sum_these` to the length ten array $(e^{-1000}, \dots, e^{-1000})$. Use `np.exp(np.repeat(-1000, 10))`. Are the elements nonzero? Can you sum them? Is the sum correct? If everything is all good, assign `True` to `all_good`.
- b) Theoretically, for which range of positive numbers is the logarithm of the number farther from 0 than the number itself? Assign the lower bound to `lower_bound`, and the upper bound to `upper_bound`. Hint:

`lower_bound` is 0 because we're only looking at positive numbers, and because the logarithm is $-\infty$.

- c) Assign the *naive* log-sum of `sum_these` to `naive_log_sum`. Is the naive log sum finite on your computer? Should it be?
- d) Compute `better_sum`, one that doesn't underflow, using the *log-sum-exp* trick. This one should be bounded away from $-\infty$.

$$\log \left(\sum_{i=1}^{10} x_i \right) = \log \left(\sum_{i=1}^{10} \exp[\log(x_i) - m] \right) + m =$$

m is usually chosen to be $\max_i \log x_i$ e) If you're writing code, and you have a bunch of very small positive numbers (e.g. probabilities, densities, etc.), is it better to store those small numbers, or store the logarithm of those numbers? Assign your answer to `which_better`. Use either the phrase "logs" or "nologs".

5.

Use `pd.read_csv` to correctly read in "2013-10_Citi_Bike_trip_data_20K.csv" as a data frame called `my_df`. Make sure to read `autograding_tips.html`.

- a) Extract the "starttime" column into a separate Series called `s_times`.
- b) Extract date strings of those elements into a Series called `date_strings`.
- c) Extract time strings of those elements into a Series called `time_strings`.

6.

We will make use of the **Monte Carlo** method below. It is a technique to approximate expectations and probabilities. If n is a large number, and $X_1, \dots, X_n$ is a random sample drawn from the distribution of interest, then

$$\mathbb{P}(X > 6) \approx \frac{1}{n} \sum_{i=1}^n \mathbf{1}(X_i > 6). \quad (3.6)$$

If you haven't seen an **indicator function** before (see [Figure 3.1](#)), it is defined as

$$\mathbf{1}(X_i > 6) = \begin{cases} 1 & X_i > 6 \\ 0 & X_i \leq 6 \end{cases}. \quad (3.7)$$

If you wanted to visualize it, $\mathbf{1}(x > 6)$ looks like this.

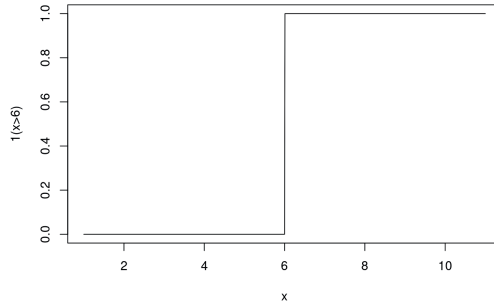


FIGURE 3.1: An indicator function.

So, the sum in this expression is just a count of the number of elements that are greater than 6.

- Evaluate exactly the probability that a normal random variable with mean 5 and standard deviation 6 is greater than 6. Assign it to the variable `exact_exceedance_prob` in Python.
- Simulate `1e3` times from a standard normal distribution (mean 0 and variance 1). Call the samples `stand_norm_samps`.
- Calculate a Monte Carlo estimate of $\mathbb{P}(X > 6)$ from these samples. Call it `approx_exceedance_prob1`.
- Simulate `1e3` times from a normal distribution with mean 5 and standard deviation 6. Call the samples `norm_samps`. Don't use the old samples in any way.
- Calculate a Monte Carlo estimate of $\mathbb{P}(X > 6)$ from these new `norm_samps`. Call it `approx_exceedance_prob2`.

7.

Alternatively, we can approximate expectations using the same technique as above. If $\mathbb{E}[g(X)]$ exists, n is a large number, and $W_1, \dots, W_n$ is a random sample drawn from the distribution of interest, then

$$\mathbb{E}[g(W)] \approx \frac{1}{n} \sum_{i=1}^n g(W_i). \quad (3.8)$$

Here's a new distribution. It is a **mixture distribution**, specifically a **finite mixture of normal distributions**. Its marginal density is plotted in [Figure 3.2](#). $f(y) = f(y | X = 1)P(X = 1) + f(y | X = 0)P(X = 0)$ where

$$Y \mid X = 0 \sim \text{Normal}(0, 2) \quad Y \mid X = 1 \sim \text{Normal}(10, 2) \quad (3.9)$$

and

$$X \sim \text{Bernoulli}(.5). \quad (3.10)$$

Both $f(y \mid X = 0)$ and $f(y \mid X = 1)$ are bell-curved, and $f(y)$ looks like this

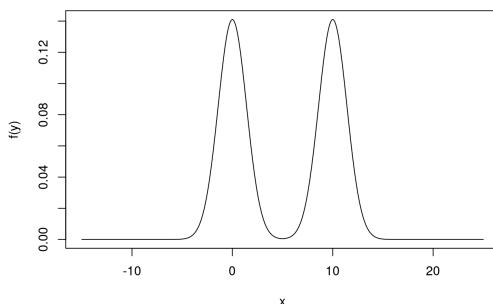


FIGURE 3.2: The marginal density of Y .

- Evaluate exactly $\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}(Y \mid X)]$. Assign it to the variable `exact_mean` in Python.
- Simulate `1e3` times from the Bernoulli distribution. Call the samples `bernoulli_flips`
- Simulate $Y_1 \mid X_1, \dots, Y_{1000} \mid X_{1000}$ and call the samples `cond_norm_samps`.
- Calculate a Monte Carlo estimate of $\mathbb{E}[Y]$ from `cond_norm_samps`. Call it `approx_ave_1`. Why is simply “ignoring” `bernoulli_flips`, the samples you condition on, “equivalent” to “integrating them out?”
- Calculate a **Rao-Blackwellized** Monte Carlo estimate of $\mathbb{E}[Y]$ from `bernoulli_flips`. Call it `approx_ave_2`. Hint: $\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}(Y \mid X)]$. Calculate $\mathbb{E}(Y \mid X_i)$ exactly, and evaluate that function on each X_i sample, and then average them together. Rao-Blackwellization is a variance-reduction technique that allows you come up with lower-variance estimates given a fixed computational budget.

Numpy ndarrays versus R's Matrix and Array Types

Sometimes you want a collection of elements that are *all the same type*, but you want to store them in a two- or three-dimensional structure. For instance, say you need to use matrix multiplication for some linear regression software you're writing, or that you need to use tensors for a computer vision project you're working on.

4.1 Numpy `ndarrays` in Python

In Python, you could still use arrays for these kinds of tasks. You will be pleased to learn that the Numpy arrays we discussed earlier are a special case of Numpy's N-dimensional arrays¹. Each array will come with an enormous amount of methods² and attributes³ (more on object-oriented program in [Chapter 14](#)) attached to it. A few are demonstrated below.

```
import numpy as np
a = np.array([[1,2],[3,4]], np.float)
a
## array([[1., 2.],
##        [3., 4.]])
a.shape
## (2, 2)
a.ndim
## 2
a.dtype
## dtype('float64')
a.max()
## 4.0
a.resize((1,4)) # modification is **in place**
```

¹<https://numpy.org/doc/stable/reference/arrays.ndarray.html>

²<https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-methods>

³<https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-attributes>

```
a
## array([[1., 2., 3., 4.]])
```

Matrix and elementwise multiplication is often useful, too.

```
b = np.ones(4).reshape((4,1))
np.dot(b,a) # matrix mult.
## array([[1., 2., 3., 4.],
##        [1., 2., 3., 4.],
##        [1., 2., 3., 4.],
##        [1., 2., 3., 4.]])
b @ a # infix matrix mult. from PEP 465
## array([[1., 2., 3., 4.],
##        [1., 2., 3., 4.],
##        [1., 2., 3., 4.],
##        [1., 2., 3., 4.]])
a * np.arange(4) # elementwise mult.
## array([[ 0.,  2.,  6., 12.]])
```

I should mention that there is also a `matrix` type in Numpy; however, this is not described in this text because it is preferable to work with Numpy arrays (Albon, 2018).



In both R and Python, there are `matrix` types and `array` types. In R, it is more common to work with `matrix`s than `arrays`, and the opposite is true in Python!

4.2 The `Matrix` and `Array` Classes in R

In Python, adding a dimension to your “container” is simple. You keep using Numpy arrays, and you just change the `.shape` attribute (perhaps with a call to `.reshape()` or something similar). In R, there is a stronger distinction between 1-, 2-, and 3-dimensional containers. Each has its own class. 2-dimensional containers that store objects of the same type are of the `matrix` class. Containers with 3 or more dimensions are of the `array` class⁴. In this section, I will provide

⁴Technically, the distinction between all of these containers is more subtle. An `array` in R can have one, two or more dimensions, and it is just a vector which is stored with additional dimension attributes. Moreover, a 2-dimensional array is the same as a `matrix`.

a quick introduction to using these two classes. For more information, see [Chapter 3](#) of (Matloff, 2011).

Just like `vectors`, `matrix` objects do not necessarily have to be used to perform matrix arithmetic. Yes, they require all the elements are of the same type, but it doesn't really make sense to “multiply” `matrix` objects that hold onto characters.

I usually create `matrix` objects with the `matrix()` function or the `as.matrix()` function. `matrix()` is to be preferred in my opinion. The first argument is explicitly a `vector` of all the flattened data that you want in your `matrix`. On the other hand, `as.matrix()` is more flexible; it takes in a variety of R objects (e.g. `data.frames`), and tries to figure out what to do with them on a case-by-case basis. In other words, `as.matrix()` is a *generic function*. More information about generic functions is provided in [Section 14.2.2](#).

Some other things to remember with `matrix()`: `byrow=` is `FALSE` by default, and you will also need to specify either `ncol=` and/or `nrow=` if you want anything that isn't a 1-column `matrix`.

```
A <- matrix(1:4)
A
##           [,1]
## [1,]      1
## [2,]      2
## [3,]      3
## [4,]      4
matrix(1:4, ncol = 2)
##           [,1] [,2]
## [1,]      1    3
## [2,]      2    4
matrix(1:4, ncol = 2, byrow = T)
##           [,1] [,2]
## [1,]      1    2
## [2,]      3    4
as.matrix(
  data.frame(
    firstCol = c(1,2,3),
    secondCol = c("a","b","c"))) # coerces numbers to characters!
##           firstCol secondCol
## [1,]  "1"         "a"
## [2,]  "2"         "b"
```

```
## [3,] "3"      "c"
dim(A)
## [1] 4 1
nrow(A)
## [1] 4
ncol(A)
## [1] 1
```

`array()` is used to create `array` objects. This type is used less than the `matrix` type, but this doesn't mean you should avoid learning about it. This is mostly a reflection of what kind of data sets people prefer to work with, and the fact that matrix algebra is generally better understood than tensor algebra. You won't be able to avoid 3-d data sets (3-dimensions, not a 3-column `matrix`) forever, though, particularly if you're working in an area such as neuroimaging or computer vision.

```
myArray <- array(rep(1:3, each = 4), dim = c(2,2,3))
myArray
## , , 1
##
##      [,1] [,2]
## [1,]    1    1
## [2,]    1    1
##
## , , 2
##
##      [,1] [,2]
## [1,]    2    2
## [2,]    2    2
##
## , , 3
##
##      [,1] [,2]
## [1,]    3    3
## [2,]    3    3
```

You can matrix-multiply `matrix` objects together with the `%*%` operator. If you're working on this, then the transpose operator (i.e. `t()`) comes in handy, too. You can still use element-wise (Hadamard) multiplication. This is defined with the more familiar multiplication operator `*`.

```
# calculate a quadratic form y'Qy
y <- matrix(c(1,2,3))
Q <- diag(1, 3) # diag() gets and sets diagonal matrices
t(y) %*% Q %*% y
##      [,1]
## [1,]    14
```

Sometimes you need to access or modify individual elements of a `matrix` object. You can use the familiar `[]` and `[-]` operators to do this. Here is a setting example. You don't need to worry about coercion to different types here.

```
Qcopy <- Q
Qcopy[1,1] <- 3
Qcopy[2,2] <- 4
Qcopy
##      [,1] [,2] [,3]
## [1,]    3    0    0
## [2,]    0    4    0
## [3,]    0    0    1
```

Here are some extraction examples. Notice that, if it can, `[]` will coerce a `matrix` to `vector`. If you wish to avoid this, you can specify `drop=FALSE`.

```
Q
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
Q[1,1]
## [1] 1
Q[2,]
## [1] 0 1 0
Q[2,,drop=FALSE]
##      [,1] [,2] [,3]
## [1,]    0    1    0
class(Q)
## [1] "matrix" "array"
class(Q[2,])
## [1] "numeric"
class(Q[2,,drop=FALSE])
```



```
## [1] "matrix" "array"
row(Q) > 1
##      [,1] [,2] [,3]
## [1,] FALSE FALSE FALSE
## [2,]  TRUE  TRUE  TRUE
## [3,]  TRUE  TRUE  TRUE
Q[row(Q) > 1] # column-wise ordering
## [1] 0 0 1 0 0 1
```

There are other functions that operate on one or more `matrix` objects in more interesting ways, but much of this will be covered in future sections. For instance, we will describe how `apply()` works with `matrix`s in Section 15, and we will discuss combining `matrix` objects in different ways in Section 12.

4.3 Exercises

4.3.1 R Questions

1.

Consider the following data set. Let $N = 20$ be the number of rows. For $i = 1, \dots, N$, define $\mathbf{x}_i \in \mathbb{R}^4$ as the data in row i .

```
d <- matrix(c(
  -1.1585476,  0.06059602, -1.854421163,  1.62855626,
  0.5619835,  0.74857327, -0.830973409,  0.38432716,
  -1.6949202,  1.24726626,  0.068601035, -0.32505127,
  2.8260260, -0.68567999, -0.109012111, -0.59738648,
  -0.3128249, -0.21192009, -0.317923437, -1.60813901,
  0.3830597,  0.68000706,  0.787044622,  0.13872087,
  -0.2381630,  1.02531172, -0.606091651,  1.80442260,
  1.5429671, -0.05174198, -1.950780046, -0.87716787,
  -0.5927925, -0.40566883, -0.309193162,  1.25575250,
  -0.8970403, -0.10111751,  1.555160257, -0.54434356,
  2.4060504, -0.08199934, -0.472715155,  0.25254794,
  -1.0145770, -0.83132666, -0.009597552, -1.71378699,
  -0.3590219,  0.84127504,  0.062052945, -1.00587841,
  -0.1335952, -0.02769315, -0.102229046, -1.08526057,
  0.1641571, -0.08308289, -0.711009361,  0.06809487,
```

```
2.2450975, 0.32619749, 1.280665384, 1.75090469,
1.2147885, 0.10720830, -2.018215962, 0.34602861,
0.7309219, -0.60083707, -1.007344145, -1.77345958,
0.1791807, -0.49500051, 0.402840566, 0.60532646,
1.0454594, 1.09878293, 2.784986486, -0.22579848), ncol = 4)
```

For the following problems, make sure to only use the transpose function `t()`, matrix multiplication (i.e. `%*%`), and scalar multiplication/division. You may use other functions in interactive mode to check your work, but please do not use them in your submission.

- a) Calculate the sample mean $\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$. Check your work with `colMeans()`, **but don't use that function in your submitted code**. Assign it to the variable `xbar`. Make sure it is a 4×1 matrix object.
- b) Calculate the 4×4 sample covariance of the following data. Call the variable `mySampCov`, and make sure it is also a matrix object. **You can check your work with `cov()`, but don't use it in your submitted code**. A formula for the sample covariance is

$$\frac{1}{N-1} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \quad (4.1)$$

2.

Create a matrix called `P` that has 100 rows, 100 columns, all of its elements nonnegative, $1/10$ on every diagonal element, and all rows summing to one. This matrix is called **stochastic** and it describes how a Markov chain moves randomly through time.

3.

Create a matrix called `x` that has one thousand rows, four columns, has every element set to either 0 or 1, has its first column set to all 1s, has the second column set to 1 in the second 250 elements and 0 elsewhere, has the third column set to 1 in the third 250 spots and 0 elsewhere, and has the fourth column set to 1 in the last 250 spots and 0 elsewhere. In other words, it looks something like

$$\begin{bmatrix} \mathbf{1}_{250} & \mathbf{0}_{250} & \mathbf{0}_{250} & \mathbf{0}_{250} \\ \mathbf{1}_{250} & \mathbf{1}_{250} & \mathbf{0}_{250} & \mathbf{0}_{250} \\ \mathbf{1}_{250} & \mathbf{0}_{250} & \mathbf{1}_{250} & \mathbf{0}_{250} \\ \mathbf{1}_{250} & \mathbf{0}_{250} & \mathbf{0}_{250} & \mathbf{1}_{250} \end{bmatrix} \quad (4.2)$$

where $\mathbf{1}_{250}$ and $\mathbf{0}_{250}$ are length 250 column vectors with all of their elements set to 1 or 0, respectively.

- a) Compute the **projection (or hat) matrix** $\mathbf{H} := \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. Make it a `matrix` and call it `H`.
- b) An **exchangeable** covariance matrix for a random vector is a covariance matrix that has all the same variances, and all the same covariances. In other words, it has two unique elements: the diagonal elements should be the same, and the off-diagonals should be the same. In R, generate ten 100×100 **exchangeable** covariance matrices, each with 2 as the variance, and have the possible covariances take values in the collection 0, .01, .02, ..., .09. Store these ten covariance matrices in a three-dimensional array. The first index should be each matrix's row index, the second should be the column index of each matrix, and the third index should be the "layer" or "slice" indicating which of the 10 matrices you have. Name this array `myCovMats`
- c) In R, generate one hundred 10×10 **exchangeable** covariance matrices, each with 2 as the variance, and have the possible covariances take values in the collection 0, 0.0009090909, ..., 0.0890909091, .09. Store these 100 covariance matrices in a three-dimensional array. The first index should be each matrix's row index, the second should be the column index of each matrix, and the third index should be the "layer" or "slice" indicating which of the 100 matrices you have. Name this array `myCovMats2`

4.3.2 Python Questions

1.

Let $\mathbf{X}$ be an $n \times 1$ random vector. It has a multivariate normal distribution with mean vector $\mathbf{m}$ and positive definite covariance matrix $\mathbf{C}$ if its probability density function can be written as

$$f(\mathbf{x}; \mathbf{m}, \mathbf{C}) = (2\pi)^{-n/2} \det(\mathbf{C})^{-1/2} \exp \left[-\frac{1}{2} (\mathbf{x} - \mathbf{m})^\top \mathbf{C}^{-1} (\mathbf{x} - \mathbf{m}) \right] \quad (4.3)$$

Evaluating this density should be done with care. There is no one function that is optimal for all situations. Here are a couple quick things to consider.

- Inverting very large matrices with either `np.linalg.solve`⁵ or `np.linalg.inv`⁶ becomes very slow if the covariance matrix is high-dimensional. If you have special assumptions about the structure of the covariance matrix, use it! Also, it's a good idea to be aware of what happens when you try to invert noninvertible matrices. For instance, can you rely on errors to be thrown, or will it return a bogus answer?
- Recall from the last lab that exponentiating numbers close to $-\infty$ risks numerical underflow. It's better to prefer evaluating log densities (base e , the natural logarithm). There are also special functions that evaluate log determinants⁷ that are less likely to underflow/overflow, too!

Complete the following problems. **Do not use pre-made functions such as `scipy.stats.norm`⁸ and `scipy.stats.multivariate_normal`⁹ in your submission, but you may use them to check your work. Use only “standard” functions and Numpy n-dimensional arrays. Use the following definitions for `x` and `m`:**

```
import numpy as np
x = np.array([1.1, .9, 1.0]).reshape((3,1))
m = np.ones(3).reshape((3,1))
```

a) Let $\mathbf{C} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 10 \end{bmatrix}$. Evaluate and assign the log density to a float-like called `log_dens1`. Can you do this without defining a numpy array for `C`?

b) Let $\mathbf{C} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 11 & 0 \\ 0 & 0 & 12 \end{bmatrix}$. Evaluate and assign the log density to a float-like called `log_dens2`. Can you do this without defining a numpy array for `C`?

⁵<https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html>

⁶<https://numpy.org/doc/stable/reference/generated/numpy.linalg.inv.html>

⁷<https://numpy.org/doc/stable/reference/generated/numpy.linalg.slogdet.html>

⁸<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.norm.html>

⁹https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.multivariate_normal.html

- c) Let $\mathbf{C} = \begin{bmatrix} 10 & -0.9 & -0.9 \\ -0.9 & 11 & -0.9 \\ -0.9 & -0.9 & 12 \end{bmatrix}$. Evaluate and assign the log density to a float-like called `log_dens3`. Can you do this without defining a numpy array for $\mathbf{C}$?

2.

Consider this wine data set¹⁰ from (Cortez et al., 2009) hosted by (Dua and Graff, 2017). Read it in with the following code. Note that you might need to use `os.chdir()` first.

```
import pandas as pd
d = pd.read_csv("winequality-red.csv", sep = ";")
d.head()
```

- a) Create the **design matrix** (denoted mathematically by $\mathbf{X}$) by removing the "quality" column, and subtracting the column mean from each element. Call the variable `x`, and make sure that it is a Numpy ndarray, not a Pandas DataFrame.
- b) Compute the **spectral decomposition** of $\mathbf{X}^\top \mathbf{X}$. In other words, find "special" matrices¹¹ $\mathbf{V}$ and Λ such that $\mathbf{X}^\top \mathbf{X} = \mathbf{V} \Lambda \mathbf{V}^\top$. Note that the *eigenvectors* are stored as columns in a matrix $\mathbf{V} := [\mathbf{V}_1 \ \dots \ \mathbf{V}_{11}]$, and the scalar *eigenvalues* are stored as diagonal elements $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_{11})$. Store the eigenvectors in an ndarray called `eig_vecs`, and store the eigenvalues in a Numpy array called `eig_vals`. Hint: use `np.linalg.eig()`¹². Also, if you're rusty with your linear algebra, don't worry too much about refreshing your memory about what eigenvectors and eigenvalues are.
- c) Compute the **singular value decomposition** of $\mathbf{X}$. In other words, find "special"¹³ matrices $\mathbf{U}$, Σ , and $\mathbf{V}$ such that $\mathbf{X} = \mathbf{U} \mathbf{V}^\top$. Use `np.linalg.svd`¹⁴, and don't worry too much about the mathematical details. These two decompositions are related. If you do it correctly, the two $\mathbf{V}$ matrices should be the same, and the elements of Σ should be the square roots of the elements of Λ . Store the eigenvectors as

¹⁰<https://archive.ics.uci.edu/ml/datasets/Wine+Quality>

¹¹Do not worry too much about the properties of these matrices for this problem

¹²<https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html>

¹³Again, do not worry too much about the properties of these matrices for this problem.

¹⁴<https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>

columns in an ndarray called `eig_vecs_v2`, and store the singular values (diagonal elements of Σ) in a Numpy array called `sing_vals`.

- d) Compute the **first principal component** vector, and call it `first_pc_v1`. The mathematical formula is $\mathbf{X}\mathbf{U}_1$ where $\mathbf{U}_1$ is the eigenvector associated with the largest eigenvalue λ_1 . This can be thought of as, in a sense, the most informative predictor that you can create by averaging together all other predictors.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

R's lists versus Python's lists and dicts

When you need to store elements in a container, but you can't guarantee that these elements all have the same type, or you can't guarantee that they all have the same size, then you need a `list` in R. In Python, you might need a `list` or `dict` (short for dictionary) (Lutz, 2013).

5.1 `lists` in R

`lists` are one of the most flexible data types in R. You can access individual elements in many different ways, each element can be of different size, and each element can be of a different type.

```
myList <- list(c(1,2,3), "May 5th, 2021", c(TRUE, TRUE, FALSE))
myList[1] # length-1 list; first element is length 3 vector
## [[1]]
## [1] 1 2 3
myList[[1]] # length-3 vector
## [1] 1 2 3
```

If you want to extract an element, you need to decide between using single square brackets or double square brackets. The former returns a `list`, while the second returns the type of the individual element.

You can also name the elements of a list. This can lead to more readable code. To see why, examine the example below that makes use of some data about cars (sas, 2021). The `lm()` function estimates a linear regression model. It returns a `list` with plenty of components.

```
dataSet <- read.csv("data/cars.csv")
results <- lm(log(Horsepower) ~ Type, data = dataSet)
length(results)
## [1] 13
```



```
# names(results) # try this <-
results$contrasts
## $Type
## [1] "contr.treatment"
results['rank']
## $rank
## [1] 6
```

! ■ `results` is a list (because `is.list(results)` returns `TRUE`), but to be more specific, it is an S3 object of class `lm`. If you do not know what this means, do not worry! S3 classes are discussed more in a later chapter. Why is this important? For one, I mention it so that you aren't confused if you type `class(results)` and see `lm` instead of `list`. Second, the fact that the authors of `lm()` wrote code that returns `result` as a “fancy list” suggests that they are encouraging another way to access elements of the `results`: to use specialized functions! For example, you can use `residuals(results)`, `coefficients(results)`, and `fitted.values(results)`. These functions do not work for all lists in R, but when they do work (for `lm` and `glm` objects only), you can be sure you are writing the kind of code that is encouraged by the authors of `lm()`.

5.2 lists in Python

Python lists¹ are very flexible, too. There are fewer choices for accessing and modifying elements of lists in Python—you'll most likely end up using the square bracket operator. Elements can be different sizes and types, just like they were with R's lists.

Unlike in R, however, you cannot name elements of lists. If you want a container that allows you to access elements by name, look into Python dictionaries² (see [Section 5.3](#)) or Pandas' `Series` objects (see [Section 3.2](#)).

From the example below, you can see that we've been introduced to lists already. We have been constructing Numpy arrays from them.

¹<https://docs.python.org/3/library/stdtypes.html#lists>

²<https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

```
import numpy as np
another_list = [np.array([1,2,3]), "May 5th, 2021", True, [42,42]]
another_list[2]
## True
another_list[2] = 100
another_list
## [array([1, 2, 3]), 'May 5th, 2021', 100, [42, 42]]
```

Python lists have methods attached to them³, which can come in handy.

```
another_list
## [array([1, 2, 3]), 'May 5th, 2021', 100, [42, 42]]
another_list.append('new element')
another_list
## [array([1, 2, 3]), 'May 5th, 2021', 100, [42, 42], 'new element']
```

Creating lists can be done as above, with the square bracket operators. They can also be created with the `list()` function, and by creating a *list comprehension*. List comprehensions are discussed more in [Section 11.2](#).

```
my_list = list(('a','b','c')) # converting a tuple to a list
your_list = [i**2 for i in range(3)] # list comprehension
my_list
## ['a', 'b', 'c']
your_list
## [0, 1, 4]
```

The code above makes reference to a type that is not extensively discussed in this text: *tuples*⁴.

5.3 Dictionaries in Python

Dictionaries⁵ in Python provide a container of key-value pairs. The keys are *unique*, and they must be *immutable*. *strings* are the most common key type, but *ints* can be used as well.

³<https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

⁴<https://docs.python.org/3.3/library/stdtypes.html?highlight=tuple#tuple>

⁵<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

Here is an example of creating a `dict` with curly braces (i.e. `{}`). This `dict` stores the current price of a few popular cryptocurrencies. Accessing an individual element's value using its key is done with the square bracket operator (i.e. `[]`), and deleting elements is done with the `del` keyword.

```
crypto_prices = {'BTC': 38657.14, 'ETH': 2386.54, 'DOGE': .308122}
crypto_prices['DOGE'] # get the current price of Dogecoin
## 0.308122
del crypto_prices['BTC'] # remove the current price of Bitcoin
crypto_prices.keys()
## dict_keys(['ETH', 'DOGE'])
crypto_prices.values()
## dict_values([2386.54, 0.308122])
```

You can also create `dicts` using **dictionary comprehensions**. Just like list comprehensions, these are discussed more in [Section 11.2](#).

```
incr_cryptos = {key:val*1.1 for (key,val) in crypto_prices.items()}
incr_cryptos
## {'ETH': 2625.194, 'DOGE': 0.3389342}
```

Personally, I don't use dictionaries as much as lists. If I have a dictionary, I usually convert it to a Pandas data frame (more information on those in [Section 8.2](#)).

```
import pandas as pd
a_dict = { 'col1': [1,2,3], 'col2' : ['a','b','c']}
df_from_dict = pd.DataFrame(a_dict)
df_from_dict
##      col1 col2
## 0      1    a
## 1      2    b
## 2      3    c
```

5.4 Exercises

5.4.1 R Questions

1.

Consider the data sets "adult.data", "car.data", "hungarian.data", "iris.data", "long-beach-va.data" and "switzerland.data" (Janosi et al., 1988), (Fisher, 1988), (mis, 1996) and (mis, 1997) hosted by (Dua and Graff, 2017). Read all of these in and store them all as a list of data.frames. Call the list `listDfs`.

2.

Here are two lists in R:

```
l1 <- list(first="a", second=1)
l2 <- list(first=c(1,2,3), second = "statistics")
```

- a) Make a new list that is these two lists above “squished together.” It has to be length 4, and each element is one of the elements of `l1` and `l2`. Call this list `l3`. Make sure to delete all the “tags” or “names” of these four elements.
- b) Extract the third element of `l3` as a length one list and assign it to the name `l4`.
- c) Extract the third element of `l3` as a vector and assign it to the name `v1`.

5.4.2 Python Questions

1.

Read in `car.data` with `pd.read_csv()`, and use a `DataFrame` method to convert that to a dict. Store your answer as `car_dict`.

2.

Here are two dicts in Python:

```
d1 = { "first" : "a", "second" : 1}
d2 = { "first" : [1,2,3], "second" : "statistics"}
```

- a) Make a new `list` that is these two `dicts` above “squished together” (why can’t it be another `dict`?) It has to be length 4, and each value is one of the values of `d1` and `d2`. Call this list `my_list`.
- b) Use a list comprehension to create a list called `special_list` of all numbers starting from zero, up to (and including) one million, but don’t include numbers that are divisible by any prime number less than seven.
- c) Assign the average of all elements in the above list to the variable `special_ave`.

6

Functions

This text has already covered how to *use* functions that come to us pre-made. At least we have discussed how to use them in a one-off way—just write the name of the function, write some parentheses after that name, and then plug in any requisite arguments by writing them in a comma-separated way between those two parentheses. This is how it works in both R and Python.

In this section we take a look at how to *define* our own functions. This will not only help us to understand pre-made functions, but it will also be useful if we need some extra functionality that isn't already provided to us.

Writing our own functions is also useful for “packaging up” computations. The utility of this will become apparent very soon. Consider the task of estimating a regression model. If you have a function that performs all of the required calculations, then

- you can estimate models without having to think about lower-level details or write any code yourself, and
- you can re-use this function every time you fit any model on any data set for any project.

6.1 Defining R Functions

To create a function in R, we need another function called `function()`. We give the output of `function()` a name in the same way we give names to any other variable in R, by using the assignment operator `<-`. Here's an example of a toy function called `addOne()`. Here `myInput` is a placeholder that refers to whatever the user of the function ends up plugging in.

```
addOne <- function(myInput){ # define the function
  myOutput <- myInput + 1
  return(myOutput)
}
```

```
addOne(41) # call/invoke/use the function
## [1] 42
```

Below the definition, the function is called with an input of 41. When this happens, the following sequence of events occurs

- The value 41 is assigned to `myInput`
- `myOutput` is given the value 42
- `myOutput`, which is 42, is returned from the function
- the temporary variables `myInput` and `myOutput` are destroyed.

We get the desired answer, and all the unnecessary intermediate variables are cleaned up and thrown away after they are no longer needed.

■ If you are interested in writing a function, I recommend that you first write the logic outside of a function. This initial code will be easier to debug because your temporary variables will not be destroyed after the final result has been obtained. Once you are happy with the working code, you can copy and paste the logic into a function definition, and replace permanent variables with function inputs like `myInput` above.

6.2 Defining Python Functions

To create a function in Python, we use the `def` statement (instead of the `function()` function in R). The desired name of the function comes next. After that, the formal parameters come, comma-separated inside parentheses, just like in R.

Defining a function in Python is a little more concise. There is no assignment operator like there is in R, there are no curly braces, and `return` isn't a function like it is in R, so there is no need to use parentheses after it. There is one syntactic addition, though—we need a colon (`:`) at the end of the first line of the definition.

Here is an example of a toy function called `add_one()`.

```
def add_one(my_input): # define the function
    my_output = my_input + 1
```

```
    return my_output
add_one(41) # call/invoke/use the function
## 42
```

Below the definition, the function is called with an input of 41. When this happens, the following sequence of events occurs

- The value 41 is assigned to `my_input`
- `my_output` is given the value 42
- `my_output`, which is 42, is returned from the function
- the temporary variables `my_input` and `my_output` are destroyed.

We get the desired answer, and all the unnecessary intermediate variables are cleaned up and thrown away after they are no longer needed.

6.3 More Details on R's User-Defined Functions

Technically, in R, functions are defined as three things bundled together¹:

1. a **formal argument list** (also known as *formals*),
2. a **body**, and
3. a **parent environment**.

The *formal argument list* is exactly what it sounds like. It is the list of arguments a function takes. You can access a function's formal argument list using the `formals()` function. Note that it is not the *actual* arguments a user will plug in—that isn't knowable at the time the function is created in the first place.

Here is another function that takes a parameter called `whichNumber` that comes with a **default argument** of 1. If the caller of the function does not specify what she wants to add to `myInput`, `addNumber()` will use 1 as the default. This default value shows up in the output of `formals(addNumber)`.

```
addNumber <- function(myInput, whichNumber = 1){
  myOutput <- myInput + whichNumber
  return(myOutput)
}
addNumber(3) # no second argument being provided by the user here
```

¹<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Function-objects>


```
## [1] 4
formals(addNumber)
## $myInput
##
##
## $whichNumber
## [1] 1
```

The function's *body* is also exactly what it sounds like. It is the work that a function performs. You can access a function's body using the `body()` function.

```
addNumber <- function(myInput, whichNumber = 1){
  myOutput <- myInput + whichNumber
  return(myOutput)
}
body(addNumber)
## {
##   myOutput <- myInput + whichNumber
##   return(myOutput)
## }
```

Every function you create also has a *parent environment*². You can get/set this using the `environment()` function. Environments help a function know which variables it is allowed to use and how to use them. The parent environment of a function is where the function was *created*, and it contains variables outside of the body that the function can also use. The rules of which variables a function can use are called *scoping*. When you create functions in R, you are primarily using **lexical scoping**. This is discussed in more detail in [Section 6.5](#).



There is a lot more information about environments that isn't provided in this text. For instance, a user-defined function also has binding, execution, and calling environments associated with it^a, and environments are used in creating package namespaces, which are important when two packages each have a function with the same name.

^a<http://adv-r.had.co.nz/Environments.html#function-envs>

²Primitive functions are functions that contain no R code and are internally implemented in C. These are the only type of function in R that don't have a parent environment.

6.4 More Details on Python's User-Defined Functions

Roughly, Python functions have the same things R functions have. They have a **formal parameter list**, a body, and there are namespaces³ created that help organize which variables the function can access, as well as which pieces of code can call this new function. A namespace is just a “mapping from names to objects.”

These three concepts are analogous to those in R. The names are just a bit different sometimes, and it isn't organized in the same way. To access these bits of information, you need to access the *special attributes* of a function. User-defined functions in Python have a lot of pieces of information attached to them. If you'd like to see all of them, you can visit this page of documentation⁴.

So, for instance, let's try to find the *formal parameter list* of a user-defined function below. This is, again, the collection of inputs a function takes. Just like in R, this is not the *actual* arguments a user will plug in—that isn't knowable at the time the function is created.⁵ Here we have another function called `add_number()` that takes a parameter `which_number` that is accompanied by a default argument of 1.

```
def add_number(my_input, which_number = 1): # define a function
    my_output = my_input + which_number
    return my_output
add_number(3) # no second argument being provided by the user here
## 4
add_number.__code__.co_varnames # note this also contains *my_output*
## ('my_input', 'which_number', 'my_output')
add_number.__defaults__
## (1,)
```

The `__code__` attribute has much more to offer. To see a list of names of all its contents, you can use `dir(add_number.__code__)`.

³<https://docs.python.org/3/tutorial/classes.html#python-scopes-and-namespaces>

⁴<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

⁵You might have noticed that Python uses two different words to prevent confusion. Unlike R, Python uses the word “parameter” (instead of “argument”) to refer to the inputs a function takes, and “arguments” to the specific values a user plugs in.



Don't worry if the notation `add_number.__code__` looks strange. The dot (`.`) operator will become more clear in the future chapter on *object-oriented programming*. For now, just think of `__code__` as being an object *belonging to* `add_number`. Objects that belong to other objects are called **attributes** in Python. The dot operator helps us access attributes *inside* other objects. It also helps us access objects belonging to modules that we **import** into our scripts.

6.5 Function Scope in R

R uses **lexical scoping**. This means, in R,

1. functions can use *local* variables that are defined inside themselves,
2. functions can use *global* variables defined in the environment where the function itself was *defined* in, and
3. functions *cannot* necessarily use *global* variables defined in the environment where the function was *called* in, and
4. functions will prefer *local* variables to *global* variables if there is a name clash.

The first characteristic is obvious. The second and third are import to distinguish between. Consider the following code below. `sillyFunction()` can access `a` because `sillyFunction()` and `a` are defined in the same place.

```
a <- 3
sillyFunction <- function(){
  return(a + 20)
}
environment(sillyFunction) # the env. it was defined in contains a
## <environment: R_GlobalEnv>
sillyFunction()
## [1] 23
```

On the other hand, the following example will not work because `a` and `anotherSillyFunc()` are not defined in the same place. Calling the function is not the same as defining a function.

```

anotherSillyFunc <- function(){
  return(a + 20)
}
highLevelFunc <- function(){
  a <- 99
  # this isn't the global environment anotherSillyFunc() was defined in
  cat("environment inside highLevelFunc(): ", environment())
  anotherSillyFunc()
}

```

Finally, here is a demonstration of a function preferring one `a` over another. When `sillyFunction()` attempts to access `a`, it first looks in its own body, and so the innermost one gets used. On the other hand, `print(a)` shows 3, the global variable.

```

a <- 3
sillyFunction <- function(){
  a <- 20
  return(a + 20)
}
sillyFunction()
## [1] 40
print(a)
## [1] 3

```

The same concept applies if you create functions within functions. The inner function `innerFunc()` looks “inside-out” for variables, but only in the place it was defined.

Below we call `outerFunc()`, which then calls `innerFunc()`. `innerFunc()` can refer to the variable `b`, because it lies in the same environment in which `innerFunc()` was created. Interestingly, `innerFunc()` can also refer to the variable `a`, because that variable was captured by `outerFunc()`, which provides access to `innerFunc()`.

```

a <- "outside both"
outerFunc <- function(){
  b <- "inside one"
  innerFunc <- function(){
    print(a)
    print(b)
  }
}

```

```

    }
    return(innerFunc())
}
outerFunc()
## [1] "outside both"
## [1] "inside one"

```

Here's another interesting example. If we ask `outerFunc()` to return the function `innerFunc()` (not the return object of `innerFunc()`...functions are objects, too!), then we might be surprised to see that `innerFunc()` can still successfully refer to `b`, even though it doesn't exist inside the *calling environment*. But don't be surprised! What matters is what was available when the function was *created*.

```

outerFuncV2 <- function(){
  b <- "inside one"
  innerFunc <- function(){
    print(b)
  }
  return(innerFunc) # note the missing inner parentheses!
}
myFunc <- outerFuncV2() # get a new function
ls(environment(myFunc)) # list all data attached to this function
## [1] "b"                "innerFunc"
myFunc()
## [1] "inside one"

```

We use this property all the time when we create functions that return other functions. This is discussed in more detail in [Chapter 15](#). In the above example, `outerFuncV2()`, the function that returned another function, is called a *function factory*.



Sometimes people will refer to R's functions as **closures** to emphasize that they are capturing variables from the parent environment in which they were created, to emphasize the data that they are bundled with.

6.6 Function Scope in Python

Python uses **lexical scoping** just like R. This means, in Python,

1. functions can use *local* variables that are defined inside themselves,
2. functions have an order of preference for which variable to prefer in the case of a name clash, and
3. functions can sometimes use variables defined outside itself, but that ability depends on where the function and variable were *defined*, not where the function was *called*.

Regarding characteristics (2) and (3), there is a famous acronym that describes the rules Python follows when finding and choosing variables: **LEGB**.

- L: Local,
- E: Enclosing,
- G: Global, and
- B: Built-in.

A Python function will search for a variable in these namespaces in this order.⁶

“*Local*” refers to variables that are defined inside of the function’s block. The function below uses the local `a` over the global one.

```
a = 3
def silly_function():
    a = 22 # local a
    print("local variables are ", locals())
    return a + 20
silly_function()
## local variables are  {'a': 22}
## 42
silly_function.__code__.co_nlocals # number of local variables
## 1
silly_function.__code__.co_varnames # names of local variables
## ('a',)
```

⁶Functions aren’t the only thing that get their own namespace. Classes do, too.⁷ More information on classes is provided in [Chapter 14](#)

⁷<https://docs.python.org/3/tutorial/classes.html#a-first-look-at-classes>

“*Enclosing*” refers to variables that were defined in the enclosing namespace, but not the global namespace. These variables are sometimes called **free variables**. In the example below, there is no local `a` variable for `inner_func()`, but there is a global one, and one in the enclosing namespace. `inner_func()` chooses the one in the enclosing namespace. Moreover, `inner_func()` has its own copy of `a` to use, even after `a` was initially destroyed upon the completion of the call to `outer_func()`.

```
a = "outside both"
def outer_func():
    a = "inside one"
    def inner_func():
        print(a)
    return inner_func
my_new_func = outer_func()
my_new_func()
## inside one
my_new_func.__code__.co_freevars
## ('a',)
```

“*Global*” scope contains variables defined in the module-level namespace. If the code in the below example was the entirety of your script, then `a` would be a global variable.

```
a = "outside both"
def outer_func():
    b = "inside one"
    def inner_func():
        print(a)
    inner_func()
outer_func()
## outside both
```

Just like in R, Python functions **cannot** necessarily find variables where the function was *called*. For example, here is some code that mimics the above R example. Both `a` and `b` are accessible from within `inner_func()`. That is due to LEGB.

However, if we start using `outer_func()` inside another function, *calling* it in another function, when it was *defined* somewhere else, well then it doesn’t have access to variables in the call site. You might be surprised at how the

following code functions. Does this print the right string: "this is the a I want to use now!" No!

```
a = "outside both"
def outer_func():
    b = "inside one"
    def inner_func():
        print(a)
        print(b)
    return inner_func()
def third_func():
    a = "this is the a I want to use now!"
    outer_func()
third_func()
```

```
## outside both
## inside one
```

If you feel like you understand lexical scoping, great! You should be ready to take on [Chapter 15](#), then. If not, keep playing around with examples. Without understanding the scoping rules R and Python share, writing your own functions will persistently feel more difficult than it really is.

6.7 Modifying a Function's Arguments

Can/should we modify a function's argument? The flexibility to do this sounds empowering; however, not doing it is recommended because it makes programs easier to reason about.

6.7.1 Passing by Value in R

In R, it is *difficult* for a function to modify one of its argument.⁸ Consider the following code.

```
a <- 1
f <- function(arg){
  arg <- 2 # modifying a temporary variable, not a
  return(arg)
```

⁸There are some exceptions to this, but it's generally true.


```
}  
print(f(a))  
## [1] 2  
print(a)  
## [1] 1
```

The function `f` has an argument called `arg`. When `f(a)` is performed, changes are made to a *copy* of `a`. When a function constructs a copy of all input variables inside its body, this is called **pass-by-value** semantics. This copy is a temporary intermediate value that only serves as a starting point for the function to produce a return value of 2.

`arg` could have been called `a`, and the same behavior will take place. However, giving these two things different names is helpful to remind you and others that R copies its arguments.

It is still possible to modify `a`, but I don't recommend doing this either. I will discuss this more in [subsection 6.7](#).

6.7.2 Passing by Assignment in Python

The story is more complicated in Python. Python functions have **pass-by-assignment** semantics. This is something that is very unique to Python. What this means is that your ability to modify the arguments of a function depends on

- what the type of the argument is, and
- what you're trying to do to it.

We will go through some examples first, and then explain why this works the way it does. Here is some code that is analogous to the example above.

```
a = 1  
def f(arg):  
    arg = 2  
    return arg  
print(f(a))  
## 2  
print(a)  
## 1
```

In this case, `a` is not modified. That is because `a` is an `int`. `ints` are **immutable** in Python, which means that their value⁹ cannot be changed after they are created, either inside or outside of the function's scope. However, consider the case when `a` is a `list`, which is a **mutable** type. A mutable type is one that can have its value changed after its created.

```
a = [999]
def f(arg):
    arg[0] = 2
    return arg

print(f(a))
## [2]
print(a) # not [999] anymore!
## [2]
```

In this case `a` *is* modified. Changing the value of the argument *inside* the function effects changes to that variable outside of the function.

Ready to be confused? Here is a tricky third example. What happens if we take in a list, but try to do something else with it.

```
a = [999]
def f(arg):
    arg = [2]
    return arg

print(f(a))
## [2]
print(a) # didn't change this time :(
## [999]
```

That time `a` did not permanently change in the global scope. Why does this happen? I thought `lists` were mutable!

The reason behind all of this doesn't even have anything to do with functions, per se. Rather, it has to do with how Python manages, objects, values, and types¹⁰. It also has to do with what happens during assignment¹¹.

⁹<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

¹⁰<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

¹¹<https://docs.python.org/3/reference/executionmodel.html#naming-and-binding>

Let's revisit the above code, but bring everything out of a function. Python is pass-by-assignment, so all we have to do is understand how assignment works. Starting with the immutable `int` example, we have the following.

```
# old code:
# a = 1
# def f(arg):
#     arg = 2
#     return arg
a = 1    # still done in global scope
arg = a  # arg is a name that is bound to the object a refers to
arg = 2  # arg is a name that is bound to the object 2
print(arg is a)
## False
print(id(a), id(arg)) # different!`
## 139835665388896 139835665388928
print(a)
## 1
```



The `id()`^a function returns the **identity** of an object, which is kind of like its memory address. Identities of objects are unique and constant. If two variables, `a` and `b` say, have the same identity, `a is b` will evaluate to `True`. Otherwise, it will evaluate to `False`.

^a<https://docs.python.org/3/library/functions.html#id>

In the first line, the *name* `a` is bound to the *object* `1`. In the second line, the name `arg` is bound to the *object* that is referred to by the *name* `a`. After the second line finishes, `arg` and `a` are two names for the same object (a fact that you can confirm by inserting `arg is a` immediately after this line).

In the third line, `arg` is bound to `2`. The variable `arg` can be changed, but only by re-binding it with a separate object. Re-binding `arg` does not change the value referred to by `a` because `a` still refers to `1`, an object separate from `2`. There is no reason to re-bind `a` because it wasn't mentioned at all in the third line.

If we go back to the first function example, it's basically the same idea. The only difference, however, is that `arg` is in its own scope. Let's look at a simplified version of our second code chunk that uses a mutable list.

```
a = [999]
# old code:
# def f(arg):
#     arg[0] = 2
#     return arg
arg = a
arg[0] = 2
print(arg)
## [2]
print(a)
## [2]
print(arg is a)
## True
```

In this example, when we run `arg = a`, the name `arg` is bound to the same object that is bound to `a`. This much is the same. The only difference here, though, is that because lists are mutable, changing the first element of `arg` is done “in place”, and all variables can access the mutated object.

Why did the third example produce unexpected results? The difference is in the line `arg = [2]`. This rebinds the name `arg` to a different variable. Lists are still mutable, but this has nothing to do with re-binding—re-binding a name works no matter what type of object you’re binding it to. In this case we are re-binding `arg` to a completely different list.

6.8 Accessing and Modifying Captured Variables

In the last section, we were talking about variables that were passed in as function arguments. Here we are talking about variables that are **captured**. They are not passed in as variables, but they are still used inside a function. In general, even though it is possible to access and modify non-local captured variables in both languages, it is not a good idea.

6.8.1 Accessing Captured Variables in R

As Hadley Wickham writes in his book¹², “[l]exical scoping determines where, but not when to look for values.” R has **dynamic lookup**, meaning code inside

¹²<https://adv-r.hadley.nz/functions.html#dynamic-lookup>

a function will only try to access a referred-to variable when the function is *running*, not when it is defined.

Consider the R code below. The `dataReadyForModeling()` function is created in the global environment, and the global environment contains a Boolean variable called `dataAreClean`.

```
# R
dataAreClean <- TRUE
dataReadyForModeling <- function(){
  return(dataAreClean)
}
dataAreClean <- FALSE
# readyToDoSecondPart() # what happens if we call it now?
```

Now imagine sharing some code with a collaborator. Imagine, further, that your collaborator is the subject-matter expert, and knows little about R programming. Suppose that he changes `dataAreClean`, a global variable in the script, after he is done. Shouldn't this induce a relatively trivial change to the overall program?

Let's explore this hypothetical further. Consider what could happen if any of the following (very typical) conditions are true:

- you or your collaborators aren't sure what `dataReadyForModeling()` will return because you don't understand dynamic lookup, or
- it's difficult to visually keep track of all assignments to `dataAreClean` (e.g. your script is quite long or it changes often), or
- you are not running code sequentially (e.g. you are repeatedly testing chunks at a time instead of clearing out your memory and `source()`ing from scratch, over and over again).

In each of these situations, understanding of the program would be compromised. However, if you follow the above principle of never referring to non-local variables in function code, all members of the group could do their own work separately, minimizing the dependence on one another.

Another reason violating this could be troublesome is if you define a function that refers to a nonexistent variable. *Defining* the function will never throw an error because R will assume that variable is defined in the global environment. *Calling* the function might throw an error, unless you accidentally defined the variable, or if you forgot to delete a variable whose name you no longer want to use. Defining `myFunc()` with the code below will not throw an error, even if you think it should!

```
# R
myFunc <- function(){
  return(varigbleNameWithTypo) #varigble?
}
```

6.8.2 Accessing Captured Variables in Python

It is the same exact situation in Python. Consider `everything_is_safe()`, a function that is analogous to `dataReadyForModeling()`.

```
# python
missile_launch_codes_set = True
def everything_is_safe():
    return not missile_launch_codes_set

missile_launch_codes_set = False
everything_is_safe()
## True
```

We can also define `my_func()`, which is analogous to `myFunc()`. Defining this function doesn't throw an error either.

```
# python
def my_func():
    return varigble_name_with_typo
```

So stay away from referring to variables outside the body of your function!

6.8.3 Modifying Captured Variables in R

Now what if we want to be extra bad, and in addition to *accessing* global variables, we *modify* them, too?

```
a <- 1
makeATwo <- function(arg){
  arg <- 2
  a <<- arg
}
print(makeATwo(a))
## [1] 2
```

```
print(a)
## [1] 2
```

In the program above, `makeATwo()` copies `a` into `arg`. It then assigns 2 to that copy. **Then it takes that 2 and writes it to the global `a` variable in the parent environment.** It does this using R's super assignment operator `<<-`. Regardless of the inputs passed in to this function, it will always assign exactly 2 to `a`, no matter what.

This is problematic because you are pre-occupying your mind with one function: `makeATwo()`. Whenever you write code that depends on `a` (or on things that depend on `a`, or on things that depended on things that depend on `a`, or ...), you'll have to repeatedly interrupt your train of thought to *try* and remember if what you're doing is going to be okay with the current and future `makeATwo()` call sites.

6.8.4 Modifying Captured Variables in Python

There is something in Python that is similar to R's super assignment operator (`<<-`). It is the `global` keyword. This keyword will let you modify global variables from inside a function.



The upside to the `global` keyword is that it makes hunting for **side effects** relatively easy (A function's side effects are changes it makes to non-local variables). Yes, this keyword should be used sparingly, even more sparingly than merely referring to global variables, but if you are ever debugging, and you want to hunt down places where variables are surprisingly being changed, you can hit `Ctrl-F` and search for the phrase “`global`.”

```
a = 1
def increment_a():
    global a
    a += 1
[increment_a() for _ in range(10)]
## [None, None, None, None, None, None, None, None, None, None]
print(a)
## 11
```

6.9 Exercises

6.9.1 R Questions

1.

Suppose you have a matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ and a column vector $\mathbf{y} \in \mathbb{R}^n$. To estimate the linear regression model

$$\mathbf{y} = \mathbf{X}\beta + \epsilon, \quad (6.1)$$

where $\beta \in \mathbb{R}^p$ is a column vector of errors, you can use calculus instead of numerical optimization. The formula for the least squares estimate of β is

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (6.2)$$

Once this p -dimensional vector is found, you can also obtain the *predicted (or fitted) values*

$$\hat{\mathbf{y}} := \mathbf{X}\hat{\beta}, \quad (6.3)$$

and the *residuals (or errors)*

$$\mathbf{y} - \hat{\mathbf{y}} \quad (6.4)$$

Write a function called `getLinModEstimates()` that takes in two arguments in the following order:

- the vector of response data $\mathbf{y}$
- the matrix of predictors $\mathbf{X}$.

Have it return a named `list` with three outputs inside:

- the coefficient estimates as a vector,
- a vector of fitted values, and
- a vector of residuals.

The three elements of the returned list should have the names `coefficients`, `fitVals`, and `residuals`.

2.

Write a function called `monteCarlo` that

- takes as an input a function `sim(n)` that simulates `n` scalar variables,
- takes as an input a function that evaluates $f(x)$ on each random variable sample and that ideally takes in all of the random variables as a vector, and

- returns a function that takes one integer-valued argument (`num_sims`) and outputs a length one **vector**.

Assume `sim(n)` only has one argument: `n`, which is the number of simulations desired. `sim(n)`'s output should be a length `n` **vector**.

The output of this returned function should be a Monte Carlo estimate of the expectation: $\mathbb{E}[f(X)] \approx \frac{1}{n} \sum_{i=1}^n f(X^i)$.

3.

Write a function called `myDFT()` that computes the **Discrete Fourier Transform** of a **vector** and returns another **vector**. Feel free to check your work against `spec.pgram()`, `fft()`, or `astsa::mvspec()`, but do not include calls to those functions in your submission. Also, you should be aware that different functions transform and scale the answer differently, so be sure to read the documentation of any function you use to test against.

Given data $x_1, x_2, \dots, x_n$, $i = \sqrt{-1}$, and the **Fourier/fundamental frequencies** $\omega_j = j/n$ for $j = 0, 1, \dots, n-1$, we define the discrete Fourier transform (DFT) as:

$$d(\omega_j) = n^{-1/2} \sum_{t=1}^n x_t e^{-2\pi i \omega_j t} \quad (6.5)$$

6.9.2 Python Questions

1.

Estimating statistical models often involves some form of optimization, and often times, optimization is performed numerically. One of the most famous optimization algorithms is **Newton's method**.

Suppose you have a function $f(x)$ that takes a scalar-valued input and returns a scalar as well. Also, suppose you have the function's derivative $f'(x)$, its second derivative $f''(x)$, and a starting point guess for what the minimizing input of $f(x)$ is: x_0 .

The algorithm repeatedly applies the following recursion:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}. \quad (6.6)$$

Under appropriate regularity conditions for f , after many iterations of the above recursion, when $\tilde{n}$ is very large, $x_{\tilde{n}}$ will be nearly the same as $x_{\tilde{n}-1}$, and

$x_{\tilde{n}}$ is pretty close to $\operatorname{argmin}_x f(x)$. In other words, $x_{\tilde{n}}$ is the minimizer of f , and a root of f' .

- a) Write a function called `f` that takes a `float x` and returns $(x-42)^2-33$.
- b) Write a function called `f_prime` that takes a `float` and returns the derivative of the above.
- c) Write a function called `f_dub_prime` that takes a `float` and returns an evaluation of the second derivative of f .
- d) Theoretically, what is the minimizer of f ? Assign your answer to the variable `best_x`.
- e) Write a function called `minimize()` that takes three arguments, and performs **ten iterations** of Newton's algorithm, after which it returns x_{10} . Don't be afraid of copy/pasting ten or so lines of code. We haven't learned loops yet, so that's fine. The ordered arguments are:
 - the function that evaluates the derivative of the function you're interested in,
 - the function that evaluates the second derivative of your objective function,
 - an initial guess of the minimizer.
- f) Test your function by plugging in the above functions, and use a starting point of 10. Assign the output to a variable called `x_ten`.

2.

Write a function called `smw_inverse(A,U,C,V)` that returns the inverse of a matrix using the **Sherman-Morrison-Woodbury formula** (Guttman, 1946). Have it take the arguments A , U , C , and V in that order and as Numpy `ndarrays`. Assume that A is a diagonal matrix.

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1} \quad (6.7)$$

Despite being difficult to remember, this formula can be quite handy for speeding up matrix inversions when A and C are easier to invert (e.g. if A is diagonal and C is a scalar). The formula often shows up a lot in applications where you multiply matrices together (there are many such examples).

To check your work, pick certain inputs, and make sure your formula corresponds with the naive, left-hand-side approach.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Categorical Data

While statisticians may describe data as being either categorical or numerical, this classification is different than classifying data by its *type* in a program. So, strictly speaking, if you have categorical data, you are not obligated to use any particular type to represent it in your script.

However, there are types that are specifically designed to be used with categorical data, and so they are especially advantageous to use if you end up with the opportunity. We describe a few of them here in this chapter.

7.1 factors in R

Categorical data in R is often stored in a `factor`¹ variable. `factors` are more special than `vectors` of integers because

- they have a `levels` attribute, which is comprised of all the possible values that each response could be;
- they may or may not be *ordered*, which will also control how they are used in mathematical functions;
- they might have a `contrasts` attribute, which will control how they are used in statistical modeling functions.

Here is a first example. Say we asked three people what their favorite season was. The data might look something like this.

```
allSeasons <- c("spring", "summer", "autumn", "winter")
responses <- factor(c("autumn", "summer", "summer"),
                    levels = allSeasons)

levels(responses)
## [1] "spring" "summer" "autumn" "winter"
is.factor(responses)
## [1] TRUE
```

¹<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Factors>

```
is.ordered(responses)
## [1] FALSE
#contrasts(responses)
# ^ controls how factor is used in different functions
```

factors always have levels, which is the collection of all possible unique values each observation can take.

You should be careful if you are not specifying them directly. What happens when you use the default option and replace the second assignment in the above code with `responses <- factor(c("autumn", "summer", "summer"))`? The documentation of `factor()` will tell you that, by default, `factor()` will just take the unique values found in the data. In this case, nobody prefers winter or spring, and so neither will show up in `levels(responses)`. This may or may not be what you want.

factors can be ordered or unordered. Ordered factors are for *ordinal* data. Ordinal data is a particular type of categorical data that recognizes the categories have a natural order (e.g. low/ medium/high and not red/green/blue).

As another example, say we asked ten people how much they liked statistical computing, and they could only respond “love it”, “it’s okay” or “hate it”. The data might look something like this.

```
ordFeelOptions <- c("hate it", "it's okay", "love it")
responses <- factor(c("love it", "it's okay", "love it",
                     "love it", "it's okay", "love it",
                     "love it", "love it", "it's okay",
                     "it's okay"),
                  levels = ordFeelOptions,
                  ordered = TRUE)

levels(responses)
## [1] "hate it" "it's okay" "love it"
is.factor(responses)
## [1] TRUE
is.ordered(responses)
## [1] TRUE
# contrasts(responses)
```

When creating ordered factors with `factor()`, be mindful that the `levels=` argument is assumed to be ordered when you plug it into `factor()`. In the above example, if you specified `levels = c("love it", "it's okay", "hate it")`, then the factor would assume `love it < it's okay < hate it`, which may or may not be what you want.

Last, factors may or may not have a `contrast` attribute. You can get or set this with the `contrasts()` function. This will influence some of the functions you use on your data that estimate statistical models.

I will not discuss specifics of contrasts in this text, but the overall motivation is important. In short, the primary reason for using factors is that they are designed to allow control over *how* you model categorical data. To be more specific, changing attributes of a factor could control the parameterization of a model you're estimating. If you're using a particular function for modeling with categorical data, you need to know how it treats factors. On the other hand, if you're writing a function that performs modeling of categorical data, you should know how to treat factors.

Here are two examples that you might come across in your studies.

1. Consider using factors as inputs to a function that performs linear regression. With linear regression models, if you have categorical inputs, there are many choices for how to write down a model. In each model, the collection of parameters will mean different things. In R, you might pick the model by creating the factor in a specific way.
2. Suppose you are interested in estimating a classification model. In this case, the *dependent* variable is categorical, not the independent variable. With these types of models, choosing whether or not your factor is ordered is critical. These options would estimate completely different models, so choose wisely!

The mathematical details of these examples is outside of the scope of this text. If you have not learned about dummy variables in a regression course, or if you have not considered the difference between multinomial logistic regression and ordinal logistic regression, or if you have but you're just a little rusty, that is totally fine. I only mention these as examples for how the factor type can trigger special behavior.

In addition to creating one with `factor()`, there are two other common ways that you can end up with factors:

1. creating factors from numerical data, and
2. when reading in an external data file, one of the columns is coerced to a **factor**.

Here is an example of (1). We can take non-categorical data, and `cut()` it into something categorical.

```
stockReturns <- rnorm(6) # not categorical here
typeOfDay <- cut(stockReturns, breaks = c(-Inf, 0, Inf))
typeOfDay
## [1] (0, Inf] (0, Inf] (-Inf,0] (-Inf,0] (-Inf,0] (0, Inf]
## Levels: (-Inf,0] (0, Inf]
levels(typeOfDay)
## [1] "(-Inf,0]" "(0, Inf]"
is.factor(typeOfDay)
## [1] TRUE
is.ordered(typeOfDay)
## [1] FALSE
```

Finally, be mindful of how different functions read in external data sets. When reading in an external file, if a particular function comes across a column that has characters in it, it will need to decide whether to store that column as a character vector, or as a **factor**. For example, `read.csv()` and `read.table()` have a `stringsAsFactors=` argument that you should be mindful of.

7.2 Two Options for Categorical Data in Pandas

Pandas provides two options for storing categorical data. They are both very similar to R's **factors**. You may use either

1. a Pandas **Series** with a special `dtype`, or
2. a Pandas **Categorical** container.

Pandas' **Series** were discussed earlier in [Sections 3.2](#) and [3.4](#). These were containers that forced every element to share the same `dtype`. Here, we specify `dtype="category"` in `pd.Series()`.

```
import pandas as pd
szn_s = pd.Series(["autumn", "summer", "summer"], dtype = "category")
```

```

szn_s.cat.categories
## Index(['autumn', 'summer'], dtype='object')
szn_s.cat.ordered
## False
szn_s.dtype
## CategoricalDtype(categories=['autumn', 'summer'], ordered=False)
type(szn_s)
## <class 'pandas.core.series.Series'>

```

The second option is to use Pandas' `Categorical` containers. They are quite similar, so the choice is subtle. Like `Series` containers, they also force all of their elements to share the same shared `dtype`.

```

szn_c = pd.Categorical(["autumn", "summer", "summer"])
szn_c.categories
## Index(['autumn', 'summer'], dtype='object')
szn_c.ordered
## False
szn_c.dtype
## CategoricalDtype(categories=['autumn', 'summer'], ordered=False)
type(szn_c)
## <class 'pandas.core.arrays.categorical.Categorical'>

```

You might have noticed that, with the `Categorical` container, methods and data members were not accessed through the `.cat` accessor. It is also more similar to R's `factors` because you can specify more arguments in the constructor.

```

all_szns = ["spring", "summer", "autumn", "winter"]
szn_c2 = pd.Categorical(["autumn", "summer", "summer"],
                        categories = all_szns,
                        ordered = False)

```

■ In Pandas, just like in R, you need to be very careful about what the `categories` (c.f. `levels`) are. If you are using ordinal data, they need to be specified in the correct order. If you are using small data sets, be cognizant of whether all the categories show up in the data—otherwise they will not be correctly inferred.

With Pandas' `Series` it's more difficult to specify a nondefault `dtype`. One option is to change them after the object has been created.

```
szn_s = szn_s.cat.set_categories(
    ["autumn", "summer", "spring", "winter"])
szn_s.cat.categories
## Index(['autumn', 'summer', 'spring', 'winter'], dtype='object')
szn_s = szn_s.cat.remove_categories(['spring', 'winter'])
szn_s.cat.categories
## Index(['autumn', 'summer'], dtype='object')
szn_s = szn_s.cat.add_categories(["fall", "winter"])
szn_s.cat.categories
## Index(['autumn', 'summer', 'fall', 'winter'], dtype='object')
```

Another option is to create the `dtype` before you create the `Series`, and pass it into `pd.Series()`.

```
cat_type = pd.CategoricalDtype(
    categories=["autumn", "summer", "spring", "winter"],
    ordered=True)
responses = pd.Series(
    ["autumn", "summer", "summer"],
    dtype = cat_type)
responses
## 0    autumn
## 1    summer
## 2    summer
## dtype: category
## Categories (4, object): ['autumn' < 'summer' < 'spring' < 'winter']
```

Just like in R, you can convert numerical data into categorical. The function even has the same name as in R: `pd.cut()`². Depending on the type of the input, it will return either a `Series` or a `Categorical`³.

```
import numpy as np
stock_returns = np.random.normal(size=10) # not categorical
# array input means Categorical output
```

²https://pandas.pydata.org/pandas-docs/stable/user_guide/categorical.html#series-creation

³<https://pandas.pydata.org/docs/reference/api/pandas.cut.html>

```

type_of_day = pd.cut(stock_returns,
                      bins = [-np.inf, 0, np.inf],
                      labels = ['bad day', 'good day'])

type(type_of_day)
# Series in means Series out
## <class 'pandas.core.arrays.categorical.Categorical'>

type_of_day2 = pd.cut(pd.Series(stock_returns),
                      bins = [-np.inf, 0, np.inf],
                      labels = ['bad day', 'good day'])

type(type_of_day2)
## <class 'pandas.core.series.Series'>

```

Finally, when reading in data from an external source, choose carefully whether you want character data to be stored as a string type, or as a categorical type. Here we use `pd.read_csv()`⁴ to read in Fisher's Iris data set (Fisher, 1988) hosted by (Dua and Graff, 2017). More information on Pandas' DataFrames can be found in the next chapter.

```

import numpy as np
# make 5th col categorical
my_data = pd.read_csv("data/iris.csv", header=None,
                      dtype = {4:"category"})

my_data.head(1)
##      0      1      2      3      4
## 0  5.1  3.5  1.4  0.2  Iris-setosa

my_data.dtypes
## 0      float64
## 1      float64
## 2      float64
## 3      float64
## 4      category
## dtype: object

np.unique(my_data[4]).tolist()
## ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']

```

⁴https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

7.3 Exercises

7.3.1 R Questions

1.

Read in this chess data set (mis, 1989), hosted by (Dua and Graff, 2017), with the following code. You will probably have to change your working directory, but if you do, make sure to comment out that code before you submit your script to me.

```
d <- read.csv("kr-vs-kp.data", header=FALSE, stringsAsFactors = TRUE)
head(d)
```

- a) Are all of the columns factors? Assign TRUE or FALSE to `allFactors`.
- b) Should any of these factors be ordered? Assign TRUE or FALSE to `ideallyOrdered`. Hint: read the data set description from <https://archive.ics.uci.edu/ml/datasets/Chess+%28King-Rook+vs.+King-Pawn%29>.
- c) Are any of these factors currently ordered? Assign TRUE or FALSE to `currentlyOrdered`.
- d) What percent (between 0 and 100) of the time is the first column equal to 'f'? Assign your answer to `percentF`.

2.

Suppose you have the following vector. Please make sure to include this code in your script.

```
normSamps <- rnorm(100)
```

- a) create a factor from `normSamps`. Map each element to "within 1 sd" or "outside 1 sd" depending on whether the element is within 1 theoretical standard deviation of 0 or not. Call the factor `withinOrNot`.

7.3.2 Python Questions

1.

Consider the following simulated letter grade data for two students:

```
import pandas as pd
import numpy as np
poss_grades = ['A+', 'A', 'A-', 'B+', 'B', 'B-',
               'C+', 'C', 'C-', 'D+', 'D', 'D-',
               'F']
grade_values = {'A+':4.0, 'A':4.0, 'A-':3.7, 'B+':3.3, 'B':3.0, 'B-':2.7,
               'C+':2.3, 'C':2.0, 'C-':1.7, 'D+':1.3, 'D':1.0, 'D-':.67,
               'F':0.0}
student1 = np.random.choice(poss_grades, size = 10, replace = True)
student2 = np.random.choice(poss_grades, size = 12, replace = True)
```

- Convert the two Numpy arrays to one of the Pandas types for categorical data that the textbook discussed. Call these two variables `s1` and `s2`.
- These data are categorical. Are they ordinal? Make sure to adjust `s1` and `s2` accordingly.
- Calculate the two student GPAs. Assign the floating point numbers to variables named `s1_gpa` and `s2_gpa`. Use `grade_values` to convert each letter grade to a number, and then average all the numbers for each student together using equal weights.
- Is each category equally-spaced? If yes, then these are said to be *interval data*. Does your answer to this question affect the legitimacy of averaging together any ordinal data? Assign a `str` response to the variable `ave_ord_data_response`. Hint: consider (any) two different data sets that happen to produce the same GPA. Is the equality of these two GPAs misleading?
- Compute the mode grade for each student. Assign your answers as `strs` to the variables `s1_mode` and `s2_mode`. If there are more than one modes, then assign the one that comes first alphabetically.

2.

Suppose you are creating a *classifier* whose job it is to predict labels. Consider the following `DataFrame` of predicted labels next to their corresponding actual labels. Please make sure to include this code in your script.

```
import pandas as pd
import numpy as np
d = pd.DataFrame({'predicted label' : [1,2,2,1,2,2,1,2,3,2,2,3],
                  'actual label': [1,2,3,1,2,3,1,2,3,1,2,3]})
```

```
dtype='category')  
d.dtypes[0]  
## CategoricalDtype(categories=[1, 2, 3], ordered=False)  
d.dtypes[1]  
## CategoricalDtype(categories=[1, 2, 3], ordered=False)
```

- a) Assign the prediction accuracy, as a percent (between 0 and 100), to the variable `perc_acc`.
- b) Create a *confusion matrix* to better assess *which* labels your classifier has a difficult time with. This should be a 3×3 Numpy ndarray of percentages. The row will correspond to the predicted label, the column will correspond to the actual label, and number in location (0,2), say, will be the percent of observations where your model predicted label 1 and the actual was a label 3. Call the variable `confusion`.

The rectangular array of information (e.g. an Excel spreadsheet) is what many think of when they hear the word “data.” Each column contains elements of a shared data type, and these data types can vary from column to column.

There is a type for this in R and Python: a data frame. It might even be the most common way that data is stored in both R and Python programs because many functions that read in data from an external source return objects of this type (e.g. `read.csv()` in R and `pd.read_csv()` in Python).

R and Python’s data frames have a number of things in common:

1. each column must be the same length as all other columns,
2. each column’s elements will all have the same type,
3. elements in any rows can have different types,
4. columns and rows can be named in different ways,
5. there are many ways to get and set different subsets of data, and
6. when reading in data, the same sorts of difficulties arise in both languages.

8.1 Data Frames in R

Let’s consider as an example Fisher’s “Iris” data set (Fisher, 1988) hosted by (Dua and Graff, 2017). We will read this data set in from a comma separated file (more information on input/output can be found in [Chapter 9](#)). This file can be downloaded from this link: <https://archive.ics.uci.edu/ml/datasets/iris>.

```
irisData <- read.csv("data/iris.csv", header = F)
head(irisData, 3)
##      V1  V2  V3  V4      V5
## 1 5.1 3.5 1.4 0.2 Iris-setosa
## 2 4.9 3.0 1.4 0.2 Iris-setosa
## 3 4.7 3.2 1.3 0.2 Iris-setosa
```

```
typeof(irisData)
## [1] "list"
class(irisData) # we'll talk more about classes later
## [1] "data.frame"
dim(irisData)
## [1] 150  5
nrow(irisData)
## [1] 150
ncol(irisData)
## [1] 5
```

There are some exceptions, but most data sets can be stored as a `data.frame`. These kinds of two-dimensional data sets are quite common. Any particular row is often an observation on one experimental unit (e.g. person, place or thing). Looking at a particular column gives you one kind of measurement stored for all observations.

Do not rely on the default arguments of `read.csv()` or `read.table()`! After you read in a data frame, always check to make sure that



1. the number of columns is correct because the correct column *separator* was used (c.f. `sep=`),
2. column names were parsed correctly, if there were some in the raw text file,
3. the first row of data wasn't used as a column name sequence, if there weren't column names in the text file, and
4. the last few rows aren't reading in empty spaces
5. character columns are read in correctly (c.f. `stringsAsFactors=`), and
6. special characters signifying missing data were correctly identified (c.f. `na.strings=`).

A `data.frame` is a special case of a `list`¹. Every element of the list is a column. Columns can be vectors or factors, and they can all be of a different type. This is one of the biggest differences between data frames and `matrix`s. They are both two-dimensional, but a `matrix` needs elements to be all the same type. Unlike a general `list`, a `data.frame` requires all of its columns to have the same number of elements. In other words, the `data.frame` is not a “*ragged*” list.

¹<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Data-frame-objects>

Often times you will need to extract pieces of information from a `data.frame`. This can be done in many ways. If the columns have names, you can use the `$` operator to access a single column. Accessing a single column might be followed up by creating a new vector. You can also use the `[` operator to access multiple columns by name.

```
colnames(irisData) <- c("sepal.length", "sepal.width",
                        "petal.length", "petal.width",
                        "species")
firstCol <- irisData$sepal.length
head(firstCol)
## [1] 5.1 4.9 4.7 4.6 5.0 5.4
firstTwoCols <- irisData[c("sepal.length", "sepal.width")]
head(firstTwoCols, 3)
##   sepal.length sepal.width
## 1          5.1          3.5
## 2          4.9          3.0
## 3          4.7          3.2
```

The `[` operator is also useful for selecting rows and columns by index numbers, or by some logical criteria.

```
topLeft <- irisData[1,1] # first row, first col
topLeft
## [1] 5.1
firstThreeRows <- irisData[1:3,] # rows 1-3, all cols
firstThreeRows
##   sepal.length sepal.width petal.length petal.width   species
## 1          5.1          3.5          1.4          0.2 Iris-setosa
## 2          4.9          3.0          1.4          0.2 Iris-setosa
## 3          4.7          3.2          1.3          0.2 Iris-setosa
# rows where species column is setosa
setosaOnly <- irisData[irisData$species == "Iris-setosa",]
setosaOnly[1:3,-1]
##   sepal.width petal.length petal.width   species
## 1          3.5          1.4          0.2 Iris-setosa
## 2          3.0          1.4          0.2 Iris-setosa
## 3          3.2          1.3          0.2 Iris-setosa
```

In the code above, `irisData$species == "Iris-setosa"` creates a logical vector (try it!) using the vectorized `==` operator. The `[` operator selects the rows for which the corresponding element of this logical vector is `TRUE`.



Be careful: depending on how you use the square brackets, you can either get a `data.frame` or a `vector`. As an example, try both `class(irisData[,1])` and `class(irisData[,c(1,2)])`.

In R, `data.frames` might have row names. You can get and set this character vector with the `rownames()` function. You can access rows by name using the square bracket operator.

```
head(rownames(irisData))
## [1] "1" "2" "3" "4" "5" "6"
rownames(irisData) <- as.numeric(rownames(irisData)) + 1000
head(rownames(irisData))
## [1] "1001" "1002" "1003" "1004" "1005" "1006"
irisData["1002",]
##      sepal.length sepal.width petal.length petal.width      species
## 1002           4.9           3           1.4           0.2 Iris-setosa
```

Code that modifies data usually looks quite similar to code extracting data. You'll notice a lot of the same symbols (e.g. `$`, `[`, etc.), but the `(<-)` will point in the other direction.

```
irisData$columnOfOnes <- rep(1, nrow(irisData))
irisData[,1] <- NULL #delete first col
irisData[1:2,1] <- rnorm(n = 2, mean = 999)
irisData['sepal.width'] <- rnorm(n = nrow(irisData), mean = -999)
irisData[irisData$species == "Iris-setosa", 'species'] <- "SETOSA!"
head(irisData, 3)
##      sepal.width petal.length petal.width species columnOfOnes
## 1001    -998.2750           1.4           0.2 SETOSA!           1
## 1002    -999.1644           1.4           0.2 SETOSA!           1
## 1003    -998.5575           1.3           0.2 SETOSA!           1
```

8.2 Data Frames in Python

The Pandas library in Python has data frames that are modeled after R's (McKinney, 2017).

```
import pandas as pd
iris_data = pd.read_csv("data/iris.csv", header = None)
iris_data.head(3)
##      0      1      2      3      4
## 0  5.1  3.5  1.4  0.2  Iris-setosa
## 1  4.9  3.0  1.4  0.2  Iris-setosa
## 2  4.7  3.2  1.3  0.2  Iris-setosa
iris_data.shape
## (150, 5)
len(iris_data) # num rows
## 150
len(iris_data.columns) # num columns
## 5
list(iris_data.dtypes)[:3]
## [dtype('float64'), dtype('float64'), dtype('float64')]
list(iris_data.dtypes)[3:]
## [dtype('float64'), dtype('O')]
```

The structure is very similar to that of R's data frame. It's two dimensional, and you can access columns and rows by name or number.² Each column is a `Series` object, and each column can have a different `dtype`, which is analogous to R's situation. Again, because the elements need to be the same type along columns only, this is a big difference between 2-d Numpy `ndarrays` and `DataFrames` (c.f. R's `matrix` versus R's `data.frame`).

Again, do not rely on the default arguments of `pd.read_csv()`! After you read in a data set, always check that



1. the number of columns is correct because the correct column *separator* was used (c.f. `sep=`),
2. column names were parsed correctly, if there were some in the raw text file,
3. the first row of data wasn't used as a column name sequence, if there weren't column names in the text file (c.f. `header=`), and
4. the last few rows aren't reading in empty spaces
5. character columns are read in correctly (c.f. `dtype=`), and
6. special characters signifying missing data were correctly identified (c.f. `na.values=`).

²https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html

Square brackets are a little different in Python than they are in R. Just like in R, you can access columns by name with square brackets, and you can also access rows. Unlike R, though, you don't have to specify both rows and columns every time you use the square brackets.

```
iris_data.columns = ["sepal.length", "sepal.width", "petal.length",
                    "petal.width", "species"]
first_col = iris_data['sepal.length']
first_col.head()
## 0      5.1
## 1      4.9
## 2      4.7
## 3      4.6
## 4      5.0
## Name: sepal.length, dtype: float64
first_two_cols = iris_data[["sepal.length", "sepal.width"]]
first_two_cols.head(3)
##      sepal.length  sepal.width
## 0              5.1           3.5
## 1              4.9           3.0
## 2              4.7           3.2
```



Notice that `iris_data['sepal.length']` returns a `Series` and `iris_data[["sepal.length", "sepal.width"]]` returns a `Pandas DataFrame`. This behavior is similar to what happened in R. For more details, click here^a.

^ahttps://pandas.pydata.org/pandas-docs/stable/user_guide/dsintro.html#indexing-selection

You can select columns and rows by number with the `.iloc` method³. `iloc` is (probably) short for “integer location.”

```
# specify rows/cols by number
top_left = iris_data.iloc[0,0]
top_left
## 5.1
first_three_rows_without_last_col = iris_data.iloc[:3,:-1]
first_three_rows_without_last_col
```

³<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

```
##      sepal.length  sepal.width  petal.length  petal.width
## 0           5.1           3.5           1.4           0.2
## 1           4.9           3.0           1.4           0.2
## 2           4.7           3.2           1.3           0.2
```

Selecting columns by anything besides integer number can be done with the `.loc()` method⁴. You should generally prefer this method to access columns because accessing things by *name* instead of *number* is more readable. Here are some examples.

```
sepal_w_to_pedal_w = iris_data.loc[:, 'sepal.width': 'petal.width']
sepal_w_to_pedal_w.head()
##      sepal.width  petal.length  petal.width
## 0           3.5           1.4           0.2
## 1           3.0           1.4           0.2
## 2           3.2           1.3           0.2
## 3           3.1           1.5           0.2
## 4           3.6           1.4           0.2
setosa_only = iris_data.loc[iris_data['species'] == "Iris-setosa",]
# don't need the redundant column anymore
del setosa_only['species']
setosa_only.head(3)
##      sepal.length  sepal.width  petal.length  petal.width
## 0           5.1           3.5           1.4           0.2
## 1           4.9           3.0           1.4           0.2
## 2           4.7           3.2           1.3           0.2
```

Notice we used a slice (i.e. `'sepal.width': 'petal.width'`) to access many columns by only referring to the left-most and the right-most. Unlike slicing with numbers, the right end is included⁵. Also note that this does not work with the regular square bracket operator (i.e. `iris_data['sepal.width': 'petal.width']`). The second example filters out the rows where the "species" column elements are equal to "Iris-setosa".

Each `DataFrame` in Pandas comes with an `.index` attribute. This is analogous to a row name in R, but it's much more flexible because the index can take on a variety of types. This can help us highlight the difference between `.loc` and

⁴<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>

⁵https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#slicing-ranges

`.iloc`. Recall that `.loc` was *label-based selection*. Labels don't necessarily have to be strings. Consider the following example.

```
iris_data.index
# reverse the index
## RangeIndex(start=0, stop=150, step=1)
iris_data = iris_data.set_index(iris_data.index[::-1])
iris_data.iloc[-2,:3] # top is now bottom
##      sepal.length  sepal.width  petal.length
## 1              6.2           3.4           5.4
## 0              5.9           3.0           5.1
iris_data.loc[0] # last row has 0 index
## sepal.length           5.9
## sepal.width             3
## petal.length           5.1
## petal.width            1.8
## species      Iris-virginica
## Name: 0, dtype: object
iris_data.iloc[0] # first row with big index
## sepal.length           5.1
## sepal.width            3.5
## petal.length           1.4
## petal.width            0.2
## species      Iris-setosa
## Name: 149, dtype: object
```

`iris_data.loc[0]` selects the 0th index. The second line reversed the indexes, so this is actually the last row. If you want the first row, use `iris_data.iloc[0]`.

Modifying data inside a data frame looks quite similar to extracting data. You'll recognize a lot of the methods mentioned earlier.

```
import numpy as np
n_rows = iris_data.shape[0]
iris_data['col_ones'] = np.repeat(1.0, n_rows)
iris_data.iloc[:,0] = np.random.normal(loc=999, size=2)
rand_nums = np.random.normal(loc=-999, size=n_rows)
iris_data.loc[:, 'sepal.width'] = rand_nums
setosa_rows = iris_data['species'] == "Iris-setosa"
iris_data.loc[setosa_rows, 'species'] = "SETOSA!"
del iris_data['petal.length']
```

```
iris_data.head(3)
##          sepal.length  sepal.width  petal.width  species  col_ones
## 149      998.388556 -1000.146000         0.2  SETOSA!        1.0
## 148      997.790911 -998.745409         0.2  SETOSA!        1.0
## 147         4.700000 -996.467804         0.2  SETOSA!        1.0
```

You can also use the `.assign()` method⁶ to create a new column. This method does not modify the data frame in place. It returns a new `DataFrame` with the additional column.

```
iris_data = iris_data.assign(new_col_name = np.arange(n_rows))
del iris_data['sepal.length']
iris_data.head(3)
##          sepal.width  petal.width  species  col_ones  new_col_name
## 149 -1000.146000         0.2  SETOSA!        1.0            0
## 148 -998.745409         0.2  SETOSA!        1.0            1
## 147 -996.467804         0.2  SETOSA!        1.0            2
```

■ Above we were assigning Numpy arrays to columns of a `DataFrame`. Be careful when you're assigning `Series` objects. You'll see in the documentation that "Pandas aligns all AXES when setting `Series` and `DataFrame` from `.loc`, and `.iloc`."^a.

^ahttps://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#basics

8.3 Exercises

8.3.1 R Questions

1.

Consider the data set "original_rt_snippets.txt" (Socher et al., 2013), which is hosted by (Dua and Graff, 2017). We will calculate the *term frequency-inverse document frequency statistics* (Jones, 1972) on this data set, which is a common data transformation technique used in text mining and natural language processing. You may use the `stringr` library for this question, if you wish.

⁶<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.assign.html>

- a) read in this data set as a `vector` and call it `corpus`.
- b) create a `vector` called `dictionary` that contains the following phrases: “charming”, “fantasy”, “hate”, and “boring”.
- c) Construct a `data.frame` with four columns called `bagOfWords` that contains the number of appearances of each word in the dictionary. Match the exact phrases. For simplicity, don’t worry about the case of letters or using regular expressions (c.f. [Section 3.9](#)). Label the columns of this `data.frame` with the phrases you’re searching for. Try to write code that is easy to modify if you decide to change the set of phrases in your dictionary.
- d) Create a `data.frame` with four columns called `termFrequency`. Each element should correspond with the count from the previous `data.frame`. Instead of the count, each element should be $\log(1 + \text{count})$.
- e) Create a `vector` of length four called `invDocFrequency`. The inverse document frequency formula for any term t is $\log([\text{number of documents in corpus}]) - \log([\text{number of documents that contain term } t])$. Make sure the names of this vector are the same as the words in the dictionary.
- f) Create a `data.frame` called `tfidf` (short for “term frequency-inverse document frequency”). For row/document d , and column/term t , the formula is the product: $[\text{term frequency of term } t \text{ and document } d] \times [\text{inverse doc. freq. of term } t]$.
- g) Extract the elements of `corpus` that have at least one nonzero element in the corresponding row of `tfidf`. Call the `vector` `informativeDocuments`.
- h) Do you see any documents that were labeled as informative, that do not actually contain the words you searched for?

2.

`mtcars` is a data set that is built into R, so you don’t need to read it in. You can read more about it by typing `?datasets::mtcars`.

- a) Create a new `data.frame` called `withoutDisp` that is the same as `mtcars`, but has its `disp` column removed.
- b) Create a new column for `withoutDisp` called `coolnessScore`. The formula is $\frac{1}{\text{mpg}} + \text{quarter mile time}$.
- c) Create a new `data.frame` called `sortedWD` that is equal to `withoutDisp`, but sorted in descending order by the coolness score.
- d) Create a new `data.frame` from `sortedWD` called `specialRows` that keeps only rows where $\text{weight (1000lbs)} + \text{rear axle ratio} < 7$
- e) Calculate the percent reduction in number of rows, going from

sortedWD to specialRows. Call it percReduction. Make sure it is between 0 and 100.

3.

This question investigates the *Zillow Home Value Index (ZHVI)*⁷ for single family homes.

- a) read in "Zip_zhvi_uc_sfr_tier_0.33_0.67_sm_sa_month.csv". Call the data.frame homeData. Remember to be careful with file paths. Also, when looking at the data set with a text editor, be sure that "word wrap" is not turned on.
- b) Extract the rows of homeData that correspond with Charlottesville, VA, and assign them as a data.frame to the variable cvilleData
- c) Assign all the unique zip codes to a character vector called cvilleZips.
- d) Extract the columns of cvilleData that correspond with home prices, and transpose them so that each row of the result corresponds with a different month. Call this new data.frame timeSeriesData. Also, make sure that the column names of this new data.frame are set equal to the appropriate zip codes.
- e) Write a function called getAveMonthlyChange that takes in a numeric vector, and returns the average change. Your function should not return an NA, so be sure to deal with NAs appropriately.
- f) Calculate the average monthly price change for each zip code. Store your results in a vector called aveMonthlyChanges. Make sure this vector has named elements so one can extract elements by zip code.

8.3.2 Python Questions

1.

This question deals with looking at historical prices of the S&P500 Index. This data was downloaded from <https://finance.yahoo.com> (gsp, 2021). It contains prices starting from "2007-01-03" and going until "2021-10-01".

- a) Read in the data file "gspc.csv" as a data.frame and call the variable gspc.
- b) Use .set_index()⁸ to change the index of gspc to its "Index" column. Store the new DataFrame as gspc_good_idx.

⁷<https://www.zillow.com/research/data/>

⁸https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.set_index.html

- c) Recall the formula for *log returns* provided in exercises of [Chapter 3](#). Add a column to `gspc_good_idx` called `log_returns`. Calculate them from the column `GSPC.Adjusted`. Make sure to scale them by 100 and put an `np.nan` in the first element where you don't have a return.
- d) Extract all the returns that are available for the year 2021, and store them as a `Series` called `this_year_returns`.
- e) Add a column to `gspc_good_idx` containing the *drawdown* time series. Call this column `drawdown`. To calculate drawdown at a given date, subtract that date's price from the running maximum price that is current at that date. Use only adjusted close prices for this calculation.
- f) Add a column to `gspc_good_idx` containing the percent drawdown time series. Call this column `perc_drawdown`. Use the previous column, but make this number a percent of the corresponding running maximum.
- g) What is the maximum drawdown of this time series? Store it as a percent in the value `mdd`.

2.

In this question we'll look at some data on radon measurements⁹ (Gelman and Hill, 2007). Instead of reading in a text file, we will load the data into Python using the `tensorflow_datasets` module¹⁰ (TFD, 2021).

Please include the following code in your submission.

```
import tensorflow_datasets as tfds
import pandas as pd
import numpy as np
d = tfds.load("radon")
d = pd.DataFrame(tfds.as_dataframe(d['train']))
```

Many of you will need to install `tensorflow` and `tensorflow_datasets` before you're able to `import` it. If that's so, please read [Section 10.2](#) for more information on how to install packages.

- a) Assign to `worst_row` the row of `d` that is associated the highest recorded level of radon. Make sure it a `DataFrame`.
- b) Assign to `nrows` and `ncols` the number of rows and columns of `d`, respectively.

⁹<https://www.tensorflow.org/datasets/catalog/radon>

¹⁰<https://www.tensorflow.org/datasets>

- c) Assign the most common column data type to `most_common_dtype`. Make sure the variable is of type `numpy.dtype`
- d) Are there any observations from Virginia in this data set? If so, assign `True` to `any_va`. Otherwise assign `False`.
- e) Create a new column in `d` called `dist_from_cville`. Use the **Haversine formula** to calculate distance between each row and the University of Virginia, in kilometers.
 - Assume the University of Virginia is at 38.0336° N, 78.5080°W
 - Assume the Earth's radius $r = 6378.137$ kilometers.
 - The formula for distance between (λ_1, ϕ_1) (signed longitude in radians, signed latitude in radians) and (λ_2, ϕ_2) is

$$2 r \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right) \quad (8.1)$$

- f) What is the average radon measurement between all measurements taken at the place that is closest to where we are now? Assign your answer as a `float` to `close_ave`



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part II

Common Tasks and Patterns



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

9.1 General Input Considerations

So far, this text has been favoring the creation of small pieces of data *within our scripts*. The avoidance of reading in data from an external file has been avoided primarily for pedagogical purposes. In general, one might have

- data read in from a plain text file (e.g. `"my_data.csv"` or `"log_file.txt"`),
- data read in from a database (e.g. MySQL, PostgreSQL, etc.), or
- data created in a script (either deterministically or randomly).

When discussing reading in data, this text mostly focuses on the first category. Here are the reasons for its doing so:

1. text-files are more readily-available to students than databases,
2. teaching the second category requires teaching SQL, and that would introduce conceptual overlap, and
3. the third category is programmatically self-explanatory.

The third reason does not imply data created by code is unimportant. For example, it is the most common approach to create data used in **simulation studies**. Authors writing statistical papers need to demonstrate that their techniques work on “nice” data: data simulated from a *known* data-generating process. In a simulation study, unlike in the “real-world,” you have access to the parameters generating your data, and you can examine data that might otherwise be unobserved or hidden. Further, with data from the real-world, there is no guarantee your model correctly matches the true model.



Can your code/technique/algorithm, at the very least, obtain parameter estimates that are “in-line” with the parameters your code is using to simulate data? Are forecasts or predictions obtained by your method accurate? These kinds of questions can often only be answered by simulating fake data. Programmatically, simulating data like this largely involves calling functions that we have seen before (e.g. `rnorm()` in R or `np.random.choice()` in Python). This may or may not involve setting a pseudorandom number seed, first, for reproducibility.

Also, *benchmark data sets* are often readily available through specialized function calls.

Even though this chapter is written to teach you how to read in files into R and Python, you should not expect that you will know how to read in *all* data sets after reading this section. For both R and Python, there are an enormous amount of functions, different functions have different return types, different functions are suited for different file types, many functions are spread across a plethora of third party libraries, and many of these functions have an enormous amount of arguments. You will probably not be able to memorize everything. In my very humble opinion, I doubt you should want to.

Instead, **focus on developing your ability to identify and diagnose data input problems**. Reading in a data set correctly is often a process of trial-and-error. After attempting to read in a data set, always check the following items. Many of these points were previously mentioned in section [@\(data-frames-in-r\)](#). Some apply to reading in text data more than reading in structured data from a database, and vice versa.

1. Check that **the correct column separator was used, or the correct “fixed-width format” was expected**. If mistakes are made, data frame columns are going to be combined or split apart in weird ways, and often the wrong types are going to be used for pieces of data (e.g. “2,3” instead of 2 and 3.) Also, watch out for

when separators are found inside data elements or column names. For example, sometimes it's unclear whether people's names in the "last, first" format can be stored in one or two columns. Also, text data might surprise you with unexpected spaces or other whitespace is a common separator.

2. Check that **the column names were parsed and stored correctly**. Column names should not be stored as data in R/Python. Functions that read in data should not expect column names when they don't exist in the actual file.
3. Check that **empty space and metadata was ignored correctly**. Data descriptions are sometimes stored in the same file as the data itself, and that should be skipped over when it's being read in. Empty space between column names and data shouldn't be stored. This can occur at the beginning of the file, and even at the end of the file.
4. **Check that type choice and recognition of special characters are performed correctly**. Are letters stored as strings or as something else such as an R factor? Are dates and times stored as a special date/time type, or as strings? Is missing data correctly identified? Sometimes data providers use outrageous numbers like `-9999` to represent missing data—don't store that as a float or integer!
5. **Be ready to prompt R or Python to recognize a specific character encoding if you are reading in text data written in another language**. All text data has a character encoding, which is a mapping of numbers to characters. Any specific encoding will dictate what characters are recognizable in a program. If you try to read in data written in another language, the function you are using will likely complain about unrecognized characters. Fortunately, these errors and warnings are easily fixed by specifying a nondefault argument such as `encoding=` or `fileEncoding=`.

This is no small task. To make matters worse:

- you can't (or shouldn't) edit the raw data to suit your needs, to make it easier to read in. You have to work with what you are given. If you were allowed to edit, say, a text file you downloaded onto your own machine, you shouldn't—it will lead to code that doesn't run anywhere else. Additionally, if you abuse write privileges on your company's database, for example—that could be very dangerous as well.
- Data sets are often quite large, so manually checking each element is often impossible. In this situation you will have to resign yourself to checking the top and bottom of a data set, or maybe anticipate a specific place where problems are likely to appear.

9.2 Reading in Text Files with R

You've seen examples of `read.csv()` used earlier in the book, so it should not surprise you that this is one of the most common ways to read in data in R. Another important function is `read.table()`.

If you look at the source code for `read.csv()` (type the name of the function without parentheses into the console and press the <Enter> key), you will see it calls `read.table()`. The primary difference between these functions is default arguments. **Mind the default arguments.** Do not be completely averse to writing a long-line of code to read in a data set correctly. Or do, and choose the function with the best default arguments.

Consider the “Challenger USA Space Shuttle O-Ring Data Set”¹ from (Dua and Graff, 2017). The first few rows of the raw text file² looks like this.

```
6 0 66 50 1
6 1 70 50 2
6 0 69 50 3
```

It does not use commas as separators, and there is no header information, so `read.csv()` used with its default arguments will produce an incorrect result. It will miss the first row by counting it as a column name, and store everything in one column with the wrong type.

```
d <- read.csv("data/o-ring-erosion-only.data")
dim(d) # one row short, only 1 col
## [1] 22 1
typeof(d[,1])
## [1] "character"
```

Specifying `header=FALSE` fixes the column name issue, but `sep = " "` does not fix the separator issue.

```
d <- read.csv("data/o-ring-erosion-only.data",
              header=FALSE, sep = " ")
```

¹<https://archive.ics.uci.edu/ml/datasets/Challenger+USA+Space+Shuttle+O-Ring>

²Open raw text files with text editor programs, not with programs that perform any kind of processing. For instance, if you open it with Microsoft Excel, the appearance of the data will change, and important information helping you to read your data into R or Python will not be available to you.

```
str(d)
## 'data.frame':    23 obs. of  7 variables:
## $ V1: int  6 6 6 6 6 6 6 6 6 6 ...
## $ V2: int  0 1 0 0 0 0 0 0 1 1 ...
## $ V3: int  66 70 69 68 67 72 73 70 57 63 ...
## $ V4: int  NA NA NA NA NA NA 100 100 200 200 ...
## $ V5: int  50 50 50 50 50 50 NA NA NA 10 ...
## $ V6: int  NA NA NA NA NA NA 7 8 9 NA ...
## $ V7: int  1 2 3 4 5 6 NA NA NA NA ...
```

One space is strictly one space. Some rows have two, though. This causes there to be two too many columns filled with NAs.

After digging into the documentation a bit further, you will notice that `"` works for “one or more spaces, tabs, newlines or carriage returns.” This is why `read.table()`, with its default arguments, works well.

```
d <- read.table("data/o-ring-erosion-only.data")
str(d)
## 'data.frame':    23 obs. of  5 variables:
## $ V1: int  6 6 6 6 6 6 6 6 6 6 ...
## $ V2: int  0 1 0 0 0 0 0 0 1 1 ...
## $ V3: int  66 70 69 68 67 72 73 70 57 63 ...
## $ V4: int  50 50 50 50 50 50 100 100 200 200 ...
## $ V5: int  1 2 3 4 5 6 7 8 9 10 ...
```

This data set has columns whose widths are “fixed”, too. It is in “fixed width format” because any given column has all its elements take up a constant amount of characters. The third column has integers with two or three digits, but no matter what, each row has the same number of characters.

You may choose to exploit this and use a specialized function that reads in data in a fixed width format (e.g. `read.fwf()`). The frustrating thing about this approach, though, is that you have to specify what those widths are. This can be quite tedious, particularly if your data set has many columns and/or many rows. The upside though, is that the files can be a little bit smaller, because the data provider does not have to waste characters on separators.

In the example below, we specify widths that include blank spaces to the left of the digits. On the other hand, if we specified `widths=c(2,2,4,4,1)`, which includes spaces to the *right* of digits, then columns would have been recognized as `characters`.

```
d <- read.fwf("data/o-ring-erosion-only.data",
              widths = c(1,2,3,4,3)) # or try c(2,2,4,4,1)
str(d)
## 'data.frame':    23 obs. of  5 variables:
## $ V1: int  6 6 6 6 6 6 6 6 6 6 ...
## $ V2: int  0 1 0 0 0 0 0 0 1 1 ...
## $ V3: int  66 70 69 68 67 72 73 70 57 63 ...
## $ V4: int  50 50 50 50 50 50 100 100 200 200 ...
## $ V5: int  1 2 3 4 5 6 7 8 9 10 ...
```

If you need to read in some text data that does not possess a tabular structure, then you may need `readLines()`. This function will read in all of the text, separate each line into an element of a `character` vector, and will not make any attempt to parse lines into columns. Further processing can be accomplished using the techniques from [Section 3.9](#).

```
html_data <- readLines("data/Google.html", warn = FALSE)
head(html_data, 1)
## [1] "<!DOCTYPE html>"
```

Some of you may have had difficulty reading in the above data. This can happen if your machine’s default character encoding is different than mine. For instance, if your character encoding is “GBK”³, then you might get a warning message like “invalid input found on input connection.” This message means that your machine didn’t recognize some of the characters in the data set.

These errors are easy to fix, though, so don’t worry. Just specify an encoding argument in your function that reads in data.

```
tmp <- read.table("data/Google.html", sep = "~",
                  fileEncoding = "UTF-8") # makes errors disappear
```

9.3 Reading in Text Files with Pandas

A wide variety of different file formats can be read in with Pandas.⁴ I will only mention a few functions here.

³[https://en.wikipedia.org/wiki/GBK_\(character_encoding\)](https://en.wikipedia.org/wiki/GBK_(character_encoding))

⁴<https://pandas.pydata.org/pandas-docs/stable/reference/io.html>

Recall R has `read.table()` and `read.csv()`, and that they are very similar. In Pandas, `pd.read_csv()`⁵ and `pd.read_table()`⁶ have a lot in common, too. Their primary difference is the default column separator, as well.

Recall the O-Ring data from above. The columns are *not* separated by commas, so if we treat it as a comma-separated file, the resulting Pandas DataFrame is going to be missing all but one of its columns.

```
import pandas as pd
d = pd.read_csv("data/o-ring-erosion-only.data")
d.shape # one column and missing a row
## (22, 1)
d.columns # column labels are data
## Index(['6 0 66 50 1'], dtype='object')
```

By default, `pd.read_csv()` is expecting column labels, which is also a problem. Unlike R, though, the `header=` argument is not expected to be a Boolean. You will need to provide a `None`, instead. The separator needs to be just right, too.

```
pd.read_csv("data/o-ring-erosion-only.data",
            header=None, sep = " ").head(2) # 1 space: no
##      0  1  2  3  4  5  6
## 0  6  0  66 NaN 50.0 NaN 1.0
## 1  6  1  70 NaN 50.0 NaN 2.0
pd.read_csv("data/o-ring-erosion-only.data",
            header=None, sep = "\t").head(2) # tabs: no
##
##      0
## 0  6 0 66 50 1
## 1  6 1 70 50 2
pd.read_table("data/o-ring-erosion-only.data",
              header=None).head(2) # default sep is tabs, so no
##
##      0
## 0  6 0 66 50 1
## 1  6 1 70 50 2
pd.read_csv("data/o-ring-erosion-only.data",
            header=None, sep = "\s+").head(2) # 1 or more spaces: yes
##      0  1  2  3  4
## 0  6  0  66 50 1
## 1  6  1  70 50 2
```

⁵https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

⁶https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_table.html

Reading in fixed width files can be done in a way that is nearly identical to the way we did it in R. Here is an example.

```
d = pd.read_fwf("data/o-ring-erosion-only.data",
               widths = [1,2,3,4,3], header=None) # try [2,2,4,4,1]
d.info()
## <class 'pandas.core.frame.DataFrame'>
## RangeIndex: 23 entries, 0 to 22
## Data columns (total 5 columns):
## #   Column  Non-Null Count  Dtype
## ---  ---
## 0     0      23 non-null    int64
## 1     1      23 non-null    int64
## 2     2      23 non-null    int64
## 3     3      23 non-null    int64
## 4     4      23 non-null    int64
## dtypes: int64(5)
## memory usage: 1.0 KB
```

If you had chosen `widths=[2,2,4,4,1]`, instead, then the trailing whitespace will cause Pandas to recognize a `dtype` of `object`. The reason it is not recognized as a string is because strings can be of different length, and all string types specify a maximum length. If you want to enforce a maximum length, there may be some speed advantages. In the below example, we use `d.astype()`⁷ to convert two columns' types to `pd.StringDtype`⁸.

```
d = pd.read_fwf("data/o-ring-erosion-only.data",
               widths = [2,2,4,4,1], header=None)
list(d.dtypes)[:4]
## [dtype('int64'), dtype('int64'), dtype('O'), dtype('O')]
d = d.astype({2:'string', 3:'string'})
list(d.dtypes)[:4]
## [dtype('int64'), dtype('int64'), StringDtype, StringDtype]
```

Just like in R, you may run into an encoding issue with a file. For instance, the following will not work because the file contains Chinese characters. If you

⁷<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.astype.html>

⁸<https://pandas.pydata.org/docs/reference/api/pandas.StringDtype.html#pandas.StringDtype>

mostly work with UTF-8 files, you will receive a `UnicodeDecodeError` if you try to run the following code.

```
pd.read_csv("data/message.txt")
```

However, the error messages disappear when you specify `encoding="gbk"`.⁹

```
pd.read_csv("data/message.txt", encoding = "gbk")
## Empty DataFrame
## Columns: [ ]
## Index: [ ]
```

You may also read in unstructured, nontabular data with Python. Use the built-in `open()`¹¹ function to open up a file in read mode, and then use `f.readlines()` to return a list of strings.

```
f = open("data/Google.html", "r")
d = f.readlines()
d[:1]
## ['<!DOCTYPE html>\n']
print(type(d), type(d[0]))
## <class 'list'> <class 'str'>
```

9.4 Saving Data in R

Storing data is important for saving your progress. For example, sometimes running a script that performs data cleaning can take a very long time. Saving your progress might free you from the responsibility of running that script many times.

In R, there are many options for storing data. I will mention two: writing data out to a plain text file, and saving a serialized object.

⁹A list of more options of encodings that are built into Python, are available here.¹⁰

¹⁰<https://docs.python.org/3/library/codecs.html#standard-encodings>

¹¹<https://docs.python.org/3/library/functions.html#open>

9.4.1 Writing Out Tabular Plain Text Data in R

If you want to write out tabular data to a text file, use `write.table()` or `write.csv()`. There are two arguments that you must specify, at a minimum: the first argument is your R object (typically a `matrix` or `data.frame`), and the second argument is the file path on your hard drive.

Here is an example of writing out `d` to a file called `"oring_out.csv"`. I choose to include column names, but not row names. I also use commas to separate columns.

```
write.table(d, file = "data/oring_out.csv",  
            col.names = TRUE, row.names = FALSE, sep = ";")
```

The above will not print anything to the R console, but we can use a text editor to take a look at the raw text file on our hard drive. Here are the first three rows.

```
"V1";"V2";"V3";"V4";"V5"  
6;0;66;50;1  
6;1;70;50;2
```

9.4.2 Serialization in R

Alternatively you may choose to store your data in a **serialized** form. With this approach, you are still saving your data in a more permanent way to your hard drive, but it is stored in format that's usually more memory efficient.

Recall that a common reason for writing out data is to save your progress. When you want to save your progress, it is important to ask yourself: “is it better to save my progress as a serialized object, or as a raw text file?”



When making this decision, consider *versatility*. On the one hand, raw text files are more versatile and can be used in more places. On the other hand, versatility is often bug prone.

For example, suppose you want to save a cleaned up `data.frame`. Are you sure you will remember to store that column of strings as `character` and not a `factor`? Does any code that uses this `data.frame` require that this column be in this format?

For instance, let's save the object `d` in a file called `oring.rds`.

```
saveRDS(d, file = "data/oring.rds")
rm(d)
exists("d")
## [1] FALSE
```

After it is saved with `saveRDS()`, we are free to delete the variable with `rm()`, because it can be read back in later on. To do this, call `readRDS()`. This is file has a special format that is recognized by R, so you will not need to worry about any of the usual struggles that occur when reading in data from a plain text file. Additionally, `.rds` files are typically smaller—`oring.rds` is only 248 bytes, while `"oring_out.csv"` is 332 bytes.

```
d2 <- readRDS(file = "data/oring.rds")
head(d2, 3)
##      V1 V2 V3 V4 V5
## 1    6  0 66 50  1
## 2    6  1 70 50  2
## 3    6  0 69 50  3
```

You can serialize multiple objects at once, too! Convention dictates that these files end with the `.RData` suffix. Save your entire global environment with `save()` or `save.image()`, and bring it back with `load()` or `attach()`.

```
rm(list=ls()) # remove everything
a <- 1
b <- 2
save.image(file = "data/my-current-workspace.RData")
rm(list=ls())
load("data/my-current-workspace.RData")
ls() # print all objects in your workspace
## [1] "a" "b"
```


9.5 Saving Data in Python

9.5.1 Writing Out Tabular Plain Text Data in Python

You can write out tabular data with a variety of `DataFrame` methods that are named `to_*`.¹² `pd.DataFrame.to_csv()`¹³ has a lot of common with `write.csv()` in R. Below we write out `d` to a file called `oring_out2.csv`.

```
import pandas as pd
d = pd.read_csv("data/o-ring-erosion-only.data",
                header=None, sep = "\s+")
d.to_csv("data/oring_out2.csv",
        header=True, index=False, sep = ",")
```

Here is how the first few rows of that file looks in a text editor.

```
0,1,2,3,4
6,0,66,50,1
6,1,70,50,2
```

9.5.2 Serialization in Python

Serialization functionality is readily available in Python, just like it is in R. In Python, the `pickle`¹⁴ and `cPickle` libraries are probably the most commonly used. Serializing objects with these libraries is known as *pickling* an object.

Pandas has a `.to_pickle()`¹⁵ wrapper method attached to every `DataFrame`. Once the pickled object is saved, the file can be read back into Python with `pd.read_pickle()`¹⁶. These functions are extremely convenient, because they call all the required `pickle` code and hide a decent amount of complexity.

Here is an example of writing out `d` and then reading the pickled object back in. In Python 3, the file suffix for pickled objects is usually `.pickle`, but there are many other choices.

¹²<https://pandas.pydata.org/pandas-docs/stable/reference/io.html#input-output>

¹³https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html#pandas.DataFrame.to_csv

¹⁴<https://docs.python.org/3/library/pickle.html>

¹⁵https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_pickle.html

¹⁶https://pandas.pydata.org/docs/reference/api/pandas.read_pickle.html#pandas.read_pickle

```
d.to_pickle("data/oring.pickle")
del d
d_is_back = pd.read_pickle("data/oring.pickle")
d_is_back.head(2)
##      0  1  2  3  4
## 0  6  0 66 50  1
## 1  6  1 70 50  2
```

Unfortunately, "oring.pickle" is much larger (1,676 bytes) than the original text file "o-ring-erosion-only.data" (322 bytes). This is for two reasons. First, the original data set is small, so the overhead of pickling this object is relatively pronounced, and second, we are not taking advantage of any compression. If you use something like `d_is_back.to_pickle("data/oring.zip")` it will become smaller.

In Python, unlike in R, it is more difficult to serialize all of the objects you currently have in memory. It is possible, but it will likely require the use of a third-party library.

Speaking of third-party code, there are many that provide alternative serialization solutions in both R and Python. I do not discuss any in this text. However, I will mention that some of them may provide combinations of the following: an increase in read and write speed, a decrease in required memory, improved security¹⁷, improved human readability and interoperability between multiple programming languages. If any of these sound potentially beneficial, I encourage you to conduct further research.

9.6 Exercises

9.6.1 R Questions

1.

Consider again the data set called "gspc.csv", which contains daily open, high, low and close values for the S&P500 Index.

¹⁷The documentation for `pickle`¹⁸ mentions that the library is "not secure against erroneous or maliciously constructed data" and recommends that you "[n]ever unpickle data received from an untrusted or unauthenticated source."

¹⁸<https://docs.python.org/2/library/pickle.html>

- a) Read in this data set as a `data.frame`, and call it `myData`. Do not include the code that achieves this in your assignment submission.
- b) Write out this object as `myData.rds`. After you are finished, remove `myData` from memory. Do not include the code that achieves this in your assignment submission.
- c) Read in `myData.rds`, and store the variable as `financialData`. *Do* include the code that achieves this in your project submission. Make sure this code assumes that `myData.rds` is in the same folder as the code file `io_lab.R`.

9.6.2 Python Questions

1.

We will use the "Google.html" data set mentioned in the chapter.

- a) Use `open()` to open the "Google.html" file. Store the output of the function as `my_file`.
- b) Use the `.readlines()` method of the file to write the contents of the file as a `list` called `html_data`
- c) Coerce the `list` to a `DataFrame` with one column called `html`
- d) Create a `Series` called `nchars_ineach` that stores the number of characters in each line of text. Hint: the `Series.str` attribute has a lot of helpful methods¹⁹.
- e) Create an `int`-like variable called `num_div_tags` that holds the total number of times the phrase "<div>" appears in the file.

2.

Consider the data set called "gspc.csv", which contains daily open, high, low and close values for the S&P500 Index.

- a) Read in this data set as a `DataFrame`, and call it `my_data`. Do not include the code that achieves this in your assignment submission.
- b) Write out this object as "`my_data.pickle`". After you are finished, remove `my_data` from memory. Do not include the code that achieves this in your assignment submission.
- c) Read in "`my_data.pickle`", and store the variable as `financial_data`. *Do* include the code that achieves this in your project submission. Make sure this code assumes that "`my_data.pickle`" is in the same folder as the code file `io_lab.py`.

¹⁹<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#api-series-str>

10

Using Third-Party Code

Before using third-party code, it must first be installed. After it is installed, it must be “loaded in” to your session. I will describe both of these steps in R and Python.

10.1 Installing Packages in R

In R, there are thousands of free, user-created **packages** (Lander, 2017). You can download most of these from the *Comprehensive R Archive Network*¹. You can also download packages from other publishing platforms like Bioconductor², or Github³. Installing from CRAN is more commonplace, and extremely easy to do. Just use the `install.packages()` function. This can be run inside your R console, so there is no need to type things into the command line.

```
install.packages("thePackage")
```

10.2 Installing Packages in Python

In Python, installing packages is more complicated. Commands must be written in the command line, and there are multiple package managers. This isn’t surprising, because Python is used more extensively than R in fields other than data science.

If you followed the suggestions provided in earlier in the text, then you installed Anaconda. This means you will usually be using the `conda` command⁴. Point-and-click interfaces are made available as well.

¹<https://cran.r-project.org/>

²<https://www.bioconductor.org/>

³<https://github.com/>

⁴<https://docs.anaconda.com/anaconda/user-guide/tasks/install-packages/>

```
conda install the_package
```

There are some packages that will not be available using this method. For more information on that situation, see here.⁵

10.3 Loading Packages in R

After they are installed on your machine, third-party code will need to be “loaded” into your R or Python session.

Loading in a package is relatively simple in R, however complications can arise when different variables share the same name. This happens relatively often because

- it’s easy to create a variable in the global environment that has the same name as another object you don’t know about, and
- different packages you load in sometimes share names accidentally.

Starting off with the basics, here’s how to load in a package of third-party code. Just type the following into your R console.

```
library(thePackage)
```

You can also use the `require()` function, which has slightly different behavior when the requested package is not found.

To understand this more deeply, we need to talk about **environments** again. We discussed these before in 6.3, but only in the context of user-defined functions. When we load in a package with `library()`, we make its contents available by putting it all in an environment for that package.

An environment⁶ holds the names of objects. There are usually several environments, and each holds a different set of functions and variables. All the variables you define are in an environment, every package you load in gets its own environment, and all the functions that come in R pre-loaded have their own environment.

⁵<https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-pkgs.html#install-non-conda-packages>

⁶<https://cran.r-project.org/doc/manuals/R-lang.html#Environment-objects>



Formally, each environment is pair of two things: a **frame** and an **enclosure**. The frame is the set of symbol-value pairs, and the enclosure is a pointer to the parent environment. If you’ve heard of a *linked list* in a computer science class, it’s the same thing.

Moreover, all of these environments are connected in a chain-like structure. To see what environments are loaded on your machine, and what order they were loaded in, use the `search()` function. This displays the **search path**⁷, or the ordered sequence of all of your environments.

Alternatively, if you’re using RStudio, the search path, and the contents of each of its environments, are displayed in the “Environment” window (see [Figure 10.1](#)). You can choose which environment you’d like to look at by selecting it from the drop-down menu. This allows you to see all of the variables in that particular environment. The **global environment** (i.e. `".GlobalEnv"`) is displayed by default, because that is where you store all the objects you are creating in the console.

Global Environment	
Data	
dataSet	428 obs. of 15 variables
firstThreeRows	3 obs. of 5 variables
firstTwoCols	150 obs. of 2 variables
irisData	150 obs. of 5 variables
myList	List of 3
results	List of 13
setosaOnly	50 obs. of 5 variables
Values	
a	2
allElements	num [1:6] 99 2 99 4 99 6
b	num [1:3] 999 2 3
babyData	num [1:5] 0 -1 9 NA 21
firstCol	num [1:150] 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
missileLaunchCodesSet	FALSE
myDouble	3.2
myInt	1
mySum	4.2
responses	Ord.factor w/ 3 levels "hate it"<"it's okay"<...: 3 2 3 3 2 ...

FIGURE 10.1: The environment window in RStudio.

When you call `library(thePackage)`, the package has an environment created for it, and it is *inserted between the global environment, and the most recently loaded package*. When you want to access an object by name, R will first search

⁷<https://cran.r-project.org/doc/manuals/R-lang.html#Search-path>

the global environment, and then it will traverse the environments in the search path in order. These has a few important implications.

- First, **don't define variables in the global environment that are already named in another environment.** There are many variables that come pre-loaded in the base package (to see them, type `ls("package:base")`), and if you like using a lot of packages, you're increasing the number of names you should avoid using.
- Second, **don't `library()` in a package unless you need it, and if you do, be aware of all the names it will mask it packages you loaded in before.** The good news is that `library` will often print warnings letting you know which names have been masked. The bad news is that it's somewhat out of your control—if you need two packages, then they might have a shared name, and the only thing you can do about it is watch the ordering you load them in.
- Third, don't use `library()` inside code that is `source()`'d in other files. For example, if you attach a package to the search path from within a function you defined, anybody that uses your function loses control over the order of packages that get attached.

All is not lost if there is a name conflict. The variables haven't disappeared. It's just slightly more difficult to refer to them. For instance, if I load in `Hmisc` (Harrell Jr et al., 2021), I get the warning warning that `format.pval` and `units` are now masked because they were names that were in "package:base". I can still refer to these masked variables with the double colon operator (`::`).

```
library(Hmisc)
# this now refers to Hmisc's format.pval
# because it was loaded more recently
format.pval
Hmisc::format.pval # in this case is the same as above
# the below code is the only way
# you can get base's format.pval now
base::format.pval
```

10.4 Loading Packages in Python

In Python, you use the `import` statement to access objects defined in another file. It is slightly more complicated than R's `library()` function, but it is

also more flexible. To make the contents of a package called, say, `the_package` available, type *one of the following* inside a Python session.

```
import the_package
import the_package as tp
from the_package import *
```

To describe the difference between these three approaches, as well as to highlight the important takeaways and compare them with the important takeaways in the last section, we need to discuss what a Python module is, what a package is, and what a Python namespace is.⁸

- A Python **module**⁹ is a `.py` file, separate from the one you are currently editing, with function and/or object definitions in it.¹⁰
- A package¹¹ is a group of modules.¹²
- A **namespace**¹³ is “a mapping from names to objects.”

With these definitions, we can define **importing**. According to the Python documentation¹⁴, “[t]he import statement combines two operations; it searches for the named module, then it binds the results of that search to a name in the local scope.”

The sequence of places Python looks for a module is called the search path. This is not the same as R’s search path, though. In Python, the search path is a list of places to look for *modules*, not a list of places to look for variables. To see it, `import sys`, then type `sys.path`.

After a module is found, the variable names inside it become available to the **importing** module. These variables are available in the global scope, but the names you use to access them will depend on what kind of `import` statement you used. From there, you are using the same scoping rules that we described in 6.6, which means the LEGB acronym still applies.

⁸I am avoiding any mention of R’s namespaces and modules. These are things that exist, but they are different from Python’s namespaces and modules, and are not within the scope of this text.

⁹<https://docs.python.org/3/tutorial/modules.html>

¹⁰The scripts you write are modules. They usually come with the intention of being run from start to finish. Other non-script modules are just a bag of definitions to be used in other places.

¹¹<https://docs.python.org/3/tutorial/modules.html#packages>

¹²Sometimes a package is called a *library* but I will avoid this terminology.

¹³<https://docs.python.org/3/tutorial/classes.html#python-scopes-and-namespaces>

¹⁴<https://docs.python.org/3/reference/import.html#the-import-system>

In both languages, an (unqualified) variable name can only refer to one object at any time. This does not necessarily have anything to do with using third-party code—you can redefine objects, but don't expect to be able to access the old object after you do it.

The same thing can happen when you use third-party code.

- In R, you have to worry about the order of `library()` and `require()` calls, because there is potential *masking* going on.
- If you don't want to worry about masking, don't use `library()` or `require()`, and just refer to variables using the `::` operator (e.g. `coolPackage::specialFunc()`).
- In Python, loading packages using either the `import package` format or the `import package as p` format means you do not need to worry about the order of imports because you will be forced to qualify variable names (e.g. `package.func()` or `p.func()`).



- In Python, if you load third-party code using either `from package import foo` or `from package import *`, you won't have to qualify variable names, but imported objects will overwrite any variables that happen to have the same name as something you're importing.

The way variable names are stored are only slightly different between R and Python.

- Python namespaces are similar to R environments in that they hold name-value pairs; however
- Python namespaces are unlike R environments in that they are not arranged into a sorted list.
- Also, Python modules may be organized into a nested or tree-like structure, whereas R packages will always have a flat structure.

10.4.1 Importing Examples

In the example below, we import the entire `numpy` package in a way that lets us refer to it as `np`. This reduces the amount of typing that is required of us, but it also protects against variable name clashing. We then use the `normal()` function to simulate normal random variables. This function is in the `random` sub-module¹⁵, which is a sub-module in `numpy` that collects all of the pseudorandom number generation functionality together.

¹⁵<https://numpy.org/doc/stable/reference/random/index.html?highlight=random#module-numpy.random>

```
import numpy as np # import all of numpy
np.random.normal(size=4)
## array([ 0.38420175,  1.11162875,  0.04221629, -0.02261209])
```

This is one use of the dot operator (.). It is also used to access attributes and methods of objects (more information on that will come later in [Chapter 14](#)). `normal` is *inside of* `random`, which is itself inside of `np`.

As a second example, suppose we were interested in the `stats` sub-module¹⁶ found inside the `scipy` package. We could import all of `scipy`, but just like the above example, that would mean we would need to consistently refer to a variable's module, the sub-module, and the variable name. For long programs, this can become tedious if we had to type `scipy.stats.norm` over and over again. Instead, let's import the sub-module (or sub-package) and ignore the rest of `scipy`.

```
from scipy import stats
stats.norm().rvs(size=4)
## array([-1.06461554,  1.18604057, -0.16099583,  1.50089071])
```

So we don't have to type `scipy` every time we use something in `scipy.stats`.

Finally, we can import the function directly, and refer to it with only one letter. This is highly discouraged, though. We are much more likely to accidentally use the name `n` twice. Further, `n` is not a very descriptive name, which means it could be difficult to understand what your program is doing later.

```
from numpy.random import normal as n
n(size=4)
## array([ 1.01757426,  0.88102327, -0.65346708, -0.683219  ])
```

Keep in mind, you're always at risk of accidentally re-using names, even if you aren't importing anything. For example, consider the following code.

```
n = 3 # now you can't use n as a function
n()
```

This is very bad, because now you cannot use the `n()` function that was imported from the `numpy.random` sub-module earlier. In other words, it is

¹⁶<https://docs.scipy.org/doc/scipy/reference/tutorial/stats.html>

longer *callable*. The error message from the above code will be something like `TypeError: 'int' object is not callable`.

Use the `dir()` function to see what is available inside a module. Here are a few examples. Type them into your own machine to see what they output.

```
dir(np) # numpy stuff
dir(__builtins__) #built-in stuff
```

10.5 Exercises

1.

What are important differences in the package installation procedures of R and Python? Select all that apply.

- a) Installing R packages can be done from within R, while installing packages in Python can be done in the command line.
- b) Installing R packages can usually be done with the same function `install.packages()`, while installing packages in Python can be done with a variety of package installers such as `pip install` and `conda install`.
- c) There is only one package repository for R, but many for Python.
- d) There is only one package repository for Python, but many for R.

2.

What are important similarities and differences in the package loading procedures of R and Python? Select all that apply.

- a) R and Python both have a search path.
- b) R's `::` operator is very similar to Python's `.` operator because they can both help access variable names inside packages.
- c) Python namespaces are unlike R environments in that they are not arranged into a sorted list.
- d) `library(package)` in R is similar to `from package import *` in Python because it will allow you to refer to all variables in `package` without qualification.
- e) Python packages might have sub-modules whereas R's packages do not.

3.

In Python, which of the following is, generally speaking, the best way to import?

- `import the_package`
- `from the_package import *`
- `import the_package as tp`

4.

In Python, which of the following is, generally speaking, the worst way to import?

- `import the_package`
- `from the_package import *`
- `import the_package as tp`

5.

In R, if you want to use a function `func()` from `package`, do you always have to use `library(package)` or `require(package)` first?

- Yes, otherwise `func()` won't be available.
- No, you can just use `package::func()` without calling any function that performs pre-loading.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

11

Control Flow

11.1 Conditional Logic

We discussed Boolean/logical objects in 2. We used these for

- counting the number of times a condition appeared, and
- subsetting vectors and data frames.

Another way to use them is to conditionally execute code. You can choose to execute code depending on whether or not a Boolean/logical value is true or not.

This is what an if statement looks like In R¹:

```
myName <- "Taylor"
if(myName != "Clare"){
  print("you are not Clare")
}
## [1] "you are not Clare"
```

In Python², you don't need curly braces, but the indentation needs to be just right, and you need a colon (Lutz, 2013).

```
my_name = "Taylor"
if my_name == "Taylor":
    print("hi Taylor")
## hi Taylor
```

There can be more than one test of truth. To test alternative conditions, you can add one or more `else if` (in R) or `elif` (in Python) blocks. The first block with a Boolean that is found to be true will execute, and none of the resulting conditions will be checked.

¹<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#if>

²<https://docs.python.org/3/tutorial/controlflow.html#if-statements>

If no `if` block or `else if/elif` block executes, an `else` block will always execute. That's why `else` blocks don't need to look at a Boolean. Whether they execute only depends on the Booleans in the previous blocks. If there is no `else` block, and none of the previous conditions are true, nothing will execute.

```
# in R
food <- "muffin"
if(food == "apple"){
  print("an apple a day keeps the doctor away")
}else if(food == "muffin"){
  print("muffins have a lot of sugar in them")
}else{
  print("neither an apple nor a muffin")
}
## [1] "muffins have a lot of sugar in them"
```

```
# in Python
my_num = 42.999
if my_num % 2 == 0:
    print("my_num is even")
elif my_num % 2 == 1:
    my_num += 1
    print("my_num was made even")
else:
    print("you're cheating by not using integers!")
## you're cheating by not using integers!
```

11.2 Loops

One line of code generally does one “thing,” unless you're using loops. Code written inside a loop will execute many times.

The most common loop for us will be a `for` loop. A simple `for` loop in R³ might look like this

³<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#for>

```
#in R
myLength <- 9
r <- vector(mode = "numeric", length = myLength)
for(i in seq_len(myLength)){
  r[i] <- i
}
r
## [1] 1 2 3 4 5 6 7 8 9
```

1. `seq_len(myLength)` gives us a `vector`.
2. `i` is a variable that takes on the values found in `seq_len(myLength)`.
3. Code inside the loop (inside the curly braces), is repeatedly executed, and it may or may not reference the dynamic variable `i`.

Here is an example of a `for` loop in Python⁴ (Lutz, 2013):

```
#in Python
my_length = 9
r = []
for i in range(my_length):
  r.append(i)
r
## [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

1. Unsurprisingly, Python's syntax opts for indentation and colons instead of curly braces and parentheses.
2. Code inside the loop (indented underneath the `for` line), is repeatedly executed, and it may or may not reference the dynamic variable `i`.
3. `for` loops in Python are more flexible because they iterate over many different types of data structures—in this case `range()` returns an object of type `range`⁵.
4. The `range`⁶ doesn't generate all the numbers in the sequence at once, so it saves on memory. This can be quite useful if you're looping over a large collection, and you don't need to store everything all at once. However, in this example, `r` is a list that *does* store all the consecutive integers.

⁴<https://docs.python.org/3/tutorial/controlflow.html#for-statements>

⁵<https://docs.python.org/3/library/stdtypes.html#range>

⁶<https://docs.python.org/3/tutorial/controlflow.html#the-range-function>

Loops are for repeatedly executing code. `for` loops are great when you know the number of iterations needed ahead of time. If the number of iterations is not known, then you'll need a `while` loop. While loops will only terminate after a condition is found to be true. Here are some examples in R⁷ and in Python⁸.

```
# in R
keepGoing <- TRUE
while(keepGoing){
  oneOrZero <- rbinom(1, 1, .5)
  print(paste("oneOrZero:", oneOrZero))
  if(oneOrZero == 1)
    keepGoing <- FALSE
}
## [1] "oneOrZero: 0"
## [1] "oneOrZero: 0"
## [1] "oneOrZero: 1"
```

```
# in Python
keep_going = True
while keep_going:
    one_or_zero = np.random.binomial(1, .5)
    print("one_or_zero: ", one_or_zero)
    if one_or_zero == 1:
        keep_going = False
## one_or_zero:  0
## one_or_zero:  1
```

⁷<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#while>

⁸https://docs.python.org/3/reference/compound_stmts.html#while

Here are some tips for writing loops:

1. If you find yourself copying and pasting code, changing only a small portion of text on each line of code, you should consider using a loop.
2. If a `for` loop works for something you are trying to do, first try to find a replacement function that does what you want. The examples above just made a `vector/list` of consecutive integers. There are many built in functions that accomplish this. Avoiding loops in this case would make your program shorter, easier to read, and (potentially) much faster.
3. A third option between looping, and a built-in function, is to try the functional approach. This will be explained more in the last chapter.
4. Watch out for **off-by-one** errors^a. Iterating over the wrong sequence is a common mistake, considering
 - Python starts counting from 0, while R starts counting from 1
 - sometimes iteration `i` references the `i-1`th element of a container
 - The behavior of loops is sometimes more difficult to understand if they're using `break`^b or `continue/next` statements^c.
5. *Don't hardcode variables.* In other words, don't write code that is specific to particulars of your script's current draft. Write code that will still run if your program is fed different data, or if you need to calculate something else that's closely-related (e.g. run the same calculations on different data sets, or vary the number of simulations, or make the same plot several times in similar situations, etc.). I can guarantee that most of the code you write will need to be run in many different situations. If, at every time you decide to make a change, you need to hunt down multiple places and make multiple changes, there is a nontrivial probability you will miss at least one. As a result, you will introduce a bug into your program, and waste (sometimes a lot of) time trying to find it.
6. Watch out for infinite `while` loops. Make sure that your stopping criterion is guaranteed to eventually become true.

^ahttps://en.wikipedia.org/wiki/Off-by-one_error

^b<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Looping>

^c<https://docs.python.org/3/tutorial/controlflow.html#break-and-continue-statements-and-else-clauses-on-loops>

Python also provides an alternative way to construct lists similar to the one we constructed in the above example. They are called **list comprehensions**⁹. These are convenient because you can incorporate iteration and conditional logic in one line of code.

```
[3*i for i in range(10) if i%2 == 0]
## [0, 6, 12, 18, 24]
```

You might also have a look at *generator expressions*¹⁰ and *dictionary comprehensions*¹¹.

R can come close to replicating the above behavior with vectorization, but the conditional part is hard to achieve without subsetting.

```
3*seq(0,9)[seq(0,9)%%2 == 0]
## [1] 0 6 12 18 24
```

11.3 Exercises

11.3.1 R Questions

1.

Suppose you have a vector of numeric data: $x_1, \dots, x_n$. Write a function called `cappedMoveAve(dataVector)` that takes in a vector and returns a 3-period “capped” moving average. Make sure to use a `for` loop. The formula you should use is

$$y_t = \min \left(10, \frac{1}{3} \sum_{i=0}^2 x_{t-i} \right). \quad (11.1)$$

The function should return $y_1, \dots, y_n$ as a vector. Let $y_1 = y_2 = 0$.

2.

Say we have a target¹² distribution that we want to sample from:

⁹<https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions>

¹⁰<https://www.python.org/dev/peps/pep-0289/>

¹¹<https://www.python.org/dev/peps/pep-0274/>

¹²This is the density of a $\text{Beta}(3, 2)$ random variable, if you’re curious.

$$p(x) = \begin{cases} \frac{x^2(1-x)}{\int_0^1 y^2(1-y)dy} & 0 < x < 1 \\ 0 & \text{otherwise} \end{cases}. \quad (11.2)$$

The denominator, $\int_0^1 y^2(1-y)dy$, is the target’s normalizing constant. You might know how to solve this integral (it’s equal to $1/12$), but let’s pretend for the sake of our example that it’s too difficult for us. We want to sample from $p(x)$ while only being able to evaluate (not sample) from its unnormalized version $f(x) := x^2(1-x)$. This is a situation that arises often—wanting to sample from some complicated distribution whose density you can only evaluate up to a constant of proportionality.

Next, let’s choose a uniform distribution for our *proposal distribution*: $q(x) = 1$ if $0 < x < 1$. This means we will sample from this distribution, because it’s easier. We just need to “adjust” our samples somehow, because it’s not the same as our target.

We can plot all three functions (see [Figure 11.1](#)). The area under the $p(x)$ and $q(x)$ curves is 1, because they are true probability density functions. $f(x)$, however, is not.

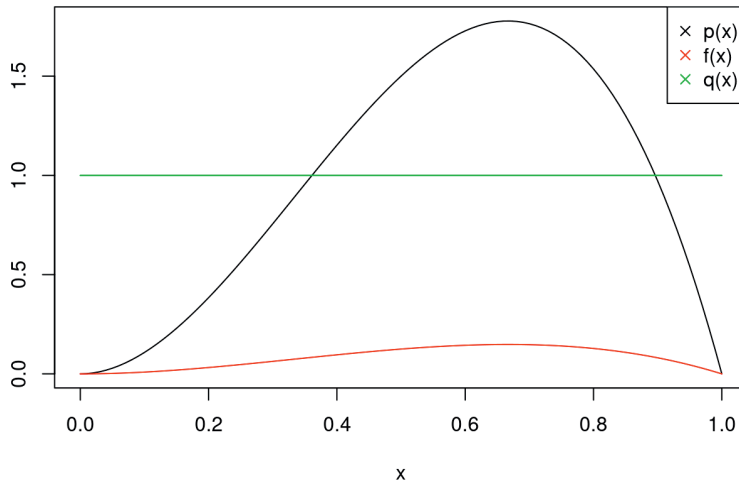


FIGURE 11.1: Visualizing our three functions.

Note that this algorithm allows for other proposal distributions. The only requirement of a proposal distribution is that its range of possible values must subsume the range of possible values of the target.

- a) Write a function called `arSamp(n)` that samples from $p(x)$ using accept-reject sampling. It should take a single argument that is equal to the number of samples desired. Below is one step of the accept-reject algorithm. You will need to do many iterations of this. The number of iterations will be random, because some of these proposals will not be accepted.

Algorithm 1: Accept-Reject Sampling (One Step)

- i. Find M such that $M > \frac{f(x)}{q(x)}$ for all possible x (the smaller the better).
- ii. Sample X from $q(x)$.
- iii. Sample $Y \mid X$ from Bernoulli $\left(\frac{f(X)}{q(X)M}\right)$.
- iv. If $Y = 1$, then return X .
- v. Otherwise, return nothing.

3.

Write a function called `multiplyTwoMats(A,B)` that performs matrix multiplication. It should take two `matrix` arguments: `A` and `B`. Then it should return the matrix product `AB`. Use two nested `for` loops to write this function. Make sure to test this function against the usual tool you use to multiply matrices together: `%*%`.

4.

Suppose you are trying to predict a value of Y given some information about a corresponding independent variable x . Suppose further that you have a historical data set of observations $(x_1, y_1), \dots, (x_n, y_n)$. One approach for coming up with predictions is to use **Nadaraya—Watson Kernel Regression** (Nadaraya, 1964) (Watson, 1964). The prediction this approach provides is simply a weighted average of all of the historically-observed data points $y_1, \dots, y_n$. The weight for a given y_i will be larger if x_i is “close” to the value x that you are obtaining predictions for. On the other hand, if x_j is far away from x , then the weight for y_j will be relatively small, and so this data point won’t influence the prediction much.

Write a function called `kernReg(xPred,xData,yData,kernFunc)` that computes the Nadaraya—Watson estimate of the prediction of Y given $X = x$. Do not use a `for` loop in your function definition. The formula is

$$\sum_{i=1}^n \frac{K(x - x_i)}{\sum_{j=1}^n K(x - x_j)} y_i, \quad (11.3)$$

where x is the point you're trying to get a prediction for.

- Your function should return one floating point number.
- The input `xPred` will be a floating point number.
- The input `xData` is a one-dimensional vector of numerical data of independent variables.
- The input `yData` is a one-dimensional vector of numerical data of dependent variables.
- `kernFunc` is a function that accepts a numeric vector and returns a floating point. It's vectorized.

Below is some code that will help you test your predictions. The kernel function, `gaussKernel()`, implements the Gaussian kernel function $K(z) = \exp[-z^2/2]/\sqrt{2\pi}$. Notice the creation of `preds` was commented out. Use a `for` loop to generate predictions for all elements of `xTest` and store them in the vector `preds`.

```
gaussKernel <- dnorm
xTest <- seq(0,10,length.out= 100)
yTest <- xTest*2
# preds <- ?
plot(xTest, yTest, type = "b")
lines(xTest, preds, col = "red")
```

11.3.2 Python Questions

1.

Suppose you go to the casino with 10 dollars. You decide that your policy is to play until you go broke, or until you triple your money. The only game you play costs \$1 to play. If you lose, you lose that dollar. If you win, you get another \$1 in addition to getting your money back.

- Write a function called `sim_night(p)` that simulates your night of gambling. Have it return a Pandas Series with the running balance of money you have over the course of a night. For example, if you lose 10 games in a row, and go home early, the returned Series contains 9, ..., 1, 0. This function will only take one input, `p`, which is the probability of winning any/every game you play.

- b) Use a `for` loop to call your function 5000 times with probability `p=.5`. Each time, store the number of games played. Store them all in a Numpy array or Pandas Series called `simulated_durations`.
 - c) Take the average of `simulated_durations`. This is your Monte Carlo estimate of the expected duration. How does it compare with what you think it should be theoretically?
 - d) Perform the same analysis to estimate the expected duration when $p = .7$. Store your answer as a `float` called `expec_duration`.
- 2.

Suppose you have the following data set. Please include the following snippet in your submission.

```
import numpy as np
import pandas as pd
my_data = pd.read_csv("sim_data.csv", header=None).values.flatten()
```

This question will demonstrate how to implement **The Bootstrap** (Efron, 1979), which is a popular nonparametric approach to understand the distribution of a statistic of interest. The main idea is to calculate your statistic over and over again on *bootstrapped* data sets, which are data sets randomly chosen, with replacement, from your original data set `my_data`. Each bootstrapped data set is of the same size as the original data set, and each bootstrapped data set will yield one statistic. Collect all of these random statistics, and it is a good approximation to the statistic's theoretical distribution.

- a) Calculate the mean of this data set and store it as a floating point number called `sample_mean`.
- b) Calculate 5,000 bootstrap sample means. Store them in a Numpy array called `bootstrapped_means`. Use a `for` loop, and inside the loop, sample with replacement 1000 times from the length 1000 data set. You can use the function `np.random.choice()` to accomplish this.
- c) Calculate the sample mean of these bootstrapped means. This is a good estimate of the *theoretical* mean/expectation of the sample mean. Call it `mean_of_means`.
- d) Calculate the sample variance of these bootstrapped means. This is a good estimate of the *theoretical* variance of the sample mean. Call it `var_of_means`.

3.

Write a function called `ar_samp(n)` that samples from $p(x)$ using accept-reject sampling. Use any proposal distribution that you'd like. It should take a single argument that is equal to the number of samples desired. Sample from the following target:

$$p(x) \propto f(x) := \exp[\cos(2\pi x)]x^2(1-x), \quad 0 < x < 1. \quad (11.4)$$



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

12

Reshaping and Combining Data Sets

12.1 Ordering and Sorting Data

Sorting a data set, in ascending order, say, is a common task. You might need to do it because

1. ordering and ranking is commonly done in *nonparametric statistics*,
2. you want to inspect the most “extreme” observations in a data set,
3. it’s a pre-processing step before generating visualizations.

In R, it all starts with **vectors**. There are two common functions you should know: `sort()` and `order()`. `sort()` returns the sorted *data*, while `order()` returns the *order indexes*.

```
sillyData <- rnorm(5)
print(sillyData)
## [1]  0.3903776  0.5796584  1.4929115  0.3704896 -1.3450719
sort(sillyData)
## [1] -1.3450719  0.3704896  0.3903776  0.5796584  1.4929115
order(sillyData)
## [1] 5 4 1 2 3
```

`order()` is useful if you’re sorting a data frame by a particular column. Below, we inspect the top 5 most expensive cars in an example data set (sas, 2021). Notice that we need to clean up the **MSRP** (a **character** vector) a little first. We use the function `gsub()` to find patterns in the text, and replace them with the empty string.

```
carData <- read.csv("data/cars.csv")
noDollarSignMSRP <- gsub("$", "", carData$MSRP, fixed = TRUE)
carData$cleanMSRP <- as.numeric(gsub(",", "", noDollarSignMSRP,
                                   fixed = TRUE))
```

```
rowIndices <- order(carData$cleanMSRP, decreasing = TRUE)[1:5]
carData[rowIndices,c("Make", "Model", "MSRP", "cleanMSRP")]
```

```
##           Make           Model    MSRP cleanMSRP
## 335      Porsche      911 GT2 2dr $192,465    192465
## 263 Mercedes-Benz      CL600 2dr $128,420    128420
## 272 Mercedes-Benz SL600 convertible 2dr $126,670    126670
## 271 Mercedes-Benz      SL55 AMG 2dr $121,770    121770
## 262 Mercedes-Benz      CL500 2dr   $94,820     94820
```

In Python, Numpy has `np.argsort()`¹ and `np.sort()`².

```
import numpy as np
silly_data = np.random.normal(size=5)
print(silly_data)
## [-0.52817175 -1.07296862  0.86540763 -2.3015387  1.74481176]
print( np.sort(silly_data) )
## [-2.3015387 -1.07296862 -0.52817175  0.86540763  1.74481176]
np.argsort(silly_data)
## array([3, 1, 0, 2, 4])
```

For Pandas' DataFrames, most of the functions I find useful are methods attached to the `DataFrame` class. That means that, as long as something is inside a `DataFrame`, you can use dot notation.

```
import pandas as pd
car_data = pd.read_csv("data/cars.csv")
car_data['no_dlr_msrp'] = car_data['MSRP'].str.replace("$", "",
                                                    regex = False)
no_commas = car_data['no_dlr_msrp'].str.replace(",", "")
car_data['clean_MSRP'] = no_commas.astype(float)
car_data = car_data.sort_values(by='clean_MSRP', ascending = False)
car_data[["Make", "Model", "MSRP", "clean_MSRP"]].head(5)
##           Make           Model    MSRP clean_MSRP
## 334      Porsche      911 GT2 2dr $192,465    192465.0
## 262 Mercedes-Benz      CL600 2dr $128,420    128420.0
## 271 Mercedes-Benz SL600 convertible 2dr $126,670    126670.0
```

¹<https://numpy.org/doc/stable/reference/generated/numpy.argsort.html>

²<https://numpy.org/doc/stable/reference/generated/numpy.sort.html>

## 270	Mercedes-Benz	SL55 AMG 2dr	\$121,770	121770.0
## 261	Mercedes-Benz	CL500 2dr	\$94,820	94820.0

Pandas' `DataFrames` and `Series` have a `.replace()`³ method. We use this to remove dollar signs and commas from the MSRP column. Note that we had to access the `.str` attribute of the `Series` column before we used it. After the string was processed, we converted it to a `Series` of floats with the `astype()` method.

Finally, sorting the overall data frame could have been done with the same approach as the code we used in R (i.e. raw subsetting by row indexes), but there is a built-in method called `sort_values()`⁴ that will do it for us.

12.2 Stacking Data Sets and Placing Them Shoulder to Shoulder

Stacking data sets on top of each other is a common task. You might need to do it if

1. you need to add a new row (or many rows) to a data frame,
2. you need to recombine data sets (e.g. recombine a train/test split), or
3. you're creating a matrix in a step-by-step way.

In R, this can be done with `rbind()` (short for “row bind”). Consider the following example that makes use of GIS data queried from (Albemarle County Geographic Data Services Office, 2021) and cleaned with code from (Ford, 2016).

```
realEstate <- read.csv("data/albemarle_real_estate.csv")
train <- realEstate[-1,]
test <- realEstate[1,]
str(rbind(test, train), strict.width = "cut")
## 'data.frame': 30381 obs. of 12 variables:
## $ YearBuilt : int 1769 1818 2004 2006 2004 1995 1900 1960 ..
## $ YearRemodeled: int 1988 1991 NA NA NA NA NA NA NA NA ...
## $ Condition : chr "Average" "Average" "Average" "Average" ..
```

³<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.replace.html>

⁴https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_values.html

```
## $ NumStories : num 1.7 2 1 1 1.5 2.3 2 1 1 1 ...
## $ FinSqFt : int 5216 5160 1512 2019 1950 2579 1530 800 9..
## $ Bedroom : int 4 6 3 3 3 3 4 2 2 2 ...
## $ FullBath : int 3 4 2 3 3 2 1 1 1 1 ...
## $ HalfBath : int 0 1 1 0 0 1 0 0 0 0 ...
## $ TotalRooms : int 8 11 9 10 8 8 6 4 4 4 ...
## $ LotSize : num 5.1 453.9 42.6 5 5.5 ...
## $ TotalValue : num 1096600 2978600 677800 453200 389200 ...
## $ City : chr "CROZET" "CROZET" "CROZET" "CROZET" ...
sum(rbind(test, train) != realEstate)
## [1] NA
```

The above example was with `data.frames`. This example of `rbind()` is with `matrix` objects.

```
rbind(matrix(1,nrow = 2, ncol = 3),
      matrix(2,nrow = 2, ncol = 3))
##      [,1] [,2] [,3]
## [1,]    1    1    1
## [2,]    1    1    1
## [3,]    2    2    2
## [4,]    2    2    2
```

In Python, you can stack data frames with `pd.concat()`⁵. It has a lot of options, so feel free to peruse them. You can also replace the call to `pd.concat()` below with `test.append(train)`⁶. Consider the example below that uses the Albemarle County real estate data (Albemarle County Geographic Data Services Office, 2021) (Ford, 2016).

```
import pandas as pd
real_estate = pd.read_csv("data/albemarle_real_estate.csv")
train = real_estate.iloc[1:,]
test = real_estate.iloc[[0],] # need the extra brackets!
stacked = pd.concat([test,train], axis=0)
stacked.iloc[:3,:3]
##      YearBuilt  YearRemodeled Condition
```

⁵<https://www.google.com/search?client=safari&rls=en&q=pandas+concat&ie=UTF-8&oe=UTF-8>

⁶<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.append.html>

```
## 0      1769      1988.0  Average
## 1      1818      1991.0  Average
## 2      2004        NaN  Average
(stacked != real_estate).sum().sum()
## 28251
```

Take note of the extra square brackets when we create `test`. If you use `real_estate.iloc[0,]` instead, it will return a `Series` with all the elements coerced to the same type, and this won't `pd.concat()` properly with the rest of the data!

12.3 Merging or Joining Data Sets

If you have two different data sets that provide information about the same experimental units, you can put the two data sets together using a **merge** (aka **join**) operation. In R, you can use the `merge()` function⁷. In Python, you can use the `.merge()` method⁸.

Merging (or joining) data sets is not the same as placing them shoulder to shoulder. Placing data sets shoulder to shoulder will not reorder the rows of your data and the operation requires that both input data sets have the same number of rows to start off with. On the other hand, merging data takes care to match rows together in an intelligent way, and it can handle situations of missing or duplicate matches. In both cases, the resulting data set is wider, but with merging, the output might end contain either more or fewer rows.

Here's a clarifying example. Suppose you have to sets of supposedly anonymized data about individual accounts on some online platforms.

```
# in R
baby1 <- read.csv("data/baby1.csv", stringsAsFactors = FALSE)
baby2 <- read.csv("data/baby2.csv", stringsAsFactors = FALSE)
head(baby1)
##   idnum height.inches.      email_address
## 1     1             74 fakeemail123@gmail.com
## 2     3             66 anotherfake@gmail.com
```

⁷<https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/merge>

⁸<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.merge.html#pandas-dataframe-merge>

```
## 3      4      62      notreal@gmail.com
## 4     23      62      notreal@gmail.com
head(baby2)
##      idnum      phone      email
## 1 3901283 5051234567      notreal@gmail.com
## 2   41823 5051234568 notrealeither@gmail.com
## 3 7198273 5051234568  anotherfake@gmail.com
```

The first thing you need to ask yourself is “*which column is the unique identifier that is shared between these two data sets?*” In our case, they both have an “identification number” column. However, these two data sets are coming from different online platforms, and these two places use different schemes to number their users.

In this case, it is better to merge on the email addresses. Users might be using different email addresses on these two platforms, but there’s a stronger guarantee that matched email addresses means that you’re matching the right accounts. The columns are named differently in each data set, so we must specify them by name.

```
# in R
merge(baby1, baby2, by.x = "email_address", by.y = "email")
##      email_address idnum.x height.inches. idnum.y      phone
## 1 anotherfake@gmail.com      3      66 7198273 5051234568
## 2      notreal@gmail.com      4      62 3901283 5051234567
## 3      notreal@gmail.com     23      62 3901283 5051234567
```

In Python, `merge()` is a method attached to each `DataFrame` instance.

```
# in Python
baby1.merge(baby2, left_on = "email_address", right_on = "email")
##      idnum_x height(inches)      email_address idnum_y      phone      email
## 0          3          66  anotherfake@gmail.com 7198273 5051234568  anotherfake@gmail.com
## 1          4          62      notreal@gmail.com 3901283 5051234567      notreal@gmail.com
## 2         23          62      notreal@gmail.com 3901283 5051234567      notreal@gmail.com
```

The email addresses anotherfake@gmail.com and notreal@gmail.com exist in both data sets, so each of these email addresses will end up in the result data frame. The rows in the result data set are wider and have more attributes for each individual.

Notice the duplicate email address, too. In this case, either the user signed up for two accounts using the same email, or one person signed up for an account

with another person's email address. In the case of duplicates, both rows will match with the same rows in the other data frame.

Also, in this case, all email addresses that weren't found in both data sets were thrown away. This does not necessarily need to be the intended behavior. For instance, if we wanted to make sure no rows were thrown away, that would be possible. In this case, though, for email addresses that weren't found in both data sets, some information will be missing. Recall that Python and R handle missing data differently (Section 3.8.2).

```
# in R
merge(baby1, baby2,
      by.x = "email_address", by.y = "email",
      all.x = TRUE, all.y = TRUE)

##           email_address idnum.x height.inches. idnum.y      phone
## 1  anotherfake@gmail.com      3           66 7198273 5051234568
## 2  fakeemail123@gmail.com      1           74          NA          NA
## 3    notreal@gmail.com      4           62 3901283 5051234567
## 4    notreal@gmail.com     23           62 3901283 5051234567
## 5 notrealeither@gmail.com     NA           NA   41823 5051234568
```

```
# in Python
le_merge = baby1.merge(baby2,
                      left_on = "email_address", right_on = "email",
                      how = "outer")

le_merge.iloc[:5,3:]

##           idnum_y      phone      email
## 0             NaN          NaN          NaN
## 1  7198273.0  5.051235e+09  anotherfake@gmail.com
## 2  3901283.0  5.051235e+09    notreal@gmail.com
## 3  3901283.0  5.051235e+09    notreal@gmail.com
## 4   41823.0  5.051235e+09 notrealeither@gmail.com
```

You can see it's slightly more concise in Python. If you are familiar with SQL, you might have heard of inner and outer joins. This is where Pandas takes some of its argument names from⁹.

⁹<https://pandas.pydata.org/pandas-docs/version/0.15/merging.html#database-style-dataframe-joining-merging>

Finally, if both data sets have multiple values in the column you're joining on, the result can have more rows than either table. This is because *every possible match* shows up.

```
# in R
first <- data.frame(category = c('a','a'), measurement = c(1,2))
merge(first, first, by.x = "category", by.y = "category")
##   category measurement.x measurement.y
## 1      a              1              1
## 2      a              1              2
## 3      a              2              1
## 4      a              2              2
```

```
# in Python
first = pd.DataFrame({'category' : ['a','a'], 'measurement' : [1,2]})
first.merge(first, left_on = "category", right_on = "category")
##   category measurement_x measurement_y
## 0      a              1              1
## 1      a              1              2
## 2      a              2              1
## 3      a              2              2
```

12.4 Long versus Wide Data

12.4.1 Long versus Wide in R

Many types of data can be stored in either a **wide** or **long** format.

The classical example is data from a *longitudinal study*. If an experimental unit (in the example below this is a person) is repeatedly measured over time, each row would correspond to an experimental unit *and* an observation time in a data set in a long form.

```
peopleNames <- c("Taylor","Taylor","Charlie","Charlie")
fakeLongData1 <- data.frame(person = peopleNames,
                             timeObserved = c(1, 2, 1, 2),
                             nums = c(100,101,300,301))

fakeLongData1
##   person timeObserved nums
```

```
## 1 Taylor      1 100
## 2 Taylor      2 101
## 3 Charlie     1 300
## 4 Charlie     2 301
```

A long format can also be used if you have multiple observations (at a single time point) on an experimental unit. Here is another example.

```
myAttrs <- c("attrA","attrB","attrA","attrB")
fakeLongData2 <- data.frame(person = peopleNames,
                             attributeName = myAttrs,
                             nums = c(100,101,300,301))

fakeLongData2
##   person attributeName  nums
## 1 Taylor      attrA    100
## 2 Taylor      attrB    101
## 3 Charlie     attrA    300
## 4 Charlie     attrB    301
```

If you would like to reshape the long data sets into a wide format, you can use the `reshape()` function. You will need to specify which columns correspond with the experimental unit, and which column is the “factor” variable.

```
fakeWideData1 <- reshape(fakeLongData1,
                          direction = "wide",
                          timevar = "timeObserved",
                          idvar = "person",
                          varying = c("before","after"))
# ^ varying= arg becomes col names in new data set

fakeLongData1
##   person timeObserved  nums
## 1 Taylor      1    100
## 2 Taylor      2    101
## 3 Charlie     1    300
## 4 Charlie     2    301

fakeWideData1
##   person before after
## 1 Taylor    100   101
## 3 Charlie   300   301
```

```
# timevar= is a misnomer here
fakeWideData2 <- reshape(fakeLongData2,
                          direction = "wide",
                          timevar = "attributeName",
                          idvar = "person",
                          varying = c("attribute A","attribute B"))

fakeLongData2
##      person attributeName  nums
## 1 Taylor          attrA   100
## 2 Taylor          attrB   101
## 3 Charlie         attrA   300
## 4 Charlie         attrB   301

fakeWideData2
##      person attribute A attribute B
## 1 Taylor          100          101
## 3 Charlie          300          301
```

`reshape()` will also go in the other direction: it can take wide data and convert it into long data

```
reshape(fakeWideData1,
        direction = "long",
        idvar = "person",
        varying = list(c("before","after")),
        v.names = "nums")

##      person time  nums
## Taylor.1 Taylor    1  100
## Charlie.1 Charlie  1  300
## Taylor.2 Taylor    2  101
## Charlie.2 Charlie  2  301

fakeLongData1
##      person timeObserved  nums
## 1 Taylor          1  100
## 2 Taylor          2  101
## 3 Charlie         1  300
## 4 Charlie         2  301

reshape(fakeWideData2,
        direction = "long",
        idvar = "person",
        varying = list(c("attribute A","attribute B")),
```

```

        v.names = "nums")
##           person time nums
## Taylor.1   Taylor    1  100
## Charlie.1  Charlie    1  300
## Taylor.2   Taylor    2  101
## Charlie.2  Charlie    2  301
fakeLongData2
##   person attributeName  nums
## 1  Taylor           attrA  100
## 2  Taylor           attrB  101
## 3  Charlie          attrA  300
## 4  Charlie          attrB  301

```

12.4.2 Long versus Wide in Python

With Pandas, we can take make long data wide with `pd.DataFrame.pivot()`¹⁰, and we can go in the other direction with `pd.DataFrame.melt()`¹¹.

When going from long to wide, make sure to use the `pd.DataFrame.reset_index()`¹² method afterwards to reshape the data and remove the index. Here is an example similar to the one above.

```

import pandas as pd
fake_long_data1 = pd.DataFrame(
    {'person' : ["Taylor","Taylor","Charlie","Charlie"],
     'time_observed' : [1, 2, 1, 2],
     'nums' : [100,101,300,301]})
fake_long_data1
##   person  time_observed  nums
## 0  Taylor             1   100
## 1  Taylor             2   101
## 2  Charlie            1   300
## 3  Charlie            2   301
pivot_data1 = fake_long_data1.pivot(index='person',
                                     columns='time_observed',

```

¹⁰<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pivot.html#>

¹¹<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.melt.html?highlight=melt>

¹²https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.reset_index.html#

```

                                values='nums')
fake_wide_data1 = pivot_data1.reset_index()
fake_wide_data1
## time_observed  person    1    2
## 0              Charlie  300  301
## 1              Taylor  100  101

```

Here's one more example showing the same functionality—going from long to wide format.

```

people_names = ["Taylor","Taylor","Charlie","Charlie"]
attribute_list = ['attrA', 'attrB', 'attrA', 'attrB']
fake_long_data2 = pd.DataFrame({'person' : people_names,
                                'attribute_name' : attribute_list,
                                'nums' : [100,101,300,301]})
fake_wide_data2 = fake_long_data2.pivot(index='person',
                                         columns='attribute_name',
                                         values='nums').reset_index()

fake_wide_data2
## attribute_name  person  attrA  attrB
## 0              Charlie    300    301
## 1              Taylor    100    101

```

Here are some examples of going in the other direction: from wide to long with `pd.DataFrame.melt()`¹³. The first example specifies value columns by integers.

```

fake_wide_data1
## time_observed  person    1    2
## 0              Charlie  300  301
## 1              Taylor  100  101
fake_wide_data1.melt(id_vars = "person", value_vars = [1,2])
##      person time_observed  value
## 0  Charlie              1    300
## 1  Taylor              1    100
## 2  Charlie              2    301
## 3  Taylor              2    101

```

¹³<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.melt.html?highlight=melt>

The second example uses strings to specify value columns.

```
fake_wide_data2
## attribute_name  person  attrA  attrB
## 0             Charlie   300    301
## 1             Taylor    100    101
fake_wide_data2.melt(id_vars = "person",
                     value_vars = ['attrA', 'attrB'])
##      person attribute_name  value
## 0  Charlie             attrA    300
## 1  Taylor             attrA    100
## 2  Charlie             attrB    301
## 3  Taylor             attrB    101
```

12.5 Exercises

12.5.1 R Questions

1.

Recall the `car.data` data set (mis, 1997), which is hosted by (Dua and Graff, 2017).

- a) Read in the data set as `carData`.
- b) Convert the third and fourth columns to *ordered factors*.
- c) Order the data by the third and then the fourth column (simultaneously). Do not change the data in place. Instead store it under the name `ordCarData1`
- d) Order the data by the fourth and then the third column (simultaneously). Do not change the data in place. Instead store it under the name `ordCarData2`

2.

```
day1Data <- data.frame(idNum = 1:10,
                       measure = rnorm(10))
day2Data <- data.frame(idNum = 11:20,
                       measure = rnorm(10))
```

- a) Pretend `day1Data` and `day2Data` are two separate data sets that possess the same type of measures but on different experimental units. Stack `day1Data` on top of `day2Data` and call the result `stackedData`.
 - b) Pretend `day1Data` and `day2Data` are different measurements on the same experimental units. Place them shoulder to shoulder and call the result `sideBySide`. Put `day1Data` first, and `day2Data` second.
- 3.

If you are dealing with random matrices, you might need to **vectorize** a matrix object. This is not the same as “vectorization” in programming. Instead, it means you write the matrix as a big column vector by stacking the columns on top of each other. Specifically, if you have a $n \times p$ real-valued matrix $\mathbf{X}$, then

$$\text{vec}(\mathbf{X}) = \begin{bmatrix} \mathbf{X}_1 \\ \vdots \\ \mathbf{X}_p \end{bmatrix} \quad (12.1)$$

where $\mathbf{X}_i$ is the i th column as an $n \times 1$ column vector. There is another operator that we will use, the **Kronecker product**:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix}. \quad (12.2)$$

If $\mathbf{A}$ is $m \times n$ and $\mathbf{B}$ is $p \times q$, then $\mathbf{A} \otimes \mathbf{B}$ is $pm \times qn$.

- a) Write a function called `vec(myMatrix)`. Its input should be one `matrix` object. Its output should be a `vector`. Hint: `matrix` objects are stored in column-major order.
- b) Write a function called `unVec(myVector, nRows)` that takes in the vectorized matrix as a `vector`, splits that into elements with `nRows` elements, and then places them together shoulder-to-shoulder as a `matrix`.
- c) Write a function called `stackUp(m, BMat)` that returns $\mathbf{1}_m \otimes \mathbf{B}$ where $\mathbf{1}_m$ is a length m column vector of ones. You may check your work with `%x%`, but do not use this in your function.
- d) Write a function called `shoulderToShoulder(n, BMat)` that returns $\mathbf{1}_n^\top \otimes \mathbf{B}$ where $\mathbf{1}_n^\top$ is a length n row vector of ones. You may check your work with `%x%`, but do not use this in your function.

4.

This problem uses the Militarized Interstate Disputes (v5.0) (Palmer et al., 0) data set from The Correlates of War Project¹⁴. There are four `.csv` files we use for this problem. `MIDA 5.0.csv` contains the essential attributes of each militarized interstate dispute from 1/1/1816 through 12/31/2014. `MIDB 5.0.csv` describes the participants in each of those disputes. `MIDI 5.0.csv` contains the essential elements of each militarized interstate incident, and `MIDIP 5.0.csv` describes the participants in each of those incidents.

- a) Read in the four data sets and give them the names `mida`, `midb`, `midi`, and `midp`. Take care to convert all instances of `-9` to `NA`.
- b) Examine all rows of `midb` where its `dispnun` column equals 2. Do not change `midb` permanently. Are these two rows corresponding to the same conflict? If so, assign `TRUE` to `sameConflict`. Otherwise, assign `FALSE`.
- c) Join the first two data sets together on the dispute number column (`dispnun`). Call the resulting `data.frame` `join1`. Do not address any concerns about duplicate columns.
- d) Is there any difference between doing an inner join and an outer join in the previous question? If there was a difference, assign `TRUE` to `theyAreNotTheSame`. Otherwise, assign `FALSE` to it.
- e) Join the last two data sets together by `incidnum` and call the result `join2`. Is there any difference between an inner and an outer join for this problem? Why or why not? Do not address any concerns about duplicate columns.
- f) The codebook mentions that the last two data sets don't go as far back in time as the first two. Suppose then that we only care about the events in `join2`. Merge `join2` and `join1` in a way where all undesired rows from `join1` are discarded, and all rows from `join2` are kept. Call the resulting `data.frame` `midData`. Do not address any concerns about duplicate columns.
- g) Use a scatterplot to display the relationship between the maximum duration and the end year. Plot each country as a different color.
- h) Create a `data.frame` called `longData` that has the following three columns from `midp`: `incidnum` (incident identification number) `stabb` (state abbreviation of participant) and `fatalpre` (precise number of fatalities). Convert this to "wide" format. Make the new table called `wideData`. Use the incident number row as a unique row-identifying variable.

¹⁴<https://correlatesofwar.org/>

- i) Bonus Question: identify all column pairs that contain duplicate information in `midData`, remove all but one of the columns, and change the column name back to its original name.

12.5.2 Python Questions

1.

Once again, recall the "car.data" data set (mis, 1997).

- a) Read in the data set as `car_data`.
- b) Order the data by the third and then the fourth column. Do not change the data in place. Instead store it under the name `ord_car_data1`
- c) Order the data by the fourth and then the third column. Do not change the data in place. Instead store it under the name `ord_car_data2`

2.

Consider the following random data set.

```
indexes = np.random.choice(np.arange(20),size=20,replace=False)
d1 = pd.DataFrame({'a' : indexes,
                  'b' : np.random.normal(size=20)})
d2 = pd.DataFrame({'a' : indexes + 20,
                  'b' : np.random.normal(size=20)})
```

- a) Pretend `d1` and `d2` are two separate data sets that possess the same type of measures but on different experimental units. Stack `d1` on top of `d2` and call the result `stacked_data_sets`. Make sure the index of the result is the numbers 0 through 39
- b) Pretend `d1` and `d2` are different measurements on the same experimental units. Place them shoulder to shoulder and call the result `side_by_side_data_sets`. Put `d1` first, and `d2` second.

3.

Consider the following two data sets:

```
import numpy as np
import pandas as pd
dog_names1 = ['Charlie','Gus', 'Stubby', 'Toni','Pearl']
```

```

dog_names2 = ['Charlie', 'Gus', 'Toni', 'Arya', 'Shelby']
nicknames = ['Charles', 'Gus The Bus', np.nan, 'Toni Bologna', 'Porl']
breed_names = ['Black Lab', 'Beagle', 'Golden Retriever', 'Husky', np.nan]
dataset1 = pd.DataFrame({'dog': dog_names1,
                        'nickname': nicknames})
dataset2 = pd.DataFrame({'dog': dog_names2,
                        'breed': breed_names})

```

- a) Join/merge the two data sets together in such a way that there is a row for every dog, whether or not both tables have information for that dog. Call the result `merged1`.
- b) Join/merge the two data sets together in such a way that there are only rows for every dog in `dataset1`, whether or not there is information about these dogs' breeds. Call the result `merged2`.
- c) Join/merge the two data sets together in such a way that there are only rows for every dog in `dataset2`, whether or not there is information about the dogs' nicknames. Call the result `merged3`.
- d) Join/merge the two data sets together in such a way that all rows possess complete information. Call the result `merged4`.

4.

Let's consider Fisher's "Iris" data set (Fisher, 1988) again.

- a) Read in `iris.csv` and store the `DataFrame` with the name `iris`. Let it have the column names 'a', 'b', 'c', 'd' and 'e'.
- b) Create a `DataFrame` called `name_key` that stores correspondences between long names and short names. It should have three rows and two columns. The long names are the unique values of column five of `iris`. The short names are either 's', 'vers' or 'virg'. Use the column names 'long name' and 'short name'.
- c) Merge/join the two data sets together to give `iris` a new column with information about short names. Do not overwrite `iris`. Rather, give the `DataFrame` a new name: `iris_with_short_names`. Remove any columns with duplicate information.
- d) Change the first four column names of `iris_with_short_names` to `s_len`, `s_wid`, `p_len`, and `p_wid`. Use Matplotlib to create a figure with 4 subplots arranged into a 2×2 grid. On each subplot, plot a

histogram of these four columns. Make sure to use x-axis labels so viewers can tell which column is being plotted in each subplot.

- e) Let's go back to `iris`. Change that to long format. Store it as a `DataFrame` called `long_iris`. Make the column names `row`, `variable` and `value`, in that order. Last, make sure it is sorted (simultaneously/once) by `row` and then `variable`.

13

Visualization

I describe a few plotting paradigms in R and Python below. Note that these descriptions are brief. More details could easily turn any of these subsections into an entire textbook.

13.1 Base R Plotting

R comes with some built-in plotting functions such as `plot()`, `hist()` and `boxplot()`. Many of these reside in `package:graphics`, which comes pre-loaded into the search path. `plot()` on the other hand, is higher up the search path in `package:base`—it is a generic method whose methods can reside in many places (e.g. in `package:graphics` or some place else).

Base plotting will usually cover most of your needs, so that is what we spend the most time learning about. However, there are a large number of third-party libraries for plotting that you might consider looking into if you want to use a certain aesthetic, or if you want plotting functionality that is specialized for certain cases (e.g. geospatial plots).

Recall our Albemarle Real Estate data set (Albemarle County Geographic Data Services Office, 2021) (Ford, 2016).

```
df <- read.csv("data/albemarle_real_estate.csv")
str(df, strict.width = "cut")
## 'data.frame':    30381 obs. of  12 variables:
## $ YearBuilt      : int  1769 1818 2004 2006 2004 1995 1900 1960 ..
## $ YearRemodeled: int  1988 1991 NA NA NA NA NA NA NA ...
## $ Condition      : chr  "Average" "Average" "Average" "Average" ..
## $ NumStories     : num  1.7 2 1 1 1.5 2.3 2 1 1 1 ...
## $ FinSqFt        : int  5216 5160 1512 2019 1950 2579 1530 800 9..
## $ Bedroom        : int  4 6 3 3 3 3 4 2 2 2 ...
## $ FullBath       : int  3 4 2 3 3 2 1 1 1 1 ...
## $ HalfBath       : int  0 1 1 0 0 1 0 0 0 0 ...
```

```
## $ TotalRooms : int 8 11 9 10 8 8 6 4 4 4 ...
## $ LotSize    : num 5.1 453.9 42.6 5 5.5 ...
## $ TotalValue : num 1096600 2978600 677800 453200 389200 ...
## $ City       : chr "CROZET" "CROZET" "CROZET" "CROZET" ...
```

If we wanted to get a general idea of how expensive homes were in Albemarle County, we could use a histogram. This helps us visualize a univariate numerical variable/column. The (natural) logarithm of home prices is plotted as a histogram in [Figure 13.1](#).

```
hist(log(df$TotalValue),
     xlab = "natural logarithm of home price",
     main = "Super-Duper Plot!")
```

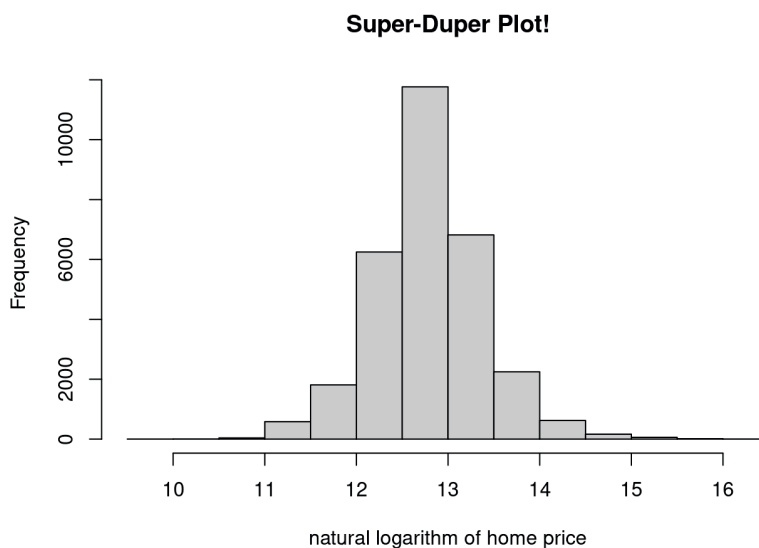


FIGURE 13.1: A simple histogram.

I specified the `xlab=` and `main=` arguments, but there are many more that could be tweaked. Make sure to skim the options in the documentation (`?hist`).

`plot()` is useful for plotting *two* univariate numerical variables. This can be done in time series plots (variable versus time) and scatter plots (one variable versus another). For an example of two scatter plots, see [Figure 13.2](#).

```

par(mfrow=c(1,2))
plot(df$TotalValue, df$LotSize,
     xlab = "total value ($)", ylab = "lot size (sq. ft.)",
     pch = 3, col = "red", type = "b")
plot(log(df$TotalValue), log(df$LotSize),
     xlab = "log. total value", ylab = "log. lot size",
     pch = 2, col = "blue", type = "p")
abline(h = log(mean(df$LotSize)), col = "green")

```

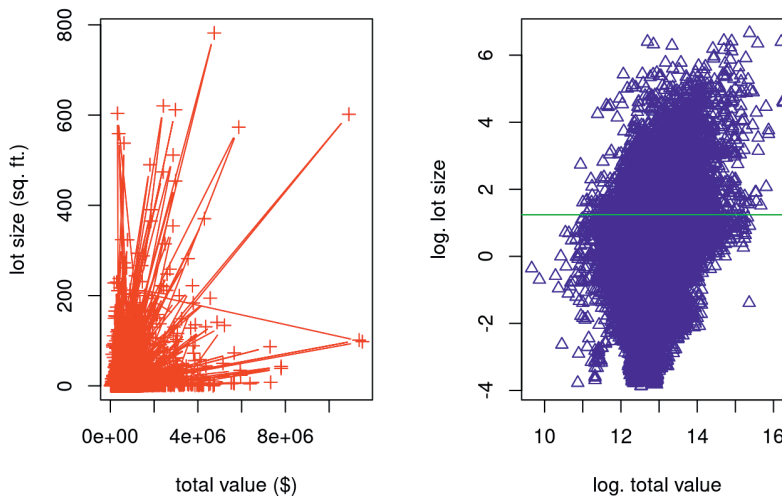


FIGURE 13.2: Some scatterplots.

```

par(mfrow=c(1,1))

```

I use some of the many arguments available (type `?plot`). `xlab=` and `ylab=` specify the x- and y-axis labels, respectively. `col=` is short for “color.” `pch=` is short for “point character.” Changing this will change the symbol shapes used for each point. `type=` is more general than that, but it is related. I typically use it to specify whether or not I want the points connected with lines.

I use a couple other functions in the above code. `abline()` is used to superimpose lines over the top of a plot. They can be horizontal, vertical, or you can specify them in slope-intercept form, or by providing a linear model object. I also used `par()` to set a graphical parameter. The graphical parameter `par()$mfrow` sets the layout of a multiple plot visualization. I then set it back to the standard 1×1 layout afterwards.

13.2 Plotting with `ggplot2`

`ggplot2`¹ is a popular third-party visualization package for R. There are also libraries in Python (e.g. `plotnine`²) that have a similar look and feel. This subsection provides a short tutorial on how to use `ggplot2` in R, and it is primarily based off of the material provided in (Wickham, 2016). Other excellent descriptions of `ggplot2` are (Kabacoff, 2015) and (Chang, 2013).

`ggplot2` code looks a lot different than the code in the above section³. There, we would write a series of function calls, and each would change some state in the current figure. Here, we call different `ggplot2` functions that create S3 objects with special behavior (more information about S3 objects in [subsection 14.2.2](#)), and then we “add” (i.e. we use the `+` operator) them together.

This new design is not to encourage you to think about S3 object-oriented systems. Rather, it is to get you thinking about making visualizations using the “grammar of graphics” (Wilkinson, 2005). `ggplot2` makes use of its own specialized vocabulary that is taken from this book. As we get started, I will try to introduce some of this vocabulary slowly.

The core function in this library is the `ggplot()` function⁴. This function initializes figures; it is the function that will take in information about *which* data set you want to plot, and *how* you want to plot it. The raw data is provided in the first argument. The second argument, `mapping=`, is more confusing. The argument should be constructed with the `aes()` function. In the parlance of `ggplot2`, `aes()` constructs an **aesthetic mapping**. Think of the “aesthetic mapping” as stored information that can be used later on—it “maps” data to visual properties of a figure.

Consider this first example by typing the following into your own console.

```
library(ggplot2)
ggplot(mpg, aes(x = displ, y = hwy))
```

¹<https://ggplot2.tidyverse.org/index.html>

²<https://plotnine.readthedocs.io/en/stable/#>

³Personally, I find its syntax more confusing, and so I tend to prefer base graphics. However, it is very popular, and so I do believe that it is important to mention it here in this text.

⁴<https://www.rdocumentation.org/packages/ggplot2/versions/3.3.5/topics/ggplot>

You'll notice a few things about the code and the result produced:

1. No geometric shapes show up!
2. A Cartesian coordinate system is displayed, and the x-axis and y-axis were created based on aesthetic mapping provided (confirm this by typing `summary(mpg$displ)` and `summary(mpg$hwy)`).
3. The axis labels are taken from the column names provided to `aes()`.

To display geometric shapes (aka *geoms* in the parlance of *ggplot2*), we need to add **layers**⁵ to the figure. “Layers” is quite a broad term—it does not only apply to geometric objects. In fact, in *ggplot2*, a layer can be pretty much anything: raw data, summarized data, transformed data, annotations, etc. However, the functions that add geometric object layers usually start with the prefix `geom_`. In RStudio, after loading *ggplot2*, type `geom_`, and then press `<Tab>` (autocomplete) to see some of the options.

Consider the function `geom_point()`⁶. It too returns an S3 instance that has specialized behavior. In the parlance of *ggplot2*, it adds a scatterplot⁷ layer to the figure. The result of the following code is displayed in [Figure 13.3](#).

```
library(ggplot2)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()
```

Notice that we did not need to provide any arguments to `geom_point()`—the aesthetic mappings were used by the new layer.

There are *many* types of layers that you can add, and you are not limited to any number of them in a given plot. For example, if we wanted to add a title, we could use the `ggtitle()` function to add a title layer. Unlike `geom_point()`, this function will need to take an argument because the desired title is not stored as an aesthetic mapping. Try running the following code on your own machine.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  ggtitle("my favorite scatterplot")
```

⁵<https://ggplot2-book.org/toolbox.html#toolbox>

⁶https://www.rdocumentation.org/packages/ggplot2/versions/3.3.5/topics/geom_point

⁷<https://ggplot2-book.org/getting-started.html#basic-use>

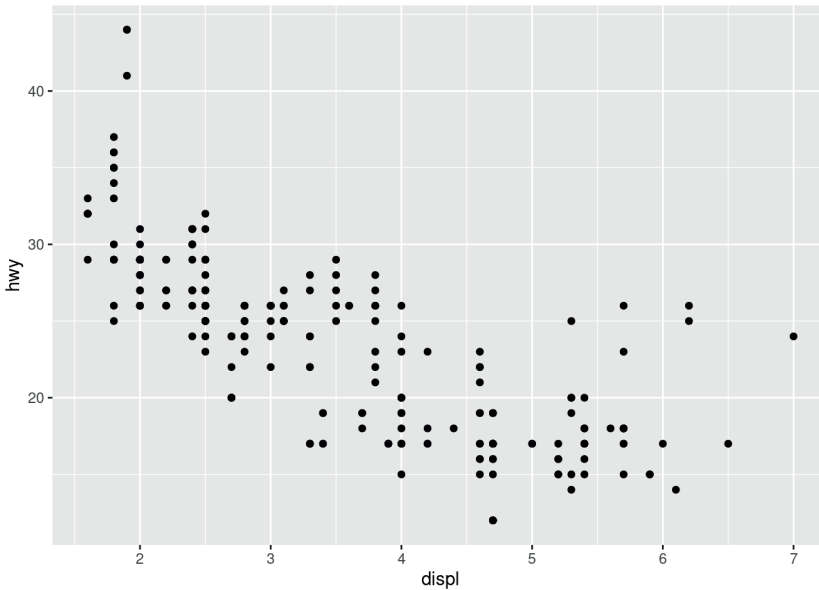


FIGURE 13.3: A second scatterplot.

Additionally, notice that the same layer will behave much differently if we change the aesthetic mapping. The result after adding a ‘color=’ aesthetic is displayed in [Figure 13.4](#).

```
ggplot(mpg, aes(x = displ, y = hwy, color = manufacturer)) +  
  geom_point() +  
  ggtitle("my favorite scatterplot")
```

If we want tighter control on the aesthetic mapping, we can use **scales**⁸. Syntactically, these are things we “add” (+) to the figure, just like layers. However, these scales are constructed with a different set of functions, many of which start with the prefix `scale_`. We can change attributes of the axes like this (see [Figure 13.5](#)).

```
base_plot <- ggplot(mpg,  
                    aes(x = displ, y = hwy, color = manufacturer)) +  
  geom_point() +  
  ggtitle("my favorite scatterplot")  
base_plot + scale_x_log10() + scale_y_reverse()
```

⁸<https://ggplot2-book.org/scales.html#scales>

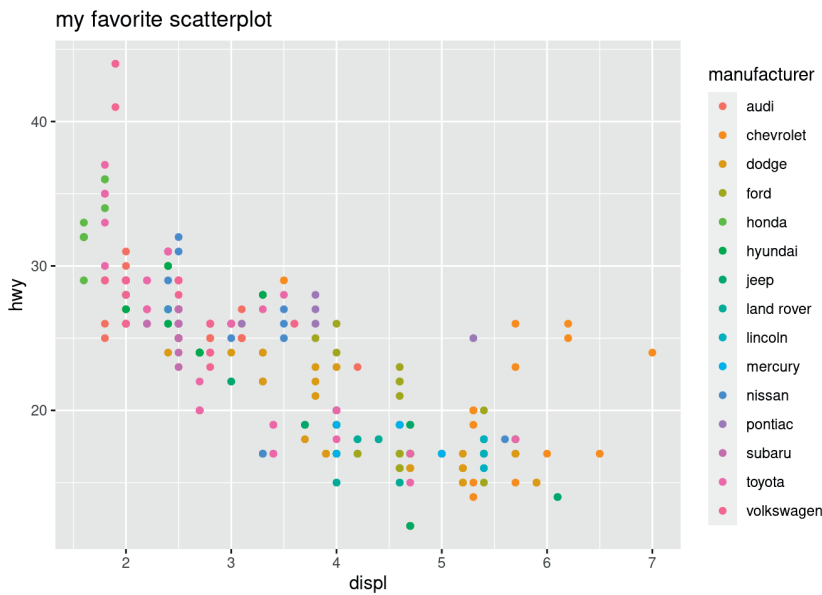


FIGURE 13.4: Adding some color.

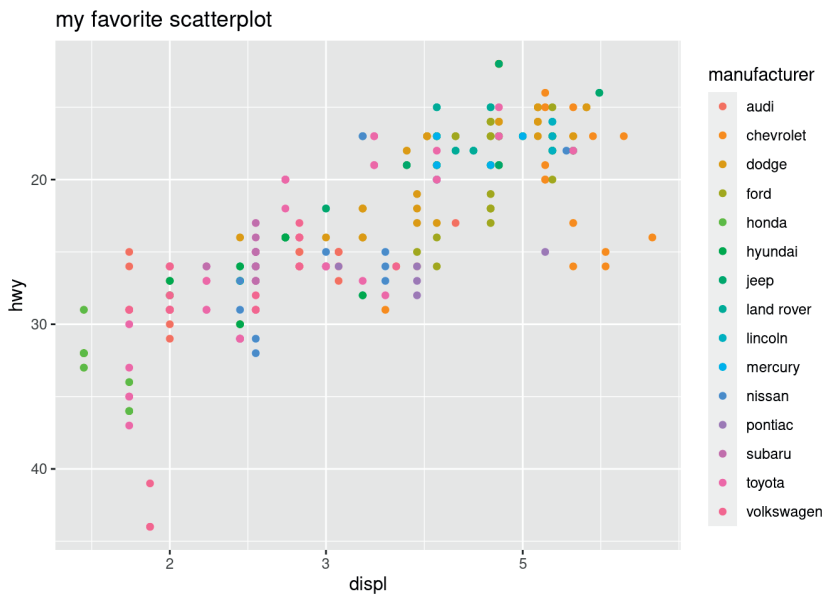


FIGURE 13.5: Changing scales.

We can also change plot colors with scale layers. Let's add an aesthetic called `fill=` so we can use colors to denote the value of a numerical (not categorical) column. This data set doesn't have any more unused numerical columns, so let's create a new one called `score`. We also use a new geom layer from a function called `geom_tile()` (see [Figure 13.6](#)).

```
mpg$score <- 1/(mpg$displ^2 + mpg$hwy^2)
ggplot(mpg, aes(x = displ, y = hwy, fill = score )) +
  geom_tile()
```

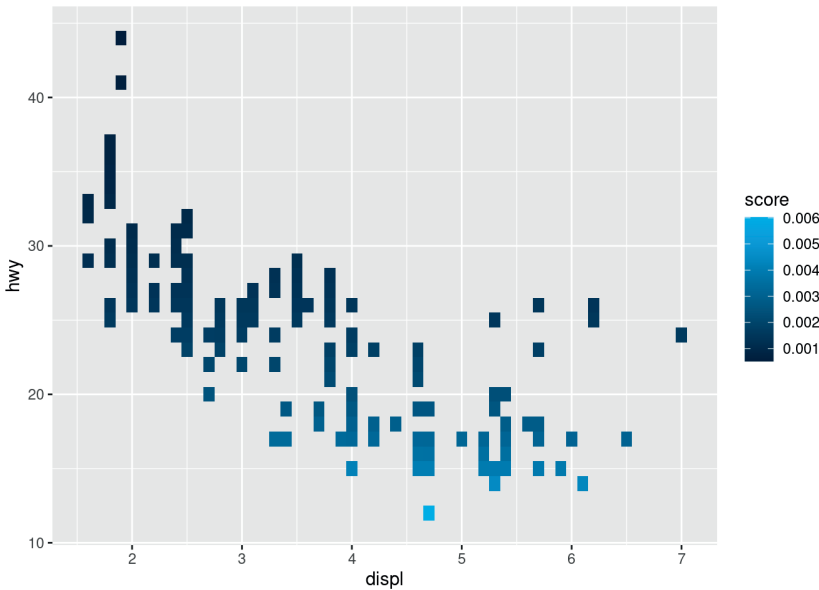


FIGURE 13.6: Changing the fill.

If we didn't like these colors, we could change them with a scale layer. Personally, I like this one (see [Figure 13.7](#)).

```
mpg$score <- 1/(mpg$displ^2 + mpg$hwy^2)
ggplot(mpg, aes(x = displ, y = hwy, fill = score )) +
  geom_tile() +
  scale_fill_viridis_b()
```

There are many to choose from, though. Try to run the following code on your own to see what it produces.

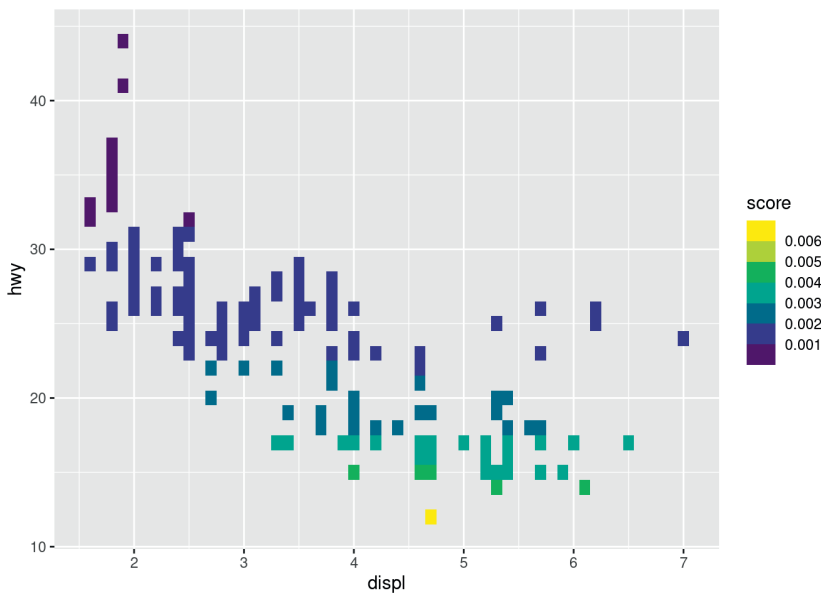


FIGURE 13.7: Changing the fill again.

```
mpg$score <- 1/(mpg$displ^2 + mpg$hwy^2)
ggplot(mpg, aes(x = displ, y = hwy, fill = score)) +
  geom_tile() +
  scale_fill_gradient2()
```

13.3 Plotting with Matplotlib

Matplotlib (Hunter, 2007) is a third-party visualization library in Python. It is the oldest and most heavily-used, so it is the best way to start making graphics in Python, in my humble opinion. It also comes installed with Anaconda.

This short introduction borrows heavily from the myriad of tutorials⁹ on Matplotlib’s website. I will start off making a simple plot, and commenting on each line of code. If you’re interested in learning more, (VanderPlas, 2016) and (McKinney, 2017) are also terrific resources.

⁹<https://matplotlib.org/stable/tutorials/index.html>



You can use either “pyplot-style” (e.g. `plt.plot()`) or “object-oriented-style” to make figures in Matplotlib. Even though using the first type is faster to make simple plots, I will only describe the second one. It is the recommended approach because it is more extensible. However, the first one resembles the syntax of MATLAB. If you’re familiar with MATLAB, you might consider learning a little about the first style, as well.

```
import matplotlib.pyplot as plt      # 1
import numpy as np                  # 2
fig, ax = plt.subplots()            # 3
_ = ax.hist(np.random.normal(size=1000)) # 4
plt.show()                          # 5
```

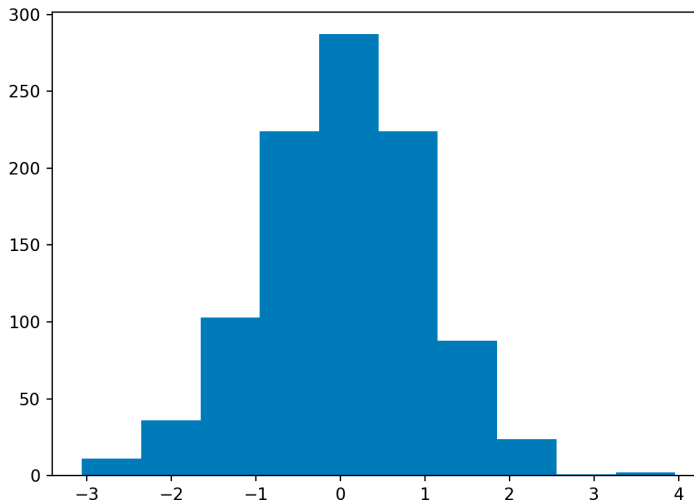


FIGURE 13.8: Another simple histogram.

Figure 13.8 displays a histogram generated from the above code. In the first line, we import the `pyplot` submodule of `matplotlib`. We rename it to `plt`, which is short, and will save us some typing. Calling it `plt` follows the most popular naming convention.

Second, we import Numpy in the same way we always have. Matplotlib is written to work with Numpy arrays. If you want to plot some data, and it isn’t in a Numpy array, you should convert it first.

Third, we call the `subplots()` function, and use *sequence unpacking* to unpack the returned container into individual objects without storing the overall container. “Subplots” sounds like it will make many different plots all on one figure, but if you look at the documentation¹⁰ the number of rows and columns defaults to one and one, respectively.

`plt.subplots()` returns a tuple¹¹¹² of two things: a `Figure` object, and one or more `Axes` object(s). These two classes will require some explanation.

1. A `Figure` object¹³ is the overall visualization object you’re making. It holds onto all of the plot elements. If you want to save all of your progress (e.g. with `fig.savefig('my_picture.png')`), you’re saving the overall `Figure` object.
2. One or more `Axes` objects¹⁴ are contained in a `Figure` object. Each is “what you think of as ‘a plot’¹⁵.” They hold onto two `Axis` objects (in the case of 2-dimensional plots) or three (in the case of 3-dimensional arguments). We are usually calling the methods of these objects to effect changes on a plot.

In line four, we call the `hist()` method¹⁶ of the `Axes` object called `ax`. We assign the output of `.hist()` to a variable `_`. This is done to suppress the printing of the method’s output, and because this variable name is a Python convention that signals the object is temporary and will not be used later in the program. There are many more plots available than plain histograms. Each one has its own method, and you can peruse the options in the documentation¹⁷.

If you want to make figures that are more elaborate, just keep calling different methods of `ax`. If you want to fit more subplots to the same figure, add more `Axes` objects. Here is an example using some code from one of the official

¹⁰https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html#matplotlib-pyplot-subplots

¹¹<https://docs.python.org/3.3/library/stdtypes.html?highlight=tuple#tuple>

¹²We didn’t talk about tuples in Chapter 2, but you can think of them as being similar to lists. They are containers that can hold elements of different types. There are a few key differences, though: they are made with parentheses (e.g. `('a')`) instead of square brackets, and they are immutable instead of mutable.

¹³https://matplotlib.org/stable/api/figure_api.html#matplotlib.figure.Figure

¹⁴https://matplotlib.org/stable/api/axes_api.html#the-axes-class

¹⁵<https://matplotlib.org/stable/tutorials/introductory/usage.html#axes>

¹⁶https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.hist.html#matplotlib.axes.Axes.hist

¹⁷https://matplotlib.org/stable/api/axes_api.html#plotting

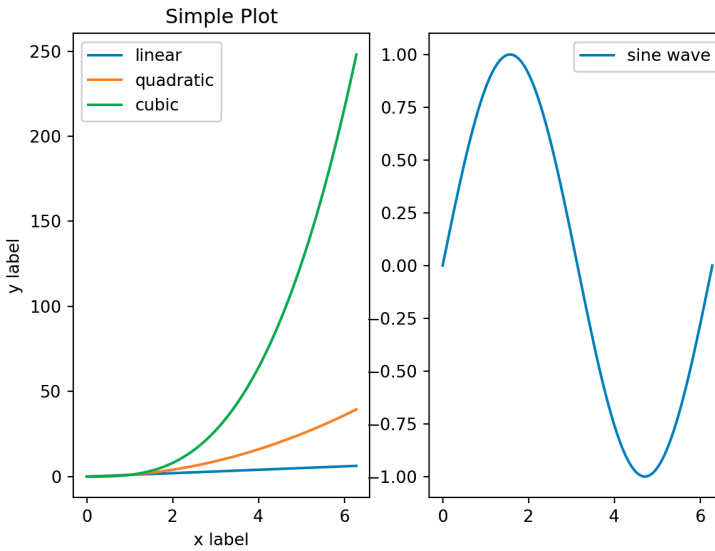


FIGURE 13.9: Side-by-side line plots in matplotlib.

Matplotlib tutorials¹⁸. The plot generated from the code is displayed in [Figure 13.9](#).

```
# x values grid shared by both subplots
x = np.linspace(0, 2*np.pi, 100)

# create two subplots...one row two columns
fig, myAxes = plt.subplots(1, 2) # kind of like par(mfrow=c(1,2)) in R

# first subplot
myAxes[0].plot(x, x, label='linear') # Plot some data on the axes.
## [<matplotlib.lines.Line2D object at 0x7f2e0412bef0>]
myAxes[0].plot(x, x**2, label='quadratic') # Plot more data
## [<matplotlib.lines.Line2D object at 0x7f2e0413a048>]
myAxes[0].plot(x, x**3, label='cubic') # ... and some more.
## [<matplotlib.lines.Line2D object at 0x7f2e0413a390>]
myAxes[0].set_xlabel('x label') # Add an x-label to the axes.
## Text(0.5, 0, 'x label')
myAxes[0].set_ylabel('y label') # Add a y-label to the axes.
## Text(0, 0.5, 'y label')
```

¹⁸<https://matplotlib.org/stable/tutorials/introductory/usage.html#the-object-oriented-interface-and-the-pyplot-interface>

```

myAxes[0].set_title("Simple Plot") # Add a title to the axes.
## Text(0.5, 1.0, 'Simple Plot')
myAxes[0].legend() # Add a legend.

# second subplot
## <matplotlib.legend.Legend object at 0x7f2e042a4ac8>
myAxes[1].plot(x,np.sin(x), label='sine wave')
## [<matplotlib.lines.Line2D object at 0x7f2e0413aba8>]
myAxes[1].legend()
## <matplotlib.legend.Legend object at 0x7f2e0415df28>
plt.show()

```

13.4 Plotting with Pandas

Pandas provides several `DataFrame` and `Series` methods that perform plotting. These methods are mostly just wrapper functions around Matplotlib code. They are written for convenience, so generally speaking, plotting can be done more quickly with Pandas compared to Matplotlib. Here I describe a few options that are available, and the documentation¹⁹ provides many more details for the curious reader.

The `.plot()`²⁰ method is very all-encompassing because it allows you to choose between many different plot types: a line plot, horizontal and vertical bar plots, histograms, boxplots, density plots, area plots, pie charts, scatterplots and hexbin plots. If you only want to remember one function name for plotting in Pandas, this is it.

If you already have Pandas imported, you can make good-looking plots in just one line of code. The default plot type for `.plot()` is a line plot, so there tends to be less typing if you're working with time series data.

```

import pandas as pd
df = pd.read_csv("data/gspc.csv")
df.head()

```

	Index	GSPC.Open	GSPC.High	GSPC.Low	GSPC.Close	GSPC.Volume	GSPC.Adjusted
## 0	2007-01-03	1418.030029	1429.420044	1407.859985	1416.599976	3.429160e+09	1416.599976
## 1	2007-01-04	1416.599976	1421.839966	1408.430054	1418.339966	3.004460e+09	1418.339966
## 2	2007-01-05	1418.339966	1418.339966	1405.750000	1409.709961	2.919400e+09	1409.709961

¹⁹https://pandas.pydata.org/docs/user_guide/visualization.html#chart-visualization

²⁰<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html>


```
## 3  2007-01-08  1409.260010  1414.979980  1403.969971  1412.839966  2.763340e+09  1412.839966
## 4  2007-01-09  1412.839966  1415.609985  1405.420044  1412.109985  3.038380e+09  1412.109985
df['GSPC.Adjusted'].plot()
## <matplotlib.axes._subplots.AxesSubplot object at 0x7f2e04117da0>
```

Choosing among nondefault plot types can be done in a variety of ways. You can either use the `.plot` accessor data member of a `DataFrame`, or you can pass in different strings to `.plot()`'s `kind=` parameter. Third, some plot types (e.g. boxplots and histograms) have their own dedicated methods.

```
df['returns'] = df['GSPC.Adjusted'].pct_change()
df['returns'].plot(kind='hist')
# same as df['returns'].plot.hist()
# same as df['returns'].hist()
## <matplotlib.axes._subplots.AxesSubplot object at 0x7f2e04117da0>
```

There are also several freestanding plotting functions²¹ (not methods) that take in `DataFrames` and `Series` objects. Each of these functions is typically imported individually from the `pandas.plotting` submodule.

The following code is an example of creating a “lag plot,” which is simply a scatterplot between a time series’ lagged and nonlagged values. The primary benefit of this function over `.plot()` is that this function does not require you to construct an additional column of lagged values, and it comes up with good default axis labels.

```
from pandas.plotting import lag_plot
lag_plot(df['returns'])
## <matplotlib.axes._subplots.AxesSubplot object at 0x7f2e04117da0>
```

13.5 Exercises

13.5.1 R Questions

1.

The density of a particular bivariate Gaussian distribution is

$$f(x, y) = \frac{1}{2\pi} \exp \left[-\frac{x^2 + y^2}{2} \right]. \quad (1)$$

²¹https://pandas.pydata.org/docs/user_guide/visualization.html#visualization-tools

The random elements X and Y , in this particular case, are independent, each have unit variance, and zero mean. In this case, the marginal for X is a mean 0, unit variance normal distribution:

$$g(x) = \frac{1}{\sqrt{2\pi}} \exp \left[-\frac{x^2}{2} \right]. \quad (2)$$

- a) Generate two plots of the bivariate density. For one, use `persp()`. For the other, use `contour()`.
- b) Generate a third plot of the univariate density.

2.

Consider again the Militarized Interstate Disputes (v5.0) (Palmer et al., 0) data set from The Correlates of War Project²². A description of this data was given in the previous chapter. Feel free to re-use the code you used in the related exercise.

- a) Use a scatterplot to display the relationship between the maximum duration and the end year. Plot each country as a different color.

13.5.2 Python Questions

1.

Reproduce [Figure 15.1](#)) in [Section 15.2.1.1](#) that displays the simple “spline” function. Feel free to use any of the visible code in the text.

2.

The “4 Cs of diamonds”—or the four most important factors that explain a diamond’s price—are cut, color, clarity and carat. How does the price of diamonds vary as these three factors vary?

- Consider the data set `diamonds.csv` from (Wickham, 2016) and plot the price against carat.
- Visualize the other categorical factors simultaneously. Determine the most elegant way of visualizing all four factors at once and create a plot that does that. Determine the worst way to visualize all factors at once, and create that plot as well.

²²<https://correlatesofwar.org/>



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part III

Programming Styles



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Object-Oriented Programming (OOP) is a way of thinking about how to organize programs. This way of thinking focuses on objects. In the next chapter, we focus on organizing programs by functions, but for now we stick to objects. We already know about objects from the last chapter, so what's new here?

The difference is that we're creating our own *types* now. In the last chapter we learned about built-in types: floating point numbers, lists, arrays, functions, etc. Now we will discuss broadly how one can create his own types in both R and Python. These user-defined types can be used as cookie cutters. Once we have the cookie cutter, we can make as many cookies as we want!

We will not go into this too deeply, but it is important to know how code works so that we can use it more effectively. For instance, in Python, we frequently write code like `my_data_frame.doSomething()`. The material in this chapter will go a long way to describe how we can make our own types with custom behavior.

Here are a few abstract concepts that will help thinking about OOP. They are not mutually exclusive, and they aren't unique to OOP, but understanding these words will help you understand the purpose of OOP. Later on, when we start looking at code examples, I will alert you to when these concepts are coming into play.

- **Composition** refers to the idea when one type of object *contains* an object of another type. For example, a linear model object could hold on to estimated regression coefficients, residuals, etc.
- **Inheritance** takes place when an object can be considered to be of another type(s). For example, an analysis of variance linear regression model might be a special case of a general linear model.
- **Polymorphism** is the idea that the programmer can use the same code on objects of different types. For example, built-in functions in both R and Python can work on arguments of a wide variety of different types.

- **Encapsulation** is another word for complexity hiding. Do you have to understand every line of code in a package you’re using? No, because a lot of details are purposefully hidden from you.
- **Modularity** is an idea related to encapsulation—it means splitting something into independent pieces. How you split code into different files, different functions, different classes—all of that has to do with modularity. It promotes encapsulation, and it allows you to think about only a few lines of code at a time.
- The **interface**, between you and the code you’re using, describes *what* can happen, but not *how* it happens. In other words, it describes some functionality so that you can decide whether you want to use it, but there are not enough details for you to make it work yourself. For example, all you have to do to be able to estimate a complicated statistical model is to look up some documentation.¹ In other words, you only need to be familiar with the interface, not the implementation.
- The **implementation** of some code you’re using describes *how* it works in detail. If you are a package author, you can change your code’s implementation “behind the scenes” and ideally, your end-users would never notice.

14.1 OOP in Python

14.1.1 Overview

In Python, classes² are user-defined types. When you define your own class, you describe what kind of information it holds onto, and how it behaves.

To define your own type, use the `class` keyword³. Objects created with a user-defined class are sometimes called **instances**. They behave according to the rules written in the class definition—they always have data and/or functions bundled together in the same way, but these instances do not all have the same data.

To be more clear, classes may have the following two things in their definition.

- **Attributes (aka data members)** are pieces of data “owned” by an instance created by the class.

¹Just because you can do this, doesn’t mean you *should*, though!

²<https://docs.python.org/3/tutorial/classes.html>

³<https://docs.python.org/3/tutorial/classes.html#class-definition-syntax>

- **(Instance) methods** are functions “owned” by an instance created by the class. They can use and/or modify data belonging to the class.

14.1.2 A First Example

Here’s a simple example. Say we are interested in calculating, from numerical data $x_1, \dots, x_n$, a sample mean:

$$\bar{x}_n = \frac{\sum_{i=1}^n x_i}{n}. \quad (14.1)$$

In Python, we can usually calculate this one number very easily using `np.average`. However, this function requires that we pass into it all of the data at once. What if we don’t have all the data at any given time? In other words, suppose that the data arrive intermittently. We might consider taking advantage of a recursive formula for the sample means.

$$\bar{x}_n = \frac{(n-1)\bar{x}_{n-1} + x_n}{n} \quad (14.2)$$

How would we program this in Python? A first option: we might create a variable `my_running_ave`, and after every data point arrives, we could

```
my_running_ave = 1.0
my_running_ave
## 1.0
my_running_ave = ((2-1)*my_running_ave + 3.0)/2
my_running_ave
## 2.0
my_running_ave = ((3-1)*my_running_ave + 2.0)/3
my_running_ave
## 2.0
```

There are a few problems with this. Every time we add a data point, the formula slightly changes. Every time we update the average, we have to write a different line of code. This opens up the possibility for more bugs, and it makes your code less likely to be used by other people and more difficult to understand. And if we were trying to code up something more complicated than a running average? That would make matters even worse.

A second option: write a class that holds onto the running average, and that has

1. an `update` method that updates the running average every time a new data point is received, and

2. a `get_current_xbar` method that gets the most up-to-date information for us.

Using our code would look like this:

```
my_ave = RunningMean() # create running average object
my_ave.get_current_xbar() # no data yet!
my_ave.update(1.) # first data point
my_ave.get_current_xbar() # xbar_1
## 1.0
my_ave.update(3.) # second data point
my_ave.get_current_xbar() # xbar_2
## 2.0
my_ave.n # my_ave.n instead of self.n
## 2
```



There is a Python convention that stipulates class names should be written in **UpperCamelCase** (e.g. `RunningMean`).

That's much better! Notice the *encapsulation*—while looking at this code we do not need to think about the mathematical formula that is used to process the data. We only need to type in the correct data being used. In other words, the *implementation* is separated from the *interface*. The interface in this case, is just the name of the class methods, and the arguments they expect. That's all we need to know about to use this code.

After seeing these new words that are unfamiliar and long, it's tempting to dismiss these new ideas as superfluous. After all, if you are confident that you can get your program working, why stress about all these new concepts? If it ain't broke, don't fix it, right?

- I urge you to try to keep an open mind, particularly if you are already confident that you understand the basics of programming in R and Python. The topics in this chapter are more centered around design choices. This material won't help you write a first draft of a script even faster, but it will make your code much better. Even though you will have to slow down a bit before you start typing, thinking about your program more deeply will prevent bugs and allow more people to use your code.

Classes (obviously) need to be defined before they are used, so here is the definition of our class.

```
class RunningMean:
    """Updates a running average"""
    def __init__(self):
        self.current_xbar = 0.0
        self.n = 0
    def update(self, new_x):
        self.n += 1
        self.current_xbar *= (self.n-1)
        self.current_xbar += new_x
        self.current_xbar /= self.n
    def get_current_xbar(self):
        if self.n == 0:
            return None
        else:
            return self.current_xbar
```



Methods that look like `__init__`, or that possess names that begin and end with two underscores, are called **dunder (double underscore) methods**, **special methods** or **magic methods**. There are many that you can take advantage of! For more information see this^a.

^a<https://docs.python.org/3/reference/datamodel.html#special-method-names>

Here are the details of the class definition:

1. Defining class methods looks exactly like defining functions! The primary difference is that the first argument must be `self`. If the definition of a method refers to `self`, then this allows the class instance to refer to its own (heretofore undefined) data attributes. Also, these method definitions are indented inside the definition of the class.
2. This class owns two data attributes. One to represent the number of data points seen up to now (`n`), and another to represent the current running average (`current_xbar`).
3. Referring to data members requires dot notation. `self.n` refers to the `n` belonging to any instance. This data attribute is free to vary between all the objects instantiated by this class.
4. The `__init__` method performs the setup operations that are performed every time any object is instantiated.

5. The `update` method provides the core functionality using the recursive formula displayed above.
6. `get_current_xbar` simply returns the current average. In the case that this function is called before any data has been seen, it returns `None`.

A few things you might find interesting:

- i. Computationally, there is never any requirement that we must hold *all* of the data points in memory. Our data set could be infinitely large, and our class will hold onto only one floating point number, and one integer.
- ii. This example is generalizable to other statistical methods. In a mathematical statistics course, you will learn about a large class of models having *sufficient statistics*. Most sufficient statistics have recursive formulas like the one above. Second, many algorithms in *time series analysis* have recursive formulas and are often needed to analyze large streams of data. They can all be wrapped into a class in a way that is similar to the above example.

14.1.3 Adding Inheritance

How can we use inheritance in statistical programming? A primary benefit of inheritance is code re-use, so one example of inheritance is writing a generic algorithm as a base class, and a specific algorithm as a class that inherits from the base class. For example, we could re-use the code in the `RunningMean` class in a variety of other classes.

Let's make some assumptions about a *parametric model* that is generating our data. Suppose I assume that the data points $x_1, \dots, x_n$ are a “random sample”⁴ from a normal distribution with mean μ and variance $\sigma^2 = 1$. μ is assumed to be unknown (this is, after all, and interval for μ), and σ^2 is assumed to be known, for simplicity.

A 95% confidence interval for the true unknown population mean μ is

$$\left(\bar{x} - 1.96\sqrt{\frac{\sigma^2}{n}}, \bar{x} + 1.96\sqrt{\frac{\sigma^2}{n}} \right). \quad (14.3)$$

The width of the interval shrinks as we get more data (as $n \rightarrow \infty$). We can write another class that, not only calculates the center of this interval, $\bar{x}$, but also returns the interval endpoints.

⁴Otherwise known as an independent and identically distributed sample

If we wrote another class from scratch, then we would need to rewrite a lot of the code that we already have in the definition of `RunningMean`. Instead, we'll use the idea of *inheritance*⁵.

```
import numpy as np

class RunningCI(RunningMean):# <-notice what's inside the parentheses
    """Updates a running average and
    gives you a known-variance confidence interval"""

    def __init__(self, known_var):
        super().__init__()
        self.known_var = known_var

    def get_current_interval(self):
        if self.n == 0:
            return None
        else:
            half_width = 1.96 * np.sqrt(self.known_var / self.n)
            left_num = self.current_xbar - half_width
            right_num = self.current_xbar + half_width
            return np.array([left_num, right_num])
```

The parentheses in the first line of the class definition signal that this new class definition is inheriting from `RunningMean`. Inside the definition of this new class, when I refer to `self.current_xbar` Python knows what I'm referring to because it is defined in the base class. Last, I am using `super()` to access the base class's methods, such as `__init__`.

```
my_ci = RunningCI(1) # create running average object
my_ci.get_current_xbar() # no data yet!
my_ci.update(1.)
my_ci.get_current_interval()
## array([-0.96,  2.96])
my_ci.update(3.)
my_ci.get_current_interval()
## array([0.61407071, 3.38592929])
```

This example also demonstrates **polymorphism**. Polymorphism comes from the Greek for “many forms.” “Forms” means “type” or “class” in this case. If

⁵<https://docs.python.org/3/tutorial/classes.html#inheritance>

the same code (usually a function or method) works on objects of different types, that's polymorphic. Here, the `update` method worked on an object of class `RunningCI`, as well as an object of `RunningMean`.

Why is this useful? Consider this example.

```
for datum in time_series:
    for thing in obj_list:
        thing.update(xt)
```

Inside the inner `for` loop, there is no need to include conditional logic that tests for what kind of type each `thing` is. We can iterate through time more succinctly.

```
for datum in time_series:
    for thing in obj_list:
        if isinstance(thing, class1):
            thing.updatec1(xt)
        if isinstance(thing, class2):
            thing.updatec2(xt)
        if isinstance(thing, class3):
            thing.updatec3(xt)
        if isinstance(thing, class4):
            thing.updatec4(xt)
        if isinstance(thing, class5):
            thing.updatec5(xt)
        if isinstance(thing, class6):
            thing.updatec6(xt)
```

If, in the future, you add a new class called `class7`, then you need to change this inner `for` loop, as well as provide new code for the class.

14.1.4 Adding in Composition

Composition also enables code re-use. Inheritance ensures an “is a” relationship between base and derived classes, and composition promotes a “has a” relationship. Sometimes it can be tricky to decide which technique to use, especially when it comes to statistical programming.

Regarding the example above, you might argue that a confidence interval isn't a particular type of a sample mean. Rather, it only *has a* sample mean. If you believe this, then you might opt for a composition based model instead. With composition, the derived class (the confidence interval class) will be decoupled

from the base class (the sample mean class). This decoupling will have a few implications. In general, composition is more flexible, but can lead to longer, uglier code.

1. You will lose polymorphism.
2. Your code might become less re-usable.
 - You have to write any derive class methods you want because you don't inherit any from the base class. For example, you won't automatically get the `.update()` or the `.get_current_xbar()` method for free. This can be tedious if there are a lot of methods you want both classes to have that should work the same exact way for both classes. If there are, you would have to re-write a bunch of method definitions.
 - On the other hand, this could be good if you have methods that behave completely differently. Each method you write can have totally different behavior in the derived class, even if the method names are the same in both classes. For instance, `.update()` could mean something totally different in these two classes. Also, in the derive class, you can still call the base class's `.update()` method.
3. Many-to-one relationships are easier. It's generally easier to "own" many base class instances rather than inherit from many base classes at once. This is especially true if this is the only book on programming you plan on reading—I completely avoid the topic of multiple inheritance!

! Sometimes it is very difficult to choose between using composition or using inheritance. However, this choice should be made very carefully. ■ If you make the wrong one, and realize too late, *refactoring* your code might be very time consuming!

Here is an example implementation of a confidence interval using composition. Notice that this class "owns" a `RunningMean` instance called `self.mean`. This is contrast with *inheriting* from the `RunningMean` class.

```
class RunningCI2:
    """Updates a running average and
    gives you a known-variance confidence interval"""

    def __init__(self, known_var):
```

```

    self.mean = RunningMean()
    self.known_var = known_var

    def update(self, new_x):
        self.mean.update(new_x)

    def get_current_interval(self):
        if self.n == 0:
            return None
        else:
            half_width = 1.96 * np.sqrt(self.known_var / self.n)
            left = self.mean.get_current_xbar() - half_width
            right = self.mean.get_current_xbar() + half_width
            return np.array([left, right])

```

14.2 OOP in R

R, unlike Python, has many different kinds of classes. In R, there is not only one way to make a class. There are many! I will discuss

- S3 classes,
- S4 classes,
- Reference classes, and
- R6 classes.

If you like how Python does OOP, you will like reference classes and R6 classes, while S3 and S4 classes will feel strange to you.

It's best to learn about them chronologically, in my opinion. S3 classes came first, S4 classes sought to improve upon those. Reference classes rely on S4 classes, and R6 classes are an improved version of Reference classes (Wickham, 2014).

14.2.1 S3 Objects: The Big Picture

With S3 (and S4) objects, calling a method `print()` will not look like this.

```
my_obj.print()
```

Instead, it will look like this:

```
print(my_obj)
```

The primary goal of S3 is *polymorphism* (Grolemund, 2014). We want functions like `print()`, `summary()` and `plot()` to behave differently when objects of a different type are passed in to them. Printing a linear model should look a lot different than printing a data frame, right? So we can write code like the following, we only have to remember fewer functions as an end-user, and the “right” thing will always happen. If you’re writing a package, it’s also nice for your users that they’re able to use the regular functions that they’re familiar with. For instance, I allow users of my package `cPseudoMaRg`⁶ (Brown, 2021) to call `print()` on objects of type `cpmResults`. In [Section 13.2](#), `ggplot2` instances, which are much more complicated than plain numeric vectors, are +ed together.

```
# print works on pretty much everything
print(myObj)
print(myObjOfADifferentClass)
print(aThirdClassObject)
```

This works because these “high-level” functions (such as `print()`), will look at its input and choose the most appropriate function to call, based on what kind of type the input has. `print()` is the high-level function. When you run some of the above code, it might not be obvious which specific function `print()` chooses for each input. You can’t see that happening, yet.

Last, recall that this discussion only applies to S3 objects. Not all objects are S3 objects, though. To find out if an object `x` is an S3 object, use `is.object(x)`.

14.2.2 Using S3 Objects

Using S3 objects is so easy that you probably don’t even know that you’re actually using them. You can just try to call functions on objects, look at the output, and if you’re happy with it, you’re good. However, if you’ve ever asked yourself: “why does `print()` (or another function) do different things all the time?” then this section will be useful to you.

`print()` is a **generic function**⁷ which is a function that looks at the type of its first argument, and then calls another, more specialized function depending on what type that argument is. Not all functions in R are generic, but some are. In addition to `print()`, `summary()` and `plot()` are the most ubiquitous generic functions. Generic functions are an *interface*, because the user does not need to concern himself with the details going on behind the scenes.

⁶<https://cran.r-project.org/web/packages/cPseudoMaRg/index.html>

⁷<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Method-dispatching>

In R, a **method** is a specialized function that gets chosen by the generic function for a particular type of input. The method is the *implementation*. When the generic function chooses a particular method, this is called **method dispatch**.

If you look at the definition of a generic function, let's take `plot()` for instance, it has a single line that calls `UseMethod()`.

```
plot
```

```
## function (x, y, ...)
## UseMethod("plot")
## <bytecode: 0x563847001820>
## <environment: namespace:base>
```

`UseMethod()` performs method dispatch. Which methods can be dispatched to? To see that, use the `methods()` function.

```
length(methods(plot))
```

```
## [1] 39
```

All of these S3 class methods share the same naming convention. Their name has the generic function's name as a prefix, then a dot (`.`), then the name of the class that they are specifically written to be used with.

! R's dot-notation is nothing like Python's dot-notation! In R, functions do not *belong* to types/classes like they do in Python! Additionally, in R, the dots are a valid part of any variable name.

Method dispatch works by looking at the `class` attribute of an S3 object argument. An object in R may or may not have a set of **attributes**⁸, which are a collection of name/value pairs that give a particular object extra functionality. Regular vectors in R don't have attributes (e.g. try running `attributes(1:3)`), but objects that are “embellished” versions of a vector might (e.g. run `attributes(factor(1:3))`). Attributes in R are similar to attributes in Python, but they are usually only used as “tags” to elicit some sort of behavior when the object is passed into a function.

⁸<https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Attributes>

❗ `class()` will return misleading results if it's called on an object that isn't an S3 object. Make sure to check with `is.object()` first.

Also, these methods are not *encapsulated* inside a class definition like they are in Python, either. They just look like loose functions—the method definition for a particular class is not defined inside the class. These class methods can be defined just as ordinary functions, out on their own, in whatever file you think is appropriate to define functions in.

As an example, let's try to `plot()` some specific objects.

```
aDF <- data.frame(matrix(rnorm(100), nrow = 10))
is.object(aDF) # is this s3?
## [1] TRUE
class(aDF)
## [1] "data.frame"
plot(aDF)
```

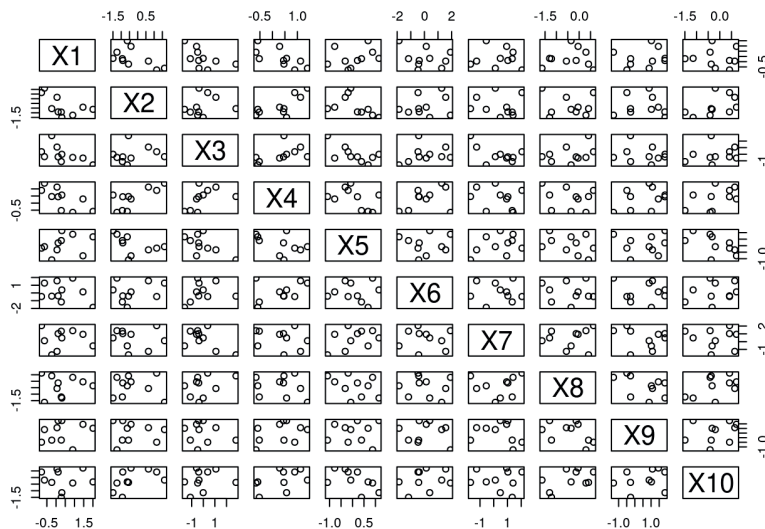


FIGURE 14.1: A scatterplot matrix.

Because `aDF` has its `class` set to `data.frame`, this causes `plot()` to try to find a `plot.data.frame()` method. If this method was not found, R would attempt to find/use a `plot.default()` method. If no default method existed, an error would be thrown. In this case, `plot()` produces a matrix of scatterplots (see [Figure 14.1](#)).

As another example, we can play around with objects created with the `ecdf()` function. This function computes an *empirical cumulative distribution function*, which takes a real number as an input, and outputs the proportion of observations that are less than or equal to that input⁹ (see [Figure 14.2](#)).

```
myECDF <- ecdf(rnorm(100))
is.object(myECDF)
## [1] TRUE
class(myECDF)
## [1] "ecdf"      "stepfun"   "function"
plot(myECDF)
```

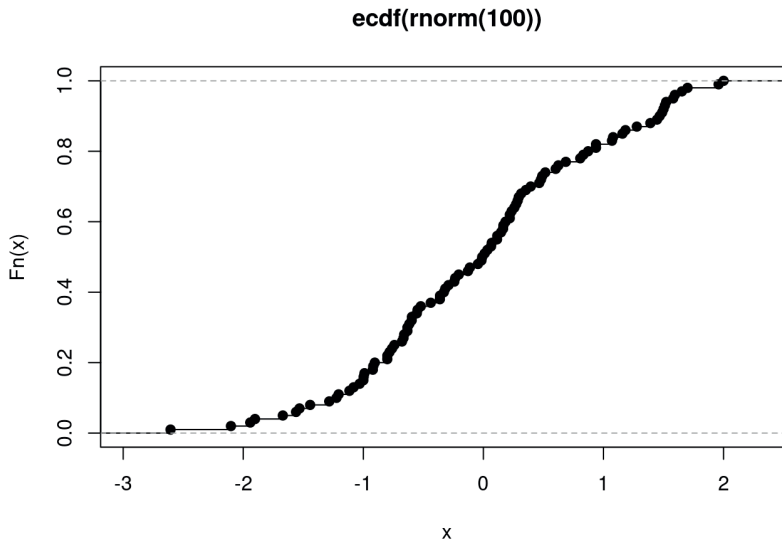


FIGURE 14.2: Plotting an empirical cumulative distribution function.

This is how *inheritance* works in S3. The `ecdf` class inherits from the `stepfun` class, which in turn inherits from the `function` class. When you call `plot(myECDF)`, ultimately `plot.ecdf()` is used on this object. However, if `plot.ecdf()` did not exist, `plot.stepfun()` would be tried. S3 inheritance in R is much simpler than Python's inheritance!

⁹It's defined as $\hat{F}(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(X_i \leq x)$.

14.2.3 Creating S3 Objects

Creating an S3 object is very easy, and is a nice way to spruce up some bundled up object that you're returning from a function, say. All you have to do is tag an object by changing its class attribute. Just assign a character vector to it!

Here is an example of creating an object of `CoolClass`.

```
myThing <- 1:3
attributes(myThing)
## NULL
class(myThing) <- "CoolClass"
attributes(myThing) # also try class(myThing)
## $class
## [1] "CoolClass"
```

`myThing` is now an instance of `CoolClass`, even though I never defined what a `CoolClass` was ahead of time! If you're used to Python, this should seem very strange. Compared to Python, this approach is very flexible, but also, kind of dangerous, because you can change the classes of existing objects. You shouldn't do that, but you could if you wanted to.

After you start creating your own S3 objects, you can write your own methods associated with these objects. That way, when a user of your code uses typical generic functions, such as `summary()`, on your S3 object, you can control what interesting things will happen to the user of your package. Here's an example.

```
summary(myThing)
## [1] "No summary available!"
## [1] "Cool Classes are too cool for summaries!"
## [1] ":"
```

The `summary()` method I wrote for this class is the following.

```
summary.CoolClass <- function(object,...){
  print("No summary available!")
  print("Cool Classes are too cool for summaries!")
  print(":")
}
```

When writing this, I kept the signature the same as `summary()`'s.

14.2.4 S4 Objects: The Big Picture

S4 was developed after S3. If you look at your search path (type `search()`), you will see `package:methods`. That's where all the code you need to do S4 is. Here are the similarities and differences between S3 and S4.

- They both use generic functions and methods work in the same way.
- Unlike in S3, S4 classes allow you to use multiple dispatch, which means the generic function can dispatch on multiple arguments, instead of just the first argument.
- S4 class definitions are strict—they aren't just name tags like in S3.
- S4 inheritance feels more like Python's. You can think of a base class object living inside a child class object.
- S4 classes can have default data members via `prototypes`.

Much more information about S4 classes can be obtained by reading [Chapter 15](#) in Hadley Wickham's book.¹⁰

14.2.5 Using S4 Objects

One CRAN package that uses S4 is the `Matrix` package.¹¹ Here is a short and simple code example.



S4 objects are also extremely popular in packages hosted on Bioconductor^a. Bioconductor is similar to CRAN, but its software has a much more specific focus on bioinformatics. To download packages from Bioconductor, you can check out the installation instructions provided here^b.

^a<https://www.bioconductor.org/>

^b<https://bioconductor.org/help/course-materials/2017/Zurich/S4-classes-and-methods.html>

```
library(Matrix)
M <- Matrix(10 + 1:28, 4, 7)
isS4(M)
## [1] TRUE
M
## 4 x 7 Matrix of class "dgeMatrix"
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]  11  15  19  23  27  31  35
```

¹⁰<https://adv-r.hadley.nz/s4.html>

¹¹<https://cran.r-project.org/web/packages/Matrix/vignettes/Intro2Matrix.pdf>

```
## [2,] 12 16 20 24 28 32 36
## [3,] 13 17 21 25 29 33 37
## [4,] 14 18 22 26 30 34 38
M@Dim
## [1] 4 7
```

Inside an S4 object, data members are called **slots**, and they are accessed with the @ operator (instead of the \$ operator). Objects can be tested if they are S4 with the function `isS4()`. Otherwise, they look and feel just like S3 objects.

14.2.6 Creating S4 Objects

Here are the key takeaways:

- create a new S4 class with `setClass()`,
- create a new S4 object with `new()`,
- S4 classes have a fixed amount of slots, a name, and a fixed inheritance structure.

Let's do an example that resembles the example we did in Python, where we defined a `RunningMean` class and a `RunningCI` class.

```
setClass("RunningMean",
  slots = list(n = "integer",
               currentXbar = "numeric"))
setClass("RunningCI",
  slots = list(knownVar = "numeric"),
  contains = "RunningMean")
```

Here, unlike in S3 class's, we actually have to define a class with `setClass()`. In the parlance of S4, `slots=` are a class' data members, and `contains=` signals that one class inherits from another (even though “contains” kind of sounds like it's suggesting *composition*).

New objects can be created with the `new()` function after this is accomplished.

```
new("RunningMean", n = 0L, currentXbar = 0)
new("RunningCI", n = 0L, currentXbar = 0, knownVar = 1.0)
```

Next we want to define an `update()` generic function that will work on objects of both types. This is what gives us *polymorphism*. The generic `update()` will call specialized methods for objects of class `RunningMean` and `RunningCI`.

Recall that in the Python example, each class had its own update method. Here, we still have a specialized method for each class, but S4 methods don't have to be defined inside the class definition, as we can see below.

```
## Creating a new generic function for 'update' in the global environment
## [1] "update"
```

```
setGeneric("update", function(oldMean, newNum) {
  standardGeneric("update")
})
setMethod("update",
  c(oldMean = "RunningMean", newNum = "numeric"),
  function(oldMean, newNum) {
    oldN <- oldMean@n
    oldAve <- oldMean@currentXbar
    newAve <- (oldAve*oldN + newNum)/(oldN + 1)
    newN <- oldN + 1L
    return(new("RunningMean", n = newN, currentXbar = newAve))
  }
)
setMethod("update",
  c(oldMean = "RunningCI", newNum = "numeric"),
  function(oldMean, newNum) {
    oldN <- oldMean@n
    oldAve <- oldMean@currentXbar
    newAve <- (oldAve*oldN + newNum)/(oldN + 1)
    newN <- oldN + 1L
    return(new("RunningCI", n = newN, currentXbar = newAve,
      knownVar = oldMean@knownVar))
  }
)
)
```

Here's a demonstration of using these two classes that mirrors the example in [subsection 14.1.3](#)

```
myAve <- new("RunningMean", n = 0L, currentXbar = 0)
myAve <- update(myAve, 3)
myAve
## An object of class "RunningMean"
## Slot "n":
## [1] 1
##
## Slot "currentXbar":
## [1] 3
```

```

myAve <- update(myAve, 1)
myAve
## An object of class "RunningMean"
## Slot "n":
## [1] 2
##
## Slot "currentXbar":
## [1] 2

myCI <- new("RunningCI", n = 0L, currentXbar = 0, knownVar = 1.0)
myCI <- update(myCI, 3)
myCI
## An object of class "RunningCI"
## Slot "knownVar":
## [1] 1
##
## Slot "n":
## [1] 1
##
## Slot "currentXbar":
## [1] 3
myCI <- update(myCI, 1)
myCI
## An object of class "RunningCI"
## Slot "knownVar":
## [1] 1
##
## Slot "n":
## [1] 2
##
## Slot "currentXbar":
## [1] 2

```

This looks more *functional* (more information on functional programming is available in [Chapter 15](#)) than the Python example because the `update()` function does not change an object with a side effect. Instead, it takes the old object, changes it, returns the new object, and uses assignment to overwrite the object. The benefit of this approach is that the assignment operator (`<-`) signals to us that something is changing. However, there is more data copying involved, so the program is presumably slower than it needs to be.

The big takeaway here is that S3 and S4 don't feel like Python's encapsulated OOP. If we wanted to write stateful programs, we might consider using Reference Classes, or R6 classes.

14.2.7 Reference Classes: The Big Picture

Reference Classes¹² are built on top of S4 classes, and were released in late 2010. They feel very different from S3 and S4 classes, and they more closely resemble Python classes, because

1. their method definitions are *encapsulated* inside class definitions like in Python, and
2. the objects they construct are *mutable*.

So it will feel much more like Python's class system. Some might say using reference classes that will lead to code that is not very R-ish, but it can be useful for certain types of programs (e.g. long-running code, code that performs many/high-dimensional/complicated simulations, or code that circumvents storing large data set in your computer's memory all at once).

14.2.8 Creating Reference Classes

Creating reference classes is done with the function `setRefClass`. I create a class called `RunningMean` that produces the same behavior as that in the previous example.

```
RunningMeanRC <- setRefClass("RunningMeanRC",
                             fields = list(current_xbar = "numeric",
                                           n = "integer"),
                             methods = list(
                               update = function(new_x){
                                 n <- n + 1L
                                 new_sum <- current_xbar*(n-1) + new_x
                                 current_xbar <- new_sum/n
                               })
)
```

This tells us a few things. First, data members are called *fields* now. Second, changing class variables is done with the `<-`. We can use it just as before.

```
my_ave <- RunningMeanRC$new(current_xbar=0, n=0L)
my_ave
## Reference class object of class "RunningMeanRC"
## Field "current_xbar":
## [1] 0
## Field "n":
## [1] 0
my_ave$update(1.)
```

¹²<https://www.rdocumentation.org/packages/methods/versions/3.6.2/topics/ReferenceClasses>

```
my_ave$current_xbar
## [1] 1
my_ave$n
## [1] 1
my_ave$update(3.)
my_ave$current_xbar
## [1] 2
my_ave$n
## [1] 2
```

Compare how similar this code looks to the code in 14.1.2! Note the paucity of assignment operators, and plenty of side effects.

14.2.9 Creating R6 Classes

I quickly implement the above example as an R6 class. A more detailed introduction to R6 classes is provided in the vignette from the package authors.¹³

You'll notice the reappearance of the `self` keyword. R6 classes have a `self` keyword just like in Python. They are similar to reference classes, but there are a few differences:

1. they have better performance than reference classes¹⁴, and
2. they don't make use of the `<-` operator.

```
library(R6)
```

```
RunningMeanR6 <- R6Class("RunningMeanR6",
  public = list(
    current_xbar = NULL,
    n = NULL,
    initialize = function(current_xbar = NA, n = NA) {
      self$current_xbar <- current_xbar
      self$n <- n
    },
    update = function(new_x) {
      newSum <- self$current_xbar*self$n + new_x
      self$n <- self$n + 1L
      self$current_xbar <- newSum / self$n
    }
  )
)
```

¹³<https://r6.r-lib.org/articles/Introduction.html>

¹⁴<https://r6.r-lib.org/articles/Performance.html>

```

my_r6_ave <- RunningMeanR6$new(current_xbar=0, n=0L)
my_r6_ave
## <RunningMeanR6>
##   Public:
##     clone: function (deep = FALSE)
##     current_xbar: 0
##     initialize: function (current_xbar = NA, n = NA)
##     n: 0
##     update: function (new_x)
my_r6_ave$update(1.)
my_r6_ave$current_xbar
## [1] 1
my_r6_ave$n
## [1] 1
my_r6_ave$update(3.)
my_r6_ave$current_xbar
## [1] 2
my_r6_ave$n
## [1] 2

```

14.3 Exercises

14.3.1 Python Questions

1.

If you are interested in estimating a linear regression model, there is a `LinearRegression` class¹⁵ that you might consider using in the `sklearn.linear_model` submodule. In this lab, we will create something similar, but a little more simplified.

A simple linear regression model will take in n independent variables $x_1, \dots, x_n$, and n dependent variables $y_1, \dots, y_n$, and try to describe their relationship with a function:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i. \quad (14.4)$$

The coefficients β_0 and β_1 are unknown, and so must be estimated with the data. Estimating the variance of the noise terms ϵ_i may also be of interest, but we do not concern ourselves with that here.

¹⁵https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

The formulas for the estimated slope (i.e. $\hat{\beta}_1$) and the estimated intercept (i.e. $\hat{\beta}_0$) are as follows:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{j=1}^n (x_j - \bar{x})^2} \quad (14.5)$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}. \quad (14.6)$$

Create a class that performs simple linear regression.

- Name it `SimpleLinReg`.
- Its `__init__()` method should not take any additional parameters. It should set two data attributes/members `est_intercept` and `est_slope`, to `np.nan`.
- Give it a `.fit()` method that takes in two 1-dimensional Numpy arrays. The first should be the array of independent values, and the second should be the set of dependent values.
- `.fit(x,y)` should not return anything, but it should store two data attributes/members: `est_intercept` and `est_slope`. Every time `.fit()` is called, it will re-calculate the coefficient/parameter estimates.
- Give it a `.get_coeffs()` method. It should not make any changes to the data attributes/members of the class. It should simply return a Numpy array with the parameter estimates inside. Make the first element the estimated intercept, and the second element the estimated slope. If no such coefficients have been estimated at the time of its calling, it should return the same size array but with the initial `np.nans` inside.

After you've finished writing your first class, you can bask in the glory and run the following test code:

```
mod = SimpleLinReg()
mod.get_coeffs()
x = np.arange(10)
y = 1 + .2 * x + np.random.normal(size=10)
mod.fit(x,y)
mod.get_coeffs()
```

2.

Reconsider the above question that asked you to write a class called `SimpleLinReg`.

- Write a new class called `LinearRegression2` that preserves all of the existing functionality of `SimpleLinReg`. Do this in a way that does not excessively copy and paste code from `SimpleLinReg`.

- Give your new class a method called `.visualize()` that takes no arguments and plots the most recent data, the data most recently provided to `.fit()`, in a scatterplot with the estimated regression line superimposed.
- Unfortunately, `SimpleLinReg().fit(x,y).get_coeffs()` will not return estimated regression coefficients. Give your new class this functionality. In other words, make `LinearRegression2().fit(x,y).get_coeffs()` spit out regression coefficients. Hint: the solution should only require one extra line of code, and it should involve the `self` keyword.

3.

Consider the following time series model (West and Harrison, 1989)

$$y_t = \beta_t + \epsilon_t \beta_t = \beta_{t-1} + w_t \beta_1 = w_1 \quad (14.7)$$

Here y_t is observed time series data, each ϵ_t is measurement noise with variance V and each w_t is also noise but with variance W . Think of β_t as a time-varying regression coefficient.

Imagine our data are arriving sequentially. The **Kalman Filter** (Kalman, 1960) cite provides an “optimal” estimate of each β_t given all of the information we have up to time t . What’s better is that the algorithm is recursive. Future estimates of β_t will be easy to calculate given our estimates of β_{t-1} .

Let’s call the mean of β_{t-1} (given all the information up to time $t-1$) m_{t-1} , and the variance of β_{t-1} (given all the information up to time $t-1$) P_{t-1} . Then the Kalman recursions for this particular model are

$$M_t = M_{t-1} + \left(\frac{P_{t-1} + W}{P_{t-1} + W + V} \right) (y_t - M_{t-1}) \quad (14.8)$$

$$P_t = \left(1 - \frac{P_{t-1} + W}{P_{t-1} + W + V} \right) (P_{t-1} + W) \quad (14.9)$$

for $t \geq 1$.

- Write a class called `TVIKalman` (TVI stands for time-varying intercept).
- Have `TVIKalman` take two floating points in to its `__init__()` method in this order: `v` and `w`. These two numbers are positive, and are the variance parameters of the model. Store these two numbers. Also, store 0 and $W + V$ as members/attributes called `filter_mean` and `filter_variance`, respectively.
- Give `TVIKalman` another method: `.update(yt)`. This function should not return anything, but it should update the filtering distribution’s mean and variance numbers, `filter_mean` and `filter_variance`, given a new data point.
- Give `TVIKalman` another method: `.get_confidence_interval()`. It should not take any arguments, and it should return a length two Numpy array. The ordered elements

of that array should be M_t plus and minus two standard deviations—a standard deviation at time t is $\sqrt{P_t}$.

- Create a `DataFrame` called `results` with the three columns called `yt`, `lower`, and `upper`. The last two columns should be a sequence of confidence intervals given to you by the method you wrote. The first column should contain the following data: `[-1.7037539, -0.5966818, -0.7061919, -0.1226606, -0.5431923]`. Plot all three columns in a single line plot. Initialize your Kalman Filter object with both `v` and `w` set equal to `.5`.

14.3.2 R Questions

1.

Which of the following classes in R produce objects that are mutable? Select all that apply: S3, S4, reference classes, and R6.

2.

Which of the following classes in R produce objects that can have methods? Select all that apply: S3, S4, reference classes, and R6.

3.

Which of the following classes in R produce objects that can store data? Select all that apply: S3, S4, reference classes, and R6.

4.

Which of the following classes in R have encapsulated definitions? Select all that apply: S3, S4, reference classes, and R6.

5.

Which of the following classes in R have “slots”? Select all that apply: S3, S4, reference classes, and R6.

6.

Which of the following class systems in R is the newest? S3, S4, reference classes, or R6?

7.

Which of the following class systems in R is the oldest? S3, S4, reference classes, or R6?

8.

Which of the following classes in R requires you to `library()` in something? Select all that apply: S3, S4, reference classes, and R6.

9.

Suppose you have the following data set: $X_1, \dots, X_n$. You assume it is a random sample from a Normal distribution with unknown mean and variance parameters, denoted by μ and σ^2 , respectively. Consider testing the null hypothesis that $\mu = 0$ at a significance level of α . To carry out this test, you calculate

$$t = \frac{\bar{X}}{S/\sqrt{n}} \quad (14.10)$$

and you reject the null hypothesis if $|t| > t_{n-1, \alpha/2}$. This is **Student's T-Test** (Student, 1908). Here $S^2 = \sum_i (X_i - \bar{X})^2 / (n - 1)$ is the sample variance, and $t_{n-1, \alpha/2}$ is the $1 - \alpha/2$ quantile of a t-distribution with $n - 1$ degrees of freedom.

- Write a function called `doTTest()` that performs the above hypothesis test. It should accept two parameters: `dataVec` (a vector of data) and `significanceLevel` (which is α). Have the second parameter default to `.05`.
- Have it return an S3 object created from a `list`. The `class` of this `list` should be `"TwoSidedTTest"`. The elements in the `list` should be named `decision` and `testStat`. The `decision` object should be either `"reject"` or `"fail to reject"`. The test stat should be equal to the calculation you made above for t .
- Create a `summary` method for this new class you created: `TwoSidedTTest`.

10.

Suppose you have a target density $p(x)$ that you are only able to evaluate up to a normalizing constant. In other words, suppose that for some $c > 0$, $p(x) = f(x)/c$, and you are only able to evaluate $f(\cdot)$. Your goal is that you would like to be able to approximate the expected value of $p(x)$ (i.e. $\int xp(x)dx$) using some proposal distribution $q(x)$. $q(x)$ is flexible in that you can sample from it, and you can evaluate it. We will use **importance sampling** (Kahn, 1950a) (Kahn, 1950b) to achieve this.¹⁶

Algorithm 1: Importance Sampling i) Sample $X^1, \dots, X^n$ from $q(x)$, ii. For each sample x^i , calculate an unnormalized weight $\tilde{w}^i := \frac{f(x^i)}{q(x^i)}$, iii. Calculate the normalized weights $w^i = \tilde{w}^i / \sum_j \tilde{w}^j$. iv. Calculate the weighted average $\sum_{i=1}^n w^i x^i$.

In practice it is beneficial to use log-densities, because it will avoid underflow issues.

¹⁶Note that this is a similar setup to the accept-reject sampling problem we had earlier, and this algorithm is closely-related with the Monte Carlo algorithm we used in the exercises of [Chapter 3](#).

After you evaluate each $\log \tilde{w}^i$, before you exponentiate them, subtract a number m from all the values. A good choice for m is $\max_i(\log \tilde{w}^i)$. These new values will produce the same normalized weights because

$$w^i = \exp[\log \tilde{w}^i - m] / \sum_j \exp[\log \tilde{w}^j - m]. \quad (14.11)$$

- Create an R6 class called `ImpSamp` that performs importance sampling.
- It should store function data values `qSamp`, `logQEval`, and `logFEval`.
- Give it an `initialize()` method that takes in three arguments: `logFEvalFunc`, `qSampFunc` and `logQEvalFunc`.
- `initialize()` will set the stored function values equal to the function objects passed in.
- The functions performing random sampling should only take a single argument referring to the number of samples desired.
- The evaluation functions should take a `vector` as an input and return a `vector` as an output.
- Write a method called `computeApproxExpec` that computes and returns the importance sampling estimate of $\int xp(x)dx$. Have this method take an integer argument that represents the number of desired samples to use for the computation.
- Is it better to make this code object-oriented, or would you prefer a simple function that spits out the answer? Why?



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Functional Programming (FP) is another way of thinking about how to organize programs. We talked about OOP—another way to organize programs—in the last chapter ([Chapter 14](#)). So how do OOP and FP differ? To put it simply, FP focuses on functions instead of objects. Because we are talking a lot about functions in this chapter, we will assume you have read and understood Section 6.

Neither R nor Python is a purely functional language. For us, FP is a style that we can choose to let guide us, or that we can disregard. You can choose to employ a more functional style, or you can choose to use a more object-oriented style, or neither. Some people tend to prefer one style to other styles, and others prefer to decide which to use depending on the task at hand.

More specifically, a functional programming style takes advantage of **first-class functions** and favors functions that are **pure**.

1. **First-class functions** are (Abelson and Sussman, 1996) functions that
 - can be passed as arguments to other functions,
 - can be returned from other functions, and
 - can be assigned to variables or stored in data structures.
2. **Pure functions**
 - return the same output if they are given the same input, and
 - do not produce **side-effects**.

Side-effects are changes made to non-temporary variables, to the “state” of the program.

We discussed (1) in the beginning of [Chapter 6](#). If you have not used any other programming languages before, you might even take (1) for granted. However, using first-class functions can be difficult in other languages not mentioned in this text.

There is more to say about definition (2). This means you should keep your functions as *modular* as possible, unless you want your overall program to be much more difficult to understand. FP stipulates that

- ideally functions will not refer to non-local variables;
- ideally functions will not (refer to and) modify non-local variables; and
- ideally functions will not modify their arguments.

Unfortunately, violating the first of these three criteria is very easy to do in both of our languages. Recall our conversation about *dynamic lookup* in [subsection 6.8](#). Both R and Python use dynamic lookup, which means you can't reliably control *when* functions look for variables. Typos in variable names easily go undiscovered, and modified global variables can potentially wreak havoc on your overall program.

Fortunately it is difficult to modify global variables inside functions in both R and Python. This was also discussed in [subsection 6.8](#). In Python, you need to make use of the `global` keyword (mentioned in [Section 6.7.2](#)), and in R, you need to use the rare super assignment operator (it looks like `<->`, and it was mentioned in [Section 6.7.1](#)). Because these two symbols are so rare, they can serve as signals to viewers of your code about when and where (in which functions) global variables are being modified.

Last, violating the third criterion is easy in Python and difficult in R. This was discussed earlier in [Section 6.7](#). Python can mutate/change arguments that have a mutable type because it has *pass-by-assignment* semantics (mentioned in [Section 6.7.2](#)), and R generally can't modify its arguments at all because it has *pass-by-value* semantics [Section 6.7.1](#).

This chapter avoids the philosophical discussion of FP. Instead, it takes the applied approach, and provides instructions on how to use FP in your own programs. I try to give examples of *how* you can use FP, and *when* these tools are especially suitable.

One of the biggest tip-offs that you should be using functional programming is if you need to evaluate a single function many times, or in many different ways. This happens quite frequently in statistical computing. Instead of copy/pasting similar-looking lines of code, you might consider *higher-order* functions that take your function as an input, and intelligently call it in all the many ways you want it to. A third option you might also consider is to use a loop (c.f. 11.2). However, that approach is not very functional, and so it will not be heavily-discussed in this section.

Another tip-off that you need FP is if you need many different functions that are all “related” to one another. Should you define each function separately, using excessive copy/paste-ing? Or should you write a function that can elegantly generate any function you need?

Not repeating yourself and re-using code is a primary motivation, but it is not the only one. Another motivation for **functional programming** is clearly explained in Advanced R¹²:

¹<https://adv-r.hadley.nz/fp.html>

²Even though this book only discusses *one* of our languages of interest, this quote applies to both languages.

A functional style tends to create functions that can easily be analysed in isolation (i.e. using only local information), and hence is often much easier to automatically optimise or parallelise.

All of these sound like a good things to have in our code, so let's get started with some examples!

15.1 Functions as Function Inputs in R

Many of the most commonly-used functionals in R have names that end in “apply”. The ones I discuss are `sapply()`, `vapply()`, `lapply()`, `apply()`, `tapply()` and `mapply()`. Each of these takes a function as one of its arguments. Recall that this is made possible by the fact that R has first-class functions.

15.1.1 `sapply()` and `vapply()`

Suppose we have a `data.frame` that has 10 rows and 100 columns. What if we want to take the mean of each column?

An amateurish way to do this would be something like the following.

```
myFirstMean <- mean(myDF[,1])
mySecondMean <- mean(myDF[,2])
# ... so on and so forth ..
myHundredthMean <- mean(myDF[,100])
```

You will need one line of code for each column in the data frame! For data frames with a lot of columns, this becomes quite tedious. You should also ask yourself what happens to you and your collaborators when the data frame changes even slightly, or if you want to apply a different function to its columns. Third, the results are not stored in a single container. You are making it difficult on yourself if you want to use these variables in subsequent pieces of code.



“Don’t repeat yourself” (DRY) is an idea that’s been around for a while and is widely accepted (Hunt and Thomas, 2000). DRY is the opposite of WET^a.

^ahttps://en.wikipedia.org/wiki/Don%27t_repeat_yourself#WET

Instead, prefer the use of `sapply()` in this situation. The “s” in `sapply()` stands for “simplified.” In this bit of code `mean()` is called on each column of the data frame. `sapply()` applies the function over columns, instead of rows, because data frames are internally a `list` of columns.

```
myMeans <- sapply(myDF, mean)
myMeans[1:5]
##           X1           X2           X3           X4           X5
## 0.18577292 0.58759539 -0.05194271 -0.07027537 -0.35365358
```

Each call to `mean()` returns a `double vector` of length 1. This is necessary if you want to collect all the results into a `vector`—remember, all elements of a `vector` have to have the same type. To get the same behavior, you might also consider using `vapply(myDF, mean, numeric(1))`.

In the above case, “simplify” referred to how one-hundred length-1 vectors were simplified into one length-100 `vector`. However, “simplified” does not necessarily imply that all elements will be stored in a `vector`. Consider the `summary` function, which returns a `double vector` of length 6. In this case, one-hundred length-6 vectors were simplified into one 6×100 matrix.

```
mySummaries <- sapply(myDF, summary)
is.matrix(mySummaries)
## [1] TRUE
dim(mySummaries)
## [1] 6 100
```



Another function that is worth mentioning is `replicate()`—it is a wrapper for `sapply()`. Consider a situation where you want to call a function many times with the same inputs. You might try something like `sapply(1:100, function(elem) { return(myFunc(someInput)) })`. Another, more readable, way to do this is `replicate(100, myFunc(someInput))`.

15.1.2 `lapply()`

For functions that do not return amenable types that fit into a `vector`, `matrix` or `array`, they might need to be stored in `list`. In this situation, you would need `lapply()`. The “l” in `lapply()` stands for “list”. `lapply()` always returns a `list` of the same length as the input.

```
regress <- function(y){ lm(y ~ 1) }
myRegs <- lapply(myDF, regress)
length(myRegs)
```

```
## [1] 100
class(myRegs[[1]])
## [1] "lm"
summary(myRegs[[12]])
##
## Call:
## lm(formula = y ~ 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6149 -0.8692 -0.2541  0.7596  2.5718
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.2104      0.4139   0.508   0.623
##
## Residual standard error: 1.309 on 9 degrees of freedom
```

15.1.3 **apply()**

I use `sapply()` and `lapply()` the most, personally. The next most common function I use is `apply()`. You can use it to apply functions to *rows* of rectangular arrays instead of columns. However, it can also apply functions over columns, just as the other functions we discussed can.³

```
dim(myDF)
## [1] 10 100
results <- apply(myDF, 1, mean)
results[1:4]
## [1] 0.18971263 -0.07595286 0.18400138 0.08895979
```

Another example where it can be useful to apply a function to rows is **predicate functions**. A predicate function is just a fancy name for a function that returns a Boolean. I use them to filter out rows of a `data.frame`. Without a predicate function, filtering rows might look something like this on our real estate data (Albemarle County Geographic Data Services Office, 2021) (Ford, 2016).

³`apply()` is everyone's favorite whipping boy whenever it comes to comparing `apply()` against the other `*apply()` functions. This is because it is generally a little slower—it is written in R and doesn't call out to compiled C code. However, in my humble opinion, it doesn't matter all that much because the fractions of a second saved don't always add up in practice.

```

albRealEstate <- read.csv("data/albemarle_real_estate.csv")
firstB <- albRealEstate$YearBuilt == 2006
secondB <- albRealEstate$Condition == "Average"
thirdB <- albRealEstate$City == "CROZET"
subDF <- albRealEstate[(firstB & secondB) | thirdB,]
str(subDF, strict.width = "cut")
## 'data.frame':    3865 obs. of  12 variables:
##  $ YearBuilt      : int  1769 1818 2004 2006 2004 1995 1900 1960 ..
##  $ YearRemodeled: int  1988 1991 NA NA NA NA NA NA NA NA ...
##  $ Condition     : chr  "Average" "Average" "Average" "Average" ..
##  $ NumStories    : num  1.7 2 1 1 1.5 2.3 2 1 1 1 ...
##  $ FinSqFt       : int  5216 5160 1512 2019 1950 2579 1530 800 9..
##  $ Bedroom       : int  4 6 3 3 3 3 4 2 2 2 ...
##  $ FullBath      : int  3 4 2 3 3 2 1 1 1 1 ...
##  $ HalfBath      : int  0 1 1 0 0 1 0 0 0 0 ...
##  $ TotalRooms    : int  8 11 9 10 8 8 6 4 4 4 ...
##  $ LotSize       : num  5.1 453.9 42.6 5 5.5 ...
##  $ TotalValue    : num  1096600 2978600 677800 453200 389200 ...
##  $ City          : chr  "CROZET" "CROZET" "CROZET" "CROZET" ...

```

Complicated filtering criteria can become quite wide, so I prefer to break the above code into three steps.

- Step 1: write a predicate function that returns TRUE or FALSE;
- Step 2: construct a logical vector by `apply()`ing the predicate over rows;
- Step 3: plug the logical vector into the `[` operator to remove the rows.

```

pred <- function(row){
  yrBuiltCorrect <- row['YearBuilt'] == 2006
  aveCond <- row['Condition'] == "Average"
  inCrozet <- row['City'] == "CROZET"
  ( yrBuiltCorrect && aveCond ) || inCrozet
}
whichRows <- apply(albRealEstate, 1, pred)
subDF <- albRealEstate[whichRows,]
str(subDF, strict.width = "cut")
## 'data.frame':    3865 obs. of  12 variables:
##  $ YearBuilt      : int  1769 1818 2004 2006 2004 1995 1900 1960 ..
##  $ YearRemodeled: int  1988 1991 NA NA NA NA NA NA NA NA ...
##  $ Condition     : chr  "Average" "Average" "Average" "Average" ..
##  $ NumStories    : num  1.7 2 1 1 1.5 2.3 2 1 1 1 ...
##  $ FinSqFt       : int  5216 5160 1512 2019 1950 2579 1530 800 9..

```

```
## $ Bedroom      : int  4 6 3 3 3 3 4 2 2 2 ...
## $ FullBath     : int  3 4 2 3 3 2 1 1 1 1 ...
## $ HalfBath     : int  0 1 1 0 0 1 0 0 0 0 ...
## $ TotalRooms   : int  8 11 9 10 8 8 6 4 4 4 ...
## $ LotSize      : num  5.1 453.9 42.6 5 5.5 ...
## $ TotalValue   : num  1096600 2978600 677800 453200 389200 ...
## $ City         : chr  "CROZET" "CROZET" "CROZET" "CROZET" ...
```

15.1.4 `tapply()`

`tapply()` can be very handy when you need it. First, we’ve alluded to the definition before in [subsection 8.1](#), but a **ragged array** is a collection of arrays that all have potentially different lengths. I don’t typically construct such an object and then pass it to `tapply()`. Rather, I let `tapply()` construct the ragged array for me. The first argument it expects is, to quote the documentation, “typically vector-like,” while the second tells us how to break that **vector** into chunks. The third argument is a function that gets applied to each **vector** chunk.

If I wanted the average home price for each city, I could use something like this.

```
str(albRealEstate, strict.width = "cut")
## 'data.frame': 30381 obs. of 12 variables:
## $ YearBuilt : int 1769 1818 2004 2006 2004 1995 1900 1960 ..
## $ YearRemodeled: int 1988 1991 NA NA NA NA NA NA NA ...
## $ Condition : chr "Average" "Average" "Average" "Average" ..
## $ NumStories : num 1.7 2 1 1 1.5 2.3 2 1 1 1 ...
## $ FinSqFt : int 5216 5160 1512 2019 1950 2579 1530 800 9..
## $ Bedroom : int 4 6 3 3 3 3 4 2 2 2 ...
## $ FullBath : int 3 4 2 3 3 2 1 1 1 1 ...
## $ HalfBath : int 0 1 1 0 0 1 0 0 0 0 ...
## $ TotalRooms : int 8 11 9 10 8 8 6 4 4 4 ...
## $ LotSize : num 5.1 453.9 42.6 5 5.5 ...
## $ TotalValue : num 1096600 2978600 677800 453200 389200 ...
## $ City : chr "CROZET" "CROZET" "CROZET" "CROZET" ...
length(unique(albRealEstate$City))
## [1] 6
tapply(albRealEstate$TotalValue, list(albRealEstate$City), mean)[1:4]
## CHARLOTTESVILLE CROZET EARLYSVILLE KESWICK
## 429926.5 436090.5 482711.4 565985.1
```

You might be wondering why we put `albRealEstate$City` into a `list`. That seems kind of unnecessary. This is because `tapply()` can be used with multiple **factors**—this will break down the **vector** input into a finer partition. The second argument must

be one object, though, so all of these `factors` must be collected into a `list`. The following code produces a “pivot table.”

```
pivTable <- taapply(albRealEstate$TotalValue,
                    list(albRealEstate$City, albRealEstate$Condition),
                    mean)

pivTable[,1:5]
##              Average Excellent      Fair      Good      Poor
## CHARLOTTESVILLE 416769.9  625887.9 306380.7 529573.4 304922.4
## CROZET            447585.8  401553.5 241568.8 466798.5 224516.2
## EARLYSVILLE       491269.6  492848.4 286980.0 522938.5 250773.3
## KESWICK           565994.3  664443.5 274270.4 676312.6 172425.0
## NORTH GARDEN      413788.8  529108.3 164646.2 591502.8 161460.0
## SCOTTSVILLE     286787.4  534500.0 196003.4 415207.5 183942.3
```



For functions that return higher-dimensional output, you will have to use something like `by()` or `aggregate()` in place of `taapply()`.

15.1.5 `mapply()`

The documentation of `mapply()`⁴ states `mapply()` is a multivariate version of `sapply()`. `sapply()` worked with univariate functions; the function was called multiple times, but each time with a single argument. If you have a function that takes *multiple arguments*, and you want those arguments to change each time the function is called, then you might be able to use `mapply()`.

Here is a short example. Regarding the `n=` argument of `rnorm()`, the documentation explains, “[i]f `length(n) > 1`, the length is taken to be the number required.” This would be a problem if we want to sample three times from a mean 0 normal first, then twice from a mean 100 normal, and then third, once from a mean 42 normal distribution. We only get three samples when we want six!

```
rnorm(n = c(3,2,1), mean = c(0,100,42), sd = c(.01, .01, .01))
```

```
## [1]  0.01435773 99.99238144 42.01743548
```

```
mapply(rnorm, n = c(3,2,1), mean = c(0,100,42), sd = c(.01, .01, .01))
```

```
## [[1]]
```

```
## [1] -0.0122999207 -0.0064744814 -0.0002297629
```

```
##
```

```
## [[2]]
```

⁴<https://stat.ethz.ch/R-manual/R-devel/library/base/html/mapply.html>

```
## [1] 100.02077 99.99853
##
## [[3]]
## [1] 41.99704
```

15.1.6 `Reduce()` and `do.call()`

Unlike the other examples of functions that take other functions as inputs, `Reduce()` and `do.call()` don't have many outputs. Instead of collecting many outputs into a container, they just output one thing.

Let's start with an example: "combining" data sets. In section 12 we talked about several different ways of combining data sets. We discussed stacking data sets on top of one another with `rbind()` (c.f. [subsection 12.2](#)), stacking them side-by-side with `cbind()` (also in 12.2), and intelligently joining them together with `merge()` (c.f. 12.3).

Now consider the task of combining *many* data sets. How can we combine three or more data sets into one? Also, how do we write DRY code and abide by the DRY principle? As the name of the subsection suggests, we can use either `Reduce()` or `do.call()` as a higher-order function. Just like the aforementioned `*apply()` functions, they take in either `cbind()`, `rbind()`, or `merge()` as a function input. Which one do we pick, though? The answer to that question deals with *how many arguments our lower-order function takes*.

Take a look at the documentation to `rbind()`. Its first argument is `...`, which is the dot-dot-dot⁵ symbol. This means `rbind()` can take a varying number of `data.frames` to stack on top of each other. In other words, `rbind()` is **variadic**.

On the other hand, take a look at the documentation of `merge()`. It only takes two `data.frames` at a time⁶. If we want to combine many data sets, `merge()` needs a helper function.

This is the difference between `Reduce()` and `do.call()`. `do.call()` calls a function once on many arguments, so its function must be able to handle many arguments. On the other hand, `Reduce()` calls a binary function many times on pairs of arguments. `Reduce()`'s function argument gets called on the first two elements, then on the first output and the third element, then on the second output and fourth element, and so on.

Here is an initial example that makes use of four data sets `d1.csv`, `d2.csv`, `d3.csv`, and `d4.csv`. To start, ask yourself how we would read all of these in. There is a temptation to copy and paste `read.csv` calls, but that would violate the DRY principle. Instead,

⁵https://cran.r-project.org/doc/manuals/r-release/R-lang.html#Dot_002ddot_002ddot

⁶Although, it is still variadic. The difference is that the dot-dot-dot symbol does not refer to a varying number of `data.frames`, just a varying number of other things we don't care about in this example.

let's use `lapply()` an anonymous function that constructs a file path string, and then uses it to read in the data set the string refers to.

```
numDataSets <- 4
dataSets <- paste0("d", 1:numDataSets)
dfs <- lapply(dataSets,
              function(name) read.csv(paste0("data/", name, ".csv")))
head(dfs[[3]])
##   id obs3
## 1  a     7
## 2  b     8
## 3  c     9
```

Notice how the above code would only need to be changed by one character if we wanted to increase the number of data sets being read in!⁷

Next, `cbind()`ing them all together can be done as follows. `do.call()` will call the function only once. `cbind()` takes many arguments at once, so this works. This code is even better than the above code in that if `dfs` becomes longer, or changes at all, *nothing* will need to be changed.

```
do.call(cbind, dfs) # DRY! :)
##   id obs1 id obs2 id obs3 id obs4
## 1  a     1  b     5  a     7  a    10
## 2  b     2  a     4  b     8  b    11
## 3  c     3  c     6  c     9  c    12
# cbind(df1,df2,df3,df4) # WET! :(
```

What if we wanted to `merge()` all these data sets together? After all, the `id` column appears to be repeating itself, and some data from `d2` isn't lining up.

```
Reduce(merge, dfs)
##   id obs1 obs2 obs3 obs4
## 1  a     1   4   7   10
## 2  b     2   5   8   11
## 3  c     3   6   9   12
```

Again, this is very DRY code. Nothing would need to be changed if `dfs` grew. Furthermore, trying to `do.call()` the `merge()` function wouldn't work because it can only take two data sets at a time.

⁷To make it even more flexible, we could write code that doesn't assume the functions are all named the same way, or in the same directory together.

15.2 Functions as Function Inputs in Python

15.2.1 Functions as Function Inputs in Base Python

I discuss two functions from base Python that take functions as input. Neither return a `list` or a `np.array`, but they do return different kinds of **iterables**, which are “objects capable of returning their members one at a time,” according to the Python documentation.⁸ `map()`, the function, will return objects of type `map`. `filter()`, the function, will return objects of type `filter`. Often times we will just convert these to the container we are more familiar with.

15.2.1.1 `map()`

`map()`⁹ can call a function repeatedly using elements of a container as inputs. Here is an example of calculating outputs of a *spline* function, which can be useful for coming up with predictors in regression models. This particular spline function is $f(x) = (x - k)1(x \geq k)$, where k is some chosen “knot point (see Figure 15.1).”

```
import numpy as np
my_inputs = np.linspace(start = 0, stop = 2*np.pi)
def spline(x):
    knot = 3.0
    if x >= knot:
        return x-knot
    else:
        return 0.0
output = list(map(spline, my_inputs))
```

We can visualize the mathematical function by plotting its outputs against its inputs. More information on visualization was given in subsection 13.

`map()` can also be used like `mapply()`. In other words, you can apply it to two containers,

```
import numpy as np
x = np.linspace(start = -1., stop = 1.0)
y = np.linspace(start = -1., stop = 1.0)
def f(x,y):
    return np.log(x**2 + y**2)
list(map(f, x, y))[:3]
## [0.6931471805599453, 0.6098017877588092, 0.5228315638793316]
```

⁸<https://docs.python.org/3/glossary.html>

⁹<https://docs.python.org/3/library/functions.html#map>

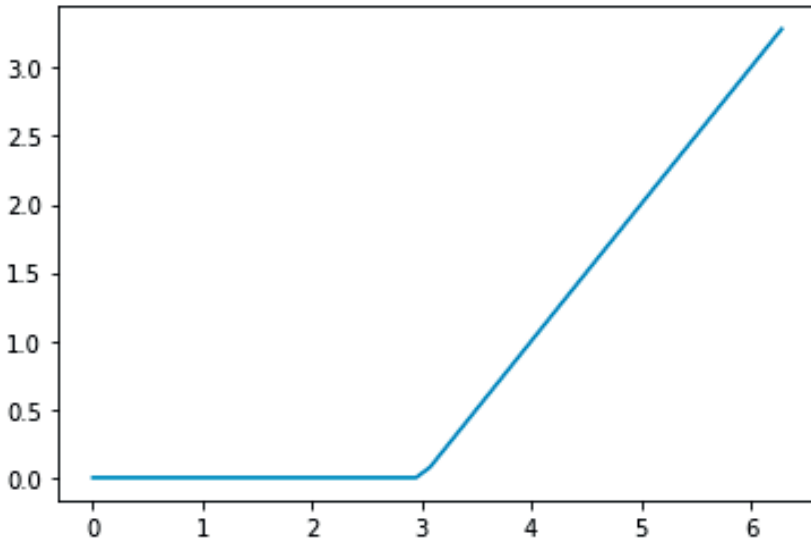


FIGURE 15.1: Our spline function.

15.2.1.2 `filter()`

`filter()`¹⁰ helps remove unwanted elements from a container. It returns an iterable of type `filter`, which we can iterate over or convert to a more familiar type of container. In this example, I iterate over it without converting it.

This code also provides our first example of a **lambda function**¹¹ (Lutz, 2013). Lambda functions are simply another way to define functions. Notice that in this example, we didn't have to name our function. In other words, it was **anonymous**. We can also save a few lines of code.

```
raw_data = np.arange(0,1.45,.01)
for elem in filter(lambda x : x**2 > 2, raw_data):
    print(elem)
## 1.42
## 1.43
## 1.44
```

15.2.2 Functions as Function Inputs in Numpy

Numpy provides a number of functions¹² that facilitate working with `np.ndarrays` in a functional style. For example, `np.apply_along_axis()`¹³ is similar to R's `apply()`.

¹⁰<https://docs.python.org/3/library/functions.html#filter>

¹¹<https://docs.python.org/3/tutorial/controlflow.html#lambda-expressions>

¹²<https://numpy.org/doc/stable/reference/routines.functional.html>

¹³https://numpy.org/doc/stable/reference/generated/numpy.apply_along_axis.html

`apply()` had a `MARGIN=` input (1 sums rows, 2 sums columns), whereas this function has a `axis=` input (0 sums columns, 1 sums rows).

```
import numpy as np
my_array = np.arange(6).reshape((2,3))
my_array
## array([[0, 1, 2],
##        [3, 4, 5]])
np.apply_along_axis(sum, 0, my_array) # summing columns
## array([3, 5, 7])
np.apply_along_axis(sum, 1, my_array) # summing rows
## array([ 3, 12])
```

15.2.3 Functional Methods in Pandas

Pandas' `DataFrames` have an `.apply()` method¹⁴ that is very similar to `apply()` in R,¹⁵ but again, just like the above function, you have to think about an `axis=` argument instead of a `MARGIN=` argument.

```
import pandas as pd
alb_real_est = pd.read_csv("data/albemarle_real_estate.csv")
alb_real_est.shape
## (30381, 12)
alb_real_est.apply(len, axis=0) # length of columns
## YearBuilt      30381
## YearRemodeled  30381
## Condition      30381
## NumStories     30381
## FinSqFt        30381
## Bedroom        30381
## FullBath       30381
## HalfBath       30381
## TotalRooms     30381
## LotSize        30381
## TotalValue     30381
## City           30381
## dtype: int64
```

¹⁴<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.apply.html>

¹⁵You should know that a lot of special-case functions that you typically apply to rows or columns come built-in as `DataFrame` methods. For instance, `.mean()` would allow you to do something like `my_df.mean()`.

```
type(alb_real_est.apply(len, axis=1)) # length of rows
## <class 'pandas.core.series.Series'>
```

Another thing to keep in mind is that `DataFrames`, unlike `ndarrays`, don't have to have the same type for all elements. If you have mixed column types, then summing rows, for instance, might not make sense. This just requires subsetting columns before `.apply()`ing a function to rows. Here is an example of computing each property's "score".

```
import pandas as pd
# alb_real_est.apply(sum, axis=1) # can't add letters to numbers!
def get_prop_score(row):
    return 2*row[0] + 3*row[1]
two_col_df = alb_real_est[['FinSqFt', 'LotSize']]
alb_real_est['Score'] = two_col_df.apply(get_prop_score, 1)
alb_real_est[['FinSqFt', 'LotSize', 'Score']].head(2)
##      FinSqFt  LotSize      Score
## 0      5216    5.102  10447.306
## 1      5160   453.893  11681.679
```

`.apply()` also works with more than one function at a time.

```
alb_real_est[['FinSqFt', 'LotSize']].apply([sum, len])
##      FinSqFt      LotSize
## sum  61730306  105063.1892
## len    30381    30381.0000
```

If you do not want to waste two lines defining a function with `def`, you can use an anonymous lambda function. Be careful, though—if your function is complex enough, then your lines will get quite wide. For instance, this example is pushing it.

```
two_col_df.apply(lambda row : sum(row*[2,3]), 1)[:3]
## 0      10447.306
## 1      11681.679
## 2       3151.770
## dtype: float64
```

The previous example `.apply()`s a *binary* function to each row. The function is binary because it takes two elements at a time. If you want to apply a *unary* function (i.e. it

takes one argument at a time) function to each row for, and for each column, then you can use `.applymap()`¹⁶.

```
alb_real_est[['FinSqFt','LotSize']].applymap(lambda e : e + 1).head(3)
##      FinSqFt  LotSize
## 0         5217    6.102
## 1         5161  454.893
## 2         1513   43.590
```

Last, we have a `.groupby()`¹⁷ method, which can be used to mirror the behavior of R's `tapply()`, `aggregate()` or `by()`. It can take the `DataFrame` it belongs to, and group its rows into multiple sub-`DataFrames`. The collection of sub-`DataFrames` has a lot of the same methods that an individual `DataFrame` has (e.g. the subsetting operators, and the `.apply()` method), which can all be used in a second step of calculating things on each sub-`DataFrame`.

```
type(alb_real_est.groupby(['City']))
## pandas.core.groupby.generic.DataFrameGroupBy
type(alb_real_est.groupby(['City'])['TotalValue'])
## pandas.core.groupby.generic.SeriesGroupBy
```

Here is an example that models some pretty typical functionality. It shows two ways to get the average home price by city. The first line groups the rows by which `City` they are in, extracts the `TotalValue` column in each sub-`DataFrame`, and then `.apply()`s the `np.average()` function on the sole column found in each sub-`DataFrame`. The second `.apply()`s a lambda function to each sub-`DataFrame` directly. More details on this “split-apply-combine” strategy can be found in the Pandas documentation.¹⁸

```
grouped = alb_real_est.groupby(['City'])
grouped['TotalValue'].apply(np.average)
## City
## CHARLOTTESVILLE    429926.502708
## CROZET                436090.502541
## EARLYSVILLE          482711.437566
## KESWICK              565985.092025
## NORTH GARDEN         399430.221519
## SCOTTSVILLE        293666.758242
## Name: TotalValue, dtype: float64
```

¹⁶<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.applymap.html#pandas.DataFrame.applymap>

¹⁷<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.groupby.html>

¹⁸https://pandas.pydata.org/pandas-docs/stable/user_guide/groupby.html


```
grouped.apply(lambda df : np.average(df['TotalValue']))
## City
## CHARLOTTESVILLE      429926.502708
## CROZET                  436090.502541
## EARLYSVILLE            482711.437566
## KESWICK                 565985.092025
## NORTH GARDEN           399430.221519
## SCOTTSVILLE          293666.758242
## dtype: float64
```

15.3 Functions as Function Outputs in R

Functions that create and return other functions are sometimes called **function factories**. Functions are first-class objects in R, so it's easy to return them. What's more interesting is that supposedly temporary objects inside the outer function can be accessed during the call of the inner function after it's returned.

Here is a first quick example.

```
funcFactory <- function(greetingMessage){
  function(name){
    paste(greetingMessage, name)
  }
}
greetWithHello <- funcFactory("Hello")
greetWithHello("Taylor")
## [1] "Hello Taylor"
greetWithHello("Charlie")
## [1] "Hello Charlie"
```

Notice that the **greetingMessage=** argument that is passed in, **"Hello"**, isn't temporary anymore. It lives on so it can be used by all the functions created by `funcFactory()`. This is the most surprising aspect of writing function factories.

Let's now consider a more complicated and realistic example. Let's implement a variance reduction technique called **common random numbers**.

Suppose $X \sim \text{Normal}(\mu, \sigma^2)$, and we are interested in approximating an expectation of a function of this random variable. Suppose that we don't know that

$$\mathbb{E}[\sin(X)] = \sin(\mu) \exp\left(-\frac{\sigma^2}{2}\right) \quad (15.1)$$

for any particular choice of μ and σ^2 , and instead, we choose to use the Monte Carlo method:

$$\hat{\mathbb{E}}[\sin(X)] = \frac{1}{n} \sum_{i=1}^n \sin(X^i) \quad (15.2)$$

where $X^1, \dots, X^n \stackrel{\text{iid}}{\sim} \text{Normal}(\mu, \sigma^2)$ is a large collection of draws from the appropriate normal distribution, probably coming from a call to `rnorm()`. In more realistic situations, the theoretical expectation might not be tractable, either because the random variable has a complicated distribution, or maybe because the functional is very complicated. In these cases, a tool like Monte Carlo might be the only available approach.

Here are two functions that calculate the above quantities for $n = 1000$. `actualExpectSin()` is a function that computes the theoretical expectation for any particular parameter pair. `monteCarloSin()` is a function that implements the Monte Carlo approximate expectation.

```
n <- 1000 # don't hardcode variables that aren't passed as arguments!
actualExpectSin <- function(params){
  stopifnot(params[2] > 0) # second parameter is sigma
  sin(params[1])*exp(-.5*(params[2]^2))
}
monteCarloSin <- function(params){
  stopifnot(params[2] > 0)
  mean(sin(rnorm(n = n, mean = params[1], sd = params[2])))
}
# monteCarloSin(c(10,1))
```

One-off approximations aren't as interesting as visualizing many expectations for many parameter inputs. Below we plot the expectations for many different parameter vectors/configurations/settings (see [Figure 15.2](#)).

```
muGrid <- seq(-10,10, length.out = 100)
sigmaGrid <- seq(.001, 5, length.out = 100)
muSigmaGrid <- expand.grid(muGrid, sigmaGrid)
actuals <- matrix(apply(muSigmaGrid, 1, actualExpectSin),
                  ncol = length(muGrid))
mcApprox <- matrix(apply(muSigmaGrid, 1, monteCarloSin),
                  ncol = length(muGrid))

par(mfrow=c(1,2))
contour(muGrid, sigmaGrid, actuals,
        xlab = "mu", ylab = "sigma", main = "actual expects")
contour(muGrid, sigmaGrid, mcApprox,
        xlab = "mu", ylab = "sigma", main = "mc without crn")
```

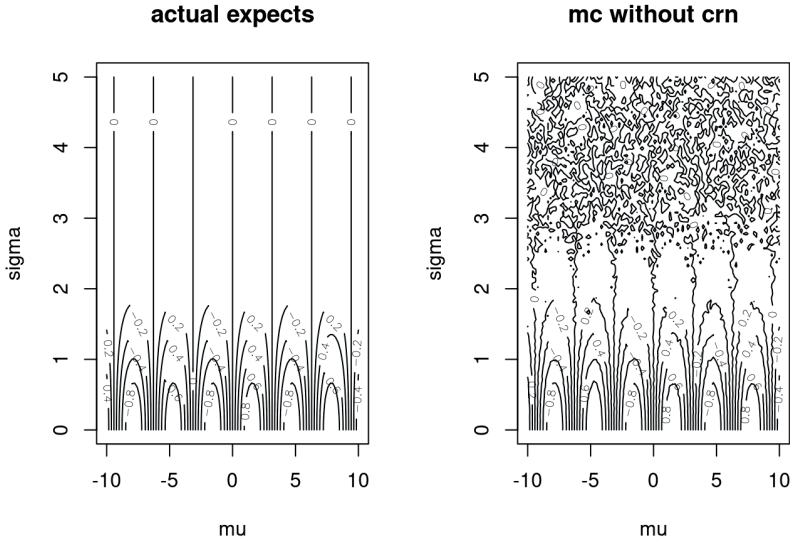


FIGURE 15.2: Monte Carlo approximations versus exact evaluations.

There are three problems with this implementation:

- `monteCarloSin()` is not pure because it captures the `n` variable,
- the only way to increase the accuracy of the plot in the right panel is to increase `n`, and
- every time we re-run this code the plot on the right looks different.

If we wanted to use common random numbers, we could generate $Z^1, \dots, Z^n \stackrel{\text{iid}}{\sim} \text{Normal}(0, 1)$, and use the fact that

$$X^i = \mu + \sigma Z^i \quad (15.3)$$

This leads to the Monte Carlo estimate

$$\tilde{\mathbb{E}}[\sin(X)] = \frac{1}{n} \sum_{i=1}^n \sin(\mu + \sigma Z^i) \quad (15.4)$$

Here is one function that naively implements Monte Carlo with common random numbers. We generate the collection of standard normal random variables once, globally. Each time you call `monteCarloSinCRNv1(c(10,1))`, you get the same answer.

```

commonZs <- rnorm(n=n)
monteCarloSinCRNv1 <- function(params){
  stopifnot(params[2] > 0)
  mean(sin(params[1] + params[2]*commonZs))
}
# monteCarloSinCRNv1(c(10,1))

```

Let's compare using common random numbers to going without. As you can see in [Figure 15.3](#), common random numbers make the plot look “smoother.” In other words, we increase our sampling accuracy without spending more computational time.

```

mcApproxCRNv1 <- matrix(apply(muSigmaGrid, 1, monteCarloSinCRNv1),
                          ncol = length(muGrid))
par(mfrow=c(1,2))
contour(muGrid, sigmaGrid, mcApprox,
        xlab = "mu", ylab = "sigma", main = "mc without crn")
contour(muGrid, sigmaGrid, mcApproxCRNv1,
        xlab = "mu", ylab = "sigma", main = "mc with crn")

```

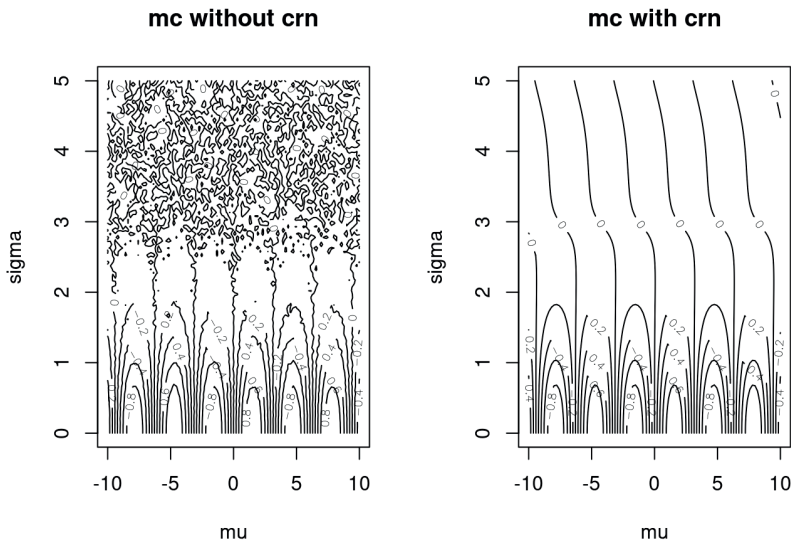


FIGURE 15.3: Monte Carlo: With and without common random numbers.

```

par(mfrow=c(1,1))

```

There are some new downsides to this implementation to consider:

- we have another global variable—a bunch of samples called `commonZs` floating around, and
- the dependence on the global variable for sample size is even further obscured.

We can fix these two problems very nicely by using a function factory.

```
makeMCFunc <- function(n = 1000){
  commonZs <- rnorm(n)
  function(params){
    stopifnot(params[2] > 0)
    mean(sin(params[1] + params[2]*commonZs))
  }
}
monteCarloSinCRNv2 <- makeMCFunc()
# now call monteCarloSinCRNv2 to approx. expectations
# e.g. monteCarloSinCRNv2(c(10,1))
```

This is much better because

- the desired sample size must be passed in as a function argument instead of being captured,
- the re-used standard normal variates are not in the global environment anymore, and
- a sensible default number of samples is provided in the event that the programmer forgets to specify one.

The inner function did in fact capture `commonZs`, but it captured from the enclosing scope, not the global scope. Capturing isn't always a terrible idea. It would be difficult to modify these variables, so we don't need to worry about function behavior changing in unpredictable ways. Actually capturing a variable instead of passing it in is an intelligent design choice—now the end-users of functions created by this factory don't need to worry about plugging in extra parameters.

Let's use 1000 samples again and make sure this function works by comparing its output to the known true function. Run the following code on your own machine. Note the new Greek letters in the axis labels.

```
mcApproxCRNv2 <- matrix(apply(muSigmaGrid, 1, monteCarloSinCRNv2),
                        ncol = length(muGrid))
par(mfrow=c(1,2))
contour(muGrid, sigmaGrid, mcApprox,
```

```

        xlab = expression(mu), ylab = expression(sigma))
contour(muGrid, sigmaGrid, mcApproxCRNv2,
        xlab = expression(mu), ylab = expression(sigma))
par(mfrow=c(1,1))

```

15.4 Functions as Function Outputs in Python

We can write function factories in Python, too. Here is another implementation of the first example from the previous section. Again, just as it did in R, `str` passed in as `greeting_message` persists well after `func_factory()` is finished working.

```

def func_factory(greeting_message):
    def func(name):
        print(greeting_message + ' ' + name)
    return func
greet_with_hello = func_factory("Hello")
greet_with_hello("Taylor")
## Hello Taylor
greet_with_hello("Charlie")
## Hello Charlie

```

Let's consider another less trivial example. Recall the spline function from earlier in the chapter:

```

import numpy as np
def spline(x):
    knot = 3.0
    if x >= knot:
        return x-knot
    else:
        return 0.0

```

This function is limited in that it takes in only one element at a time. Unfortunately, we would not be able to provide an entire Numpy array as an argument (e.g. `spline(np.arange(3))`). Many functions do possess this behavior, and it is generally advantageous to take advantage of it. If you recall our discussion about universal functions in [section 3.4](#), you might have grown accustomed to taking advantage of writing vectorized code.

Fortunately there's a way to automatically vectorize functions like the one above: `np.vectorize()`¹⁹. `np.vectorize()` takes in a unary function, and outputs a vectorized version of it that is able to take entire arrays as an input. Here's an example. Compare this to us using `map()` before.

```
my_inputs = np.linspace(start = 0, stop = 2*np.pi)
# make a vectorized function
vec_spline = np.vectorize(spline)
vec_spline(my_inputs)[:4]
# alternatively output = list(map(spline, my_inputs)) from last time
## array([0., 0., 0., 0.]
```

The above code doesn't just demonstrate how to return functions from a function. It is also an example of using functions as function inputs. When a function takes in and spits out functions, there is an alternative way to use it that is unique to Python. You can use **function decorators**²⁰. You can *decorate* a function by using the `@` operator (Lutz, 2013).

If you decorate a function, it is equivalent to passing that function in to a function factory (aka outer function). That function will take the function you defined, alter it, and then give it back to you with the same name that you chose in the first place.

```
# instead of spline = np.vectorize(spline)
@np.vectorize
def spline(x):
    knot = 3.0
    if x >= knot:
        return x-knot
    else:
        return 0.0
spline(my_inputs)[:4]
## array([0., 0., 0., 0.]
```

15.4.1 Writing Our Own Decorators

We can write our own function factory that can be used as decoration. The main restriction is that this function factory must take a function as an argument, too. This can sometimes be restrictive. You might have noticed that the definition of `func_factory()` from earlier in this section did not do that. If you don't believe me, as an exercise, after you read this section, you might consider trying to rewrite the

¹⁹<https://numpy.org/doc/stable/reference/generated/numpy.vectorize.html#numpy.vectorize>

²⁰<https://www.python.org/dev/peps/pep-0318/>

example from [Section 15.3](#) that implements Monte Carlo sampling using common random numbers.

Before we get too ahead of ourselves, let's describe the basics. Here is our first decorator function `add_greeting()`.

```
def add_greeting(func):  
    def wrapper(name):  
        print('Salutations, ')  
        func(name)  
    return wrapper
```

The decorator `add_greeting()` returns a function that is an embellished version of the function it is given. When we decorate a function with it, it looks like this.

```
@add_greeting  
def print_name(first_name):  
    print(first_name)
```

You could get the same behavior by typing the following. They are equivalent!

```
def print_name(first_name):  
    print(first_name)  
print_name = add_greeting(print_name)
```

Things can get a little more complicated when your decorators take additional arguments.

```
@add_greeting("How you doin'")  
def print_name(first_name):  
    print(first_name)  
print_name('Taylor')  
## How you doin'  
## Taylor
```

So how do we write decorators that accomplish this? The important thing to remember is that `@add_greeting("How you doin'")` in the previous code block is equivalent to writing this after the function definition: `print_name = add_greeting("How you doin'")(print_name)`. This is a function returning a function returning a function! The definition of `add_greeting()` could look something like this.

```
def add_greeting(greet):  
    def decorator(func):
```



```
def wrapper(name):
    print(greet)
    func(name)
    return wrapper
return decorator
```

Now that you know how decorators work, you can feel comfortable using third-party ones. You might come across, for example, the `@jit` decorator from Numba²¹, which will translate your Python function into faster machine code, the `@lru_cache` decorator from the `functools` module²²—this can make your code faster by saving some of its outputs—or decorators that perform application specific tasks like `@tf.function`²³ from Tensorflow.

15.5 Exercises

15.5.1 Python Questions

1.

Write a function decorator called `@log_dens_eval(left_bound, right_bound)`. When it decorates a function, say `func(x)`, it will not change that function's input or output, but it will verify that the input to the function (in this case `x`) is between `left_bound` and `right_bound`. If it is not, it will return negative infinity.

2.

The Split-Apply-Combine strategy might be useful in writing code for a tree-based model (Breiman et al., 1984). We won't discuss how these models are estimated, but we will write a function that generates another function that is able to generate predictions by stratifying the input/predictor/feature space.

- a) Import the data "winequality-red.csv", call it `wine`, and remove all columns except for `fixed acidity`, `volatile acidity`, and `quality`.
- b) Write a function called `generate_pred_func(fixed_cutoff, vol_cutoff, dataset)`.
 - The `dataset` argument should be a Pandas `DataFrame` that has three columns called `fixed acidity`, `volatile acidity`, and `quality`.
 - The `fixed_cutoff` argument should be a floating point number that separates `fixed acidity` into two regions.

²¹<https://numba.pydata.org/>

²²<https://docs.python.org/3/library/functools.html>

²³https://www.tensorflow.org/api_docs/python/tf/function

- The `vol_cutoff` argument should be a floating point number that separates `volatile acidity` into two regions.
- The function should return a function, say `func(fixed_acidity, volatile_acidity)`. The two arguments are floating points. This function should return the most frequent `quality` observation out of all points whose inputs lie in the corresponding region in the feature space.

After you finish the problem, you should have a definition of a `generate_pred_func()` that could be used as follows:

```
predict = generate_pred_func(fixed_cutoff=8, vol_cutoff=.5, dataset=wine)
predict(10,10)
```

3.

Let's predict what type of activity some is doing based on measurements taken from their cell phone. We will begin implementing a **K-Nearest Neighbors (KNN) classifier** (Fix and Hodges, 1989) (Cover and Hart, 1967).

Consider the data files "`x_train.txt`" and "`y_train.txt`" from (Anguita et al., 2013), which is available from the UCI Machine Learning Repository (Dua and Graff, 2017). The first data set consists of recorded movements from a cell phone, and the second data set consists of activity labels of people. Labels 1 through 6 correspond to walking, walking upstairs, walking downstairs, sitting, standing and laying, respectively.

- a) Read in "`x_train.txt`" as a `DataFrame` called `x_train`.
- b) Read in "`y_train.txt`" as a `DataFrame` called `y_train`
- c) Define a function called `standardize(arr)` that takes in an array-like and returns a standardized version of the array-like. Do this by subtracting from each element the overall mean and dividing each element by the standard deviation (use the length as a denominator, not length minus 1).
- d) Apply `standardize()` to each column and transform `x_train` by replacing all of its column by their standardized versions. Make sure to overwrite `x_train`. Do this in one line of code.
- e) Write a function called `euclid_dist(arr1, arr2)` that calculates Euclidean distance between two points/array-likes.
- f) What is the most common label among the 5 rows that are closest to the first row? Assign your answer to `my_predict`. Assume that the two data sets you imported have the same order. Don't include the first row in these 5 *nearest neighbors*. Take care not to modify `x_train` or `y_train`.

15.5.2 R Questions

1.

The density of a particular bivariate Gaussian distribution is

$$f(x, y) = \frac{1}{2\pi} \exp \left[-\frac{x^2 + y^2}{2} \right]. \quad (1)$$

The random elements X and Y , in this particular case, are independent, each have unit variance, and zero mean. In this case, the marginal for X is a mean 0, unit variance normal distribution:

$$g(x) = \frac{1}{\sqrt{2\pi}} \exp \left[-\frac{x^2}{2} \right]. \quad (2)$$

- a) Write a function called `fTwoArgs(x,y)` that takes two arguments, and returns the value of the above density in equation (1) at those two points.
- b) Write a function called `fOneArg(vec)` that takes one argument: a length two `vector`. It should return a density in equation (1) evaluated at that point.
- c) Write a function called `gOneArg(x)` that evaluates the density in (2).
- d) Generate two sequences called `xPoints` and `yPoints`. Have them contain the twenty equally-spaced numbers going from -3 to 3 , inclusive.
- e) Use `expand.grid()` to create a `data.frame` called `myGrid`. It should have two columns, and it should contain in its rows every possible pair of two points from the above sequences. The “x” coordinates should be in the first column.
- f) Use `mapply()` to evaluate the bivariate density on every grid point. Store your results in a `vector` `mEvals`.
- g) Use `apply()` to evaluate the bivariate density on every grid point. Store your results in a `vector` `aEvals`.
- h) Use `sapply()` to evaluate the univariate density on every element of `xPoints`. Store your results in a `vector` `sEvals`.
- i) Use `vapply` to evaluate the univariate density on every element of `xPoints`. Store your results in `vector` `vEvals`.
- j) Use `lapply` to evaluate the univariate density on every element of `xPoints`. Store your results in a `list` `lEvals`.
- k) Generate two plots of the bivariate density. For one, use `persp()`. For the other, use `contour()`. Feel free to revive the code you used in [Chapter 13](#)’s Exercise 1.
- l) Generate a third plot of the univariate density. Feel free to revive the code you used in [Chapter 13](#)’s Exercises.

2.

Write a function that reads in all of the data sets contained in any given folder.

- The function should be called `readAllData(path, extensions)`.
- The first argument, `path`, should be a string representing which folder you would like to search for files.
- The second argument, `extensions`, should be a character vector of all the file extensions you are interested in reading in (e.g. `c('.txt', '.csv')`). Be careful about using regular expressions!
- The function should return a list of `data.frames`.

3.

Consider the Militarized Interstate Disputes (v5.0) (Palmer et al., 0) data sets again: `"MIDA 5.0.csv"`, `"MIDB 5.0.csv"`, `"MIDI 5.0.csv"`, and `"MIDIP 5.0.csv"`.

- a) Read these data sets in as a list of `data.frames`. Call it `dfList`.
- b) Use `lapply()` and `do.call()` to calculate the biggest column average for all of these data sets. Store this average as `biggestAve`. Store the name of the column that had this biggest average as `whichBiggestAve`. Don't worry about storing which data set this column mean was found in.
- c) Use `Reduce()` and `lapply()` to, once again, calculate the biggest column average. Store this number as `biggestAve2`



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Bibliography

- (1989). Chess (King-Rook vs. King-Pawn). UCI Machine Learning Repository.
- (1996). Adult. UCI Machine Learning Repository.
- (1997). Car Evaluation. UCI Machine Learning Repository.
- (2021). Gspc data. <https://finance.yahoo.com/quote/%5EGSPC/history?p=%5EGSPC>. Accessed: 2021-10-03.
- (2021). Sas[®] viya[®] example data sets. <https://support.sas.com/documentation/onlinedoc/viya/examples.htm>. Accessed: 2021-12-12.
- (2021). TensorFlow Datasets, a collection of ready-to-use datasets. <https://www.tensorflow.org/datasets>.
- Abelson, H. and Sussman, G. J. (1996). *Structure and Interpretation of Computer Programs*. MIT Press, Cambridge, MA, USA, 2nd edition.
- Albemarle County Geographic Data Services Office (2021). Albemarle County GIS Web. Accessed on 12.07.2021.
- Albon, C. (2018). *Machine Learning with Python Cookbook: Practical Solutions from Preprocessing to Deep Learning*. O'Reilly Media, Inc., 1st edition.
- Anguita, D., Ghio, A., Oneto, L., Parra, X., and Reyes-Ortiz, J. L. (2013). A public domain dataset for human activity recognition using smartphones. In *ESANN*.
- Beazley, D. M. and Jones, B. K. B. K. (2014). *Python Cookbook: Recipes for Mastering Python 3*. Third edition.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth and Brooks, Monterey, CA.
- Brown, T. (2021). *cPseudoMaRg: Constructs a Correlated Pseudo-Marginal Sampler*. R package version 1.0.0.
- Carvalho, C. M., Polson, N. G., and Scott, J. G. (2009). Handling sparsity via the horseshoe. In van Dyk, D. and Welling, M., editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 73–80, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA. PMLR.
- Chang, W. (2013). *R Graphics Cookbook*. O'Reilly Media, Inc.

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., and Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decis. Support Syst.*, 47(4):547–553.
- Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.
- Dua, D. and Graff, C. (2017). UCI machine learning repository.
- Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1):1–26.
- Fisher, R.A. & Creator, T. (1988). Iris. UCI Machine Learning Repository.
- Fix, E. and Hodges, J. L. (1989). Discriminatory analysis. nonparametric discrimination: Consistency properties. *International Statistical Review / Revue Internationale de Statistique*, 57(3):238–247.
- Ford, C. (2016). ggplot: Files for UVA StatLab workshop, Fall 2016. <https://github.com/clayford/ggplot2>.
- Gelman, A. and Hill, J. (2007). *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Analytical methods for social research. Cambridge University Press.
- Grolemund, G. (2014). *Hands-On Programming with R: Write Your Own Functions and Simulations*. O’Reilly Media.
- Guttman, L. (1946). Enlargement Methods for Computing the Inverse Matrix. *The Annals of Mathematical Statistics*, 17(3):336–343.
- Harrell Jr, F. E., with contributions from Charles Dupont, and many others. (2021). *Hmisc: Harrell Miscellaneous*. R Package Version 4.5-0.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.
- Hunt, A. and Thomas, D. (2000). *The Pragmatic Programmer : From Journey-Man to Master*. Addison-Wesley, Boston [etc.].
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95.
- Janosi, A., Steinbrunn, W., Pfisterer, M., and Detrano, R. (1988). Heart Disease. UCI Machine Learning Repository.
- Jones, K. S. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28:11–21.

- Kabacoff, R. I. (2015). *R in Action*. Manning, 2nd edition.
- Kahn, H. (1950a). Random sampling (monte carlo) techniques in neutron attenuation problems–i. *Nucleonics*, 6 5.
- Kahn, H. (1950b). Random sampling (monte carlo) techniques in neutron attenuation problems. ii. *Nucleonics (U.S.) Ceased publication*, Vol: 6, No. 6.
- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1):35–45.
- Lander, J. P. (2017). *R for Everyone: Advanced Analytics and Graphics (2nd Edition)*. Addison-Wesley Professional, 2nd edition.
- López, F. (2014). *Mastering Python regular expressions : leverage regular expressions in Python even for the most complex features*. Packt Pub, Birmingham, UK.
- Lutz, M. (2013). *Learning Python*. O'Reilly, Beijing, 5th edition.
- Matloff, N. (2011). *The Art of R Programming: A Tour of Statistical Software Design*. No Starch Press, USA, 1st edition.
- McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media, Inc., 2nd edition.
- Nadaraya, E. A. (1964). On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142.
- Palmer, G., McManus, R. W., D’Orazio, V., Kenwick, M. R., Karstens, M., Bloch, C., Dietrich, N., Kahn, K., Ritter, K., and Soules, M. J. (0). The mid5 dataset, 2011–2014: Procedures, coding rules, and description. *Conflict Management and Peace Science*, 0(0):0738894221995743.
- Robert, C. P. and Casella, G. (2005). *Monte Carlo Statistical Methods (Springer Texts in Statistics)*. Springer-Verlag, Berlin, Heidelberg.
- Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C., Ng, A., and Potts, C. (2013). Parsing With Compositional Vector Grammars. In *EMNLP*.
- Student (1908). The probable error of a mean. *Biometrika*, pages 1–25.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, Inc., 1st edition.
- Watson, G. S. (1964). Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A (1961-2002)*, 26(4):359–372.
- West, M. A. and Harrison, J. (1989). Bayesian forecasting and dynamic models.
- Wickham, H. (2014). *Advanced R*. Chapman & Hall/CRC The R Series. Taylor & Francis.

- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York.
- Wickham, H. and Grolemund, G. (2017). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media, Inc., 1st edition.
- Wilkinson, L. (2005). *The Grammar of Graphics (Statistics and Computing)*. Springer-Verlag, Berlin, Heidelberg.

Index

- array in R, [48](#)
- assignment operator
 - assignment operator in Python, [8](#)
 - assignment operator in R, [5](#), [65](#), [203](#)
 - super assignment operator in R, [82](#)
- broadcasting, *see* [recycling](#)
- capture groups, [39](#)
- capturing variables, [79](#)
- closures in R, [72](#)
- console
 - console in RStudio, [3](#)
 - console in Spyder, [7](#)
- copy
 - deep copy, [29](#)
 - shallow copy, [28](#)
 - view in Python, [27](#)
- data frame, [97](#)
 - data frames in Python, [100](#)
 - data frames in R, [98](#)
- dicts, [59](#)
- DRY code, *see* [WET code](#)
- dynamic lookup, [79](#), [214](#)
- environments in R, [68](#), [128](#)
- escape character, [10](#), [34](#), [35](#)
- escape sequence, [35](#)
- factors in R, [87](#)
- for loops, [138](#)
- functions, [65](#)
 - creating functions in Python, [66](#)
 - creating functions in R, [65](#)
 - first-class functions, [213](#)
 - function decorators, [234](#)
 - function factory, [72](#)
 - functional programming, [213](#)
 - pure functions, [213](#)
- ggplot2, [170](#)
- if statement
 - if statement in Python, [137](#)
 - if statement in R, [137](#)
- indexing
 - indexing Numpy arrays, [24](#)
 - indexing Pandas data frames, [101](#)
 - indexing Pandas series, [25](#)
 - indexing vectors in R, [24](#)
- integrated development (IDE)
 - RStudio, [3](#)
- integrated development environment
 - (IDE), [3](#)
 - Spyder, [3](#)
- join, *see* [merge](#)
- lists
 - lists in Python, [60](#)
 - lists in R, [59](#)
- Matplotlib, [175](#)
- matrix in R, [48](#)
- merge
 - merging in Python, [155](#)
 - merging in R, [153](#)
- missing data, [30](#)
- modules in Python, [131](#)
- mutable in Python, [77](#), [214](#)
- namespace in Python, [131](#), [132](#)
- Numpy arrays in Python, [18](#), [47](#)
- object-oriented programming, [185](#)

- attributes in Python, *see* data member
- attributes in R, 196
- class keyword in Python, 186
- classes in Python, 186
- composition, 185
- encapsulation, 186
- generic functions in R, 195
- implementation, 186
- inheritance, 185
- instance, 186
- interface, 186
- method dispatch in R, 196
- methods in Python, 187
- methods in R, 194, 196
- modularity, 186
- polymorphism, 185
- R6 classes in R, 205
- Reference Classes in R, 204
- S3 classes in R, 194
- S4 classes in R, 200
- packages
 - packages in Python, 127, 131
 - packages in R, 127
- Pandas, 19, 60, 62, 90, 100, 119, 150, 179, 225
- Pandas series, 19
- pass-by-assignment, 76, 214
- pass-by-value, 76, 214
- pickling in Python, 124
- recycling, *see* broadcasting
- regular expressions, 33
- scope
 - function scope in Python, 73
 - function scope in R, 70
 - LEGB in Python, 73
- script, 4
 - script in Python, 7
 - scripts in R, 4
- search path
 - search path in Python, 131
 - search path in R, 129
- side effects, 82
- slicing, 103
- tuples in Python, 61, 177
- vectorization, 19
 - universal functions, 20
- vectors in R, 14, 17
- WET code, *see* DRY code
- working directory, 5
 - working directory in Python, 8
 - working directory in R, 5